

The micrOMEGAs user's manual, version 7.1

G. Alguero⁴, D. Barducci¹, G. Bélanger², A. Belyaev¹⁰, F. Boudjema²,
S.Chakraborti⁹, J. Da Silva², A. Goudelis³, S. Kraml⁴, U. Laa⁵,
A. Mjallal², A. Pukhov⁶, A. Semenov⁷, B. Zaldivar⁸.

- 1) *Università degli Studi di Roma la Sapienza and INFN Section of Roma 1, Piazzale Aldo Moro 5, 00185, Roma, Italy*
- 2) *LAPTh, CNRS, USMB, 9 Chemin de Bellevue, F-74940 Annecy, France*
- 3) *LPC, CNRS/IN2P3, Univ. Clermont- Auvergne, 4 Av. Blaise Pascal, F-63178 Aubière Cedex, France*
- 4) *Laboratoire de Physique Subatomique et de Cosmologie, Université Grenoble-Alpes, CNRS/IN2P3, 53 Avenue des Martyrs, F-38026 Grenoble, France*
- 5) *Institute of Statistics, BOKU, Peter-Jordan-Straße 82, 1190 Wien, Austria*
- 6) *Skobeltsyn Inst. of Nuclear Physics, Moscow State Univ., Moscow 119992, Russia*
- 7) *Joint Institute for Nuclear Research (JINR) 141980, Dubna, Russia*
- 8) *Departamento de Física Teórica, UAM, 28049 Madrid, Spain*
- 9) *IPPP, Department of Physics, Durham University, Durham, DH1 3LE, UK*
- 10) *School of Physics and Astronomy, University of Southampton, Highfield, Southampton SO17 1BJ, UK*

Abstract

We give an up-to-date description of the micrOMEGAs functions. Only the routines which are available for the users are described. Examples on how to use these functions can be found in the sample main programs distributed with the code.

Contents

1	Introduction	3
2	Discrete symmetry in micrOMEGAs	4
3	Downloading and compilation of micrOMEGAs	5
3.1	File structure of micrOMEGAs	5
3.2	Compilation of CalcHEP and micrOMEGAs routines	6
3.3	External packages	7
3.4	Module structure of main programs	8
3.5	Compilation of codes for specific models	9
3.6	Command line parameters of main programs	9
4	Global Parameters and constants	10
5	The Inert Doublet Model : an example of BSM presentation in micrOMEGAs	10
5.1	Theoretical structure of the IDM	11
5.2	Translation into CalcHEP syntax	12

6	Setting of model parameters, spectrum calculation, parameter display	14
7	Thermodynamics of the Universe.	16
7.1	Modification of <code>geff</code> , <code>heff</code>	18
8	Relic density calculation	19
8.1	Switches and auxiliary routines	19
8.2	Temperature interval and Error codes for routines calculating relic density	20
8.3	Calculation of relic density for one-component Dark Matter models	21
8.3.1	darkOmegaExt	24
8.3.2	vSigmaCC	25
8.4	Calculation of relic density for two-component Dark Matter models	26
8.5	Calculation of relic density for N-component DM taking into account coscat- tering processes.	27
8.6	Scenario with late decaying field	30
8.7	Calculation of relic density for freeze-in	32
9	Direct detection	35
9.1	Amplitudes for elastic scattering	35
9.2	Scattering on nuclei	36
9.2.1	Nucleus form factors	37
9.2.2	Velocity distribution	39
9.3	Comparison with Direct Detection experiments	40
9.3.1	Experimental data	40
9.3.2	Recasting the experimental limits with micrOMEGAs	40
9.3.3	Direct detection limit for photon exchange processes.	42
9.3.4	Setting spin-dependent form factors	42
10	Indirect detection	43
10.1	Interpolation and display of spectra	43
10.2	Annihilation spectra	44
10.3	Distribution of Dark Matter in Galaxy	47
10.4	Photon signal	48
10.5	Propagation of charged particles	49
11	Planck CMB constraint	50
12	Photons from Dwarf Spheroidal galaxies	50
13	Neutrino signal from the Sun and the Earth	50
13.1	Comparison with IceCube results	52
14	Relic density of sterile neutrinos.	52
15	Constraints from colliders	53
15.1	The Higgs sector	53
15.1.1	Lilith	53
15.1.2	HiggsBounds and HiggsSignals	54

15.1.3 Automatic generation of interface files	55
15.2 Searches for New particles	56
15.2.1 SModelS	56
15.3 Other constraints	59
16 Additional routines for specific models	59
16.1 The MSSM	60
16.2 The NMSSM	62
16.3 The CPVMSSM	62
17 Tools for model independent analysis	63
18 Squared Matrix Elements, cross sections, decay widths.	65
18.1 Squared matrix elements with CalcHEP.	65
18.2 Testing the content of the code for matrix elements.	66
18.3 Integration of matrix elements.	66
18.4 CalcHEP Monte Carlo session	68
18.5 Decays	69
18.6 Extraction of couplings for vertices.	70
19 Implementation of a new model	71
19.1 Main steps	71
19.2 Tools for model files generation	72
19.3 Automatic width calculation	72
19.4 One-loop scalar vertices	72
19.5 QCD functions	74
19.6 SLHA reader	74
19.6.1 Writing an SLHA input file	77
19.7 Routines for diagonalisation	78
20 Mathematical tools	78
20.1 Integration	78
20.2 Solution of differential equations.	80
20.3 Interpolation	80
20.4 Functions with additional parameters	81
20.5 Plots	81
20.6 Bessel functions	82
20.7 Statistics	82
A An updated routine for $b \rightarrow s\gamma$ in the MSSM	82

1 Introduction

micrOMEGAs is a code to calculate the properties of cold dark matter (CDM) in a generic model of particle physics. First developed to compute the relic density of dark matter, the code also computes the rates for dark matter direct and indirect detection. **micrOMEGAs** computes CDM properties in the framework of a model of particle interactions presented

in **CalcHEP** format [1]. It is assumed that the model is invariant under a discrete symmetry like R-parity (even for all standard particles and odd for some new particles including the dark matter candidate) which ensures the stability of the lightest odd particle. Similarly in multi-component dark matter models, a discrete symmetry that guarantees the stability of the lightest particle in each of the dark matter sectors is assumed. The **CalcHEP** package is included in **micrOMEGAs** and used for matrix elements calculations. All annihilation and coannihilation channels are included in the computation of the relic density. This manual gives an up-to-date description of all **micrOMEGAs** functions. The methods used to compute the different dark matter properties are described in references [2–10]. These references also contain a more complete description of the code. In the following the cold dark matter candidate also called weakly-interactive massive particle (WIMP) will be denoted by χ . Starting with version 5.0, **micrOMEGAs** also allows to compute the abundance of feebly interacting dark matter candidates (FIMP) through the freeze-in mechanism [10].

micrOMEGAs is written in C and uses some Fortran routines mostly in external packages. The complete format for all functions can be found in `include/.h` (for C). Examples on how to use these functions are provided in the `MSSM/main.c` file.

2 Discrete symmetry in **micrOMEGAs**

micrOMEGAs exploits the fact that models of dark matter exhibit a discrete symmetry and that the fields of the model transform as $\phi \rightarrow e^{i2\pi X_\phi} \phi$ where the charge $|X_\phi| < 1$. The particles of the Standard Model are assumed to transform trivially under the discrete symmetry, $X_\phi = 0$. In the following all particles with charge $X_\phi \neq 0$ will be called *odd* and the lightest odd particle will be stable. If neutral, it can be considered as a DM candidate. Typical examples of discrete symmetries used for constructing single DM models are Z_2 and Z_3 . Multi-component DM can arise in models with larger discrete groups. A simple example is a model with $Z_2 \times Z'_2$ symmetry, the particles charged under $Z_2(Z'_2)$ will belong to the first (second) dark sector. The lightest particle of each sector will be stable and therefore a potential DM candidate. Another example is a model with a Z_4 symmetry. The two dark sectors contain particles with $X_\phi = \pm 1/4$ and $X_\phi = 1/2$ respectively. The lightest particle with charge $1/4$ is always stable while the lightest particle of charge $1/2$ is stable only if its decay into two particles of charge $1/4$ is kinematically forbidden. **micrOMEGAs** assumes that all odd particles have names starting with '~', for example, ~o1 for the lightest neutralino. To distinguish the particles with different transformation properties with respect to the discrete group, that is particles belonging to different 'dark' sectors, we use the number of '~' at the beginning of the name of the particles. The maximal number of sectors is limited by the maximal length of the particle names (11 in the current version). For the relic density calculation we also assume that all particles in a given sector are in thermal equilibrium with each other. This last assumption is not necessarily valid, therefore the user has the possibility to split default sectors in subsectors where thermal equilibrium is maintained. See Section 8.5 for details.

Note that **micrOMEGAs** does not check the symmetry of the Lagrangian, it assumes that the name convention correctly identifies all particles with the same discrete symmetry quantum numbers. For models with FIMPs, new particles are considered to be in thermal

equilibrium with the SM bath ($\mathcal{B}$) at high temperatures unless explicitly defined as being feeble, ie belonging to $\mathcal{F}$. Both $\mathcal{B}$ and $\mathcal{F}$ can contain odd or even particles, see section 8.7.

3 Downloading and compilation of micrOMEGAs

To download micrOMEGAs, go to

<https://zenodo.org/records/17597395>

and unpack the file received, `micromegas_7.1.tgz`, with the command

```
tar -xvzf micromegas_7.1.tgz
```

This should create the directory `micromegas_7.1/` which occupies about 100Mb of disk space. You will need more disk space after compilation of specific models and generation of matrix elements. More information can also be found at

<http://lapth.cnrs.fr/micromegas>

In case of problems and questions

email: `micromegas@lapth.cnrs.fr`

3.1 File structure of micrOMEGAs

<code>calc</code>	calculator
<code>calchep.ini</code>	specify the fonts for graphics in CalcHEP
<code>Makefile</code>	to compile the kernel of the package
<code>CalcHEP_src/</code>	generator of matrix elements for micrOMEGAs
<code>Packages/</code>	external codes
<code>clean</code>	to remove compiled files
<code>fileMap.txt</code>	contains the list of all files in the code
<code>history</code>	contains a list of changes in recent versions
<code>man/</code>	contains the manual: description of micrOMEGAs routines
<code>newProject</code>	to create a new model directory structure
<code>sources/</code>	micrOMEGAs code
<code>include/</code>	include files for micrOMEGAs routines or external codes
<code>lib/</code>	contains library <code>micromegas.a</code> when micrOMEGAs is compiled
<i>MSSM model directory</i>	
<code>MSSM/</code>	
<code>Makefile</code>	to compile the code and executable for this model
<code>main.c[pp]</code>	files with sample <i>main</i> programs
<code>README</code>	Brief description on how to use the code
<code>mssmX.par</code>	Sample input files
<code>lib/</code>	directory for routines specific to this model
<code>Makefile</code>	to compile the auxiliary code library <i>lib/aLib.a</i>
<code>*.c *.f *.h</code>	source codes of auxiliary functions
<code>work/</code>	CalcHEP working directory for the generation of matrix elements
<code>Makefile</code>	to compile the library <i>work/work_aux.a</i>
<code>calchep</code>	Executable for CalcHEP

lanhep/	directory containing lanhep source model files
models/	directory for files which specifies the model
	files *1.mdl are used in micrOMEGAs sessions. Other *.mdl files are intended for CalcHEP interactive sessions
vars1.mdl	free variables
func1.mdl	constrained variables
prtcls1.mdl	particles
lgrng1.mdl	Feynman rules
results/	
so_generated/	storage of matrix elements generated by CalcHEP
<i>Directories of other models which have the same structure as MSSM/</i>	
NMSSM/	Next-to-Minimal Supersymmetric Model [11, 12]
CPVMSSM/	MSSM with complex parameters [13, 14]
IDM/	Inert Doublet Model [15]
LLL_scalar/	Simplified model with singlet charged lepton and real scalar DM
LHM/	Little Higgs Model [16]
RDM/	Scalar Leptoquark and two singlet fermions [17]
SingletDM/	Singlet scalar DM model with Z_2 symmetry [18]
STFM/	Singlet-triplet fermionic model [19]
Z3IDM/	Inert doublet model with Z_3 discrete symmetry [20, 21]
Z4IDSM/	Inert doublet and singlet model with Z_4 symmetry [20, 21]
Z5M/	Two scalar singlets model with Z_5 symmetry [22]
ZpPortal/	Simplified model with a Z' portal and fermion DM
mdlIndep/	For model independent computation of DM signals

Other models can be downloaded on the web, <http://lapth.cnrs.fr/micromegas>, for example : RHN, a right-handed Neutrino Model [23], SM4, a toy model with a 4th generation of leptons and neutrino DM, UMSSM, an $U(1)$ extension of the MSSM [24, 25].

3.2 Compilation of CalcHEP and micrOMEGAs routines

The graphical interface of CalcHEP and micrOMEGAs uses X11 routines. Therefore X11 header files

```
/use/include/X11/*.h
```

are needed to compile these codes. If you do not have these files on your computer, you have to install the *X11-devel* package. Its name depends on the operating system, namely,

libX11-devel	for Fedora/Scientific, old Darwin(Mac)
Xquartz (https://www.xquartz.org)	new Mac
libX11-dev	for Ubuntu/Debian [old Ubuntu]
libx11-dev	for Ubuntu/Debian [new Ubuntu]
xorg-x11-devel	for SUSE

CalcHEP and micrOMEGAs are compiled by *gmake*. Go to the micrOMEGAs directory and launch

```
gmake
```

If `gmake` is not available, then `make` should work like `gmake`. In principle `micrOMEGAs` defines automatically the names of *C* and *Fortran* compilers and the flags for compilation. If you meet a problem, open the file which contains the compiler specifications, `CalcHEP_src/FlagsForSh`, improve it, and launch `[g]make` again. The file is written in `sh` script format and looks like

```
# C compiler
CC="gcc"
# Flags for C compiler
CFLAGS="-g -fsigned-char"
# Disposition of header files for X11
HX11=
# Disposition of lX11
LX11="-lX11"
# Fortran compiler
FC="gfortran"
FFLAGS="-fno-automatic"
.....
```

After a successful definition of compilers and their flags, `micrOMEGAs` rewrites the file *FlagsForSh* into *FlagsForMake* and substitutes its contents in all *Makefiles* of the package.

`[g]make clean` deletes all generated files, but asks permission to delete *FlagsForSh*.
`[g]make flags` only generates *FlagsForSh*. It allows to check and change flags before compiling the codes.

3.3 External packages

`micrOMEGAs` is interfaced with a number of external packages, some of which are directly included in the `micrOMEGAs` distributions, those are stored in the directory `/Packages`, while others are downloaded automatically upon request.

The codes included in the `micrOMEGAs` distribution are:

Suspect2.41 [26]	spectrum calculator for MSSM
NMSSMTools5.6.0 [27, 28]	spectrum calculator for NMSSM
CPsuperH2.3 [13, 29]	spectrum calculator for the MSSM and CPVMSSM
LoopTools [30]	for computing some loop-induced processes
lf2c	auxilliary library for Fortran-C converter
LanHEP [31]	for generating model files
maxGap [32]	for recasting experimental limits using the Optimal Intervals method
Lilith-2.1 [33–35]	for checking Higgs signal strengths
FermiDwarfs [36–38]	To determine the limit from the FermiLAT experiment on photons from Dwarf Spheroidal galaxies

The packages that are downloaded automatically during runtime are:

SOFTSUSY [39]	SUSY spectrum generator
SPheno [40]	SUSY spectrum generator
SModelS [41–45]	for simplified-model constraints from LHC searches
superIso [46]	for flavor constraints
HiggsBounds [47, 48]	for checking Higgs-sector constraints
HiggsSignals [49, 50]	

The versions of the codes to be downloaded can be defined by the user via the parameter `VERSION` in the corresponding files:

```
MSSM/lib/ssusy_call.c    include/SMODELS.inc    include/hBandS.inc
MSSM/lib/spheno_call.c  sources/superIso.c
```

The URL for the source code and compilation instructions are presented in *makefiles*

```
SSUSY.makef      SModelS.makef    HBOUNDS.makef
SPHENO.makef     SUPERISO.makef  H SIGNALS.makef
```

Note however, that care must be taken that new versions remain compatible with the existing interface structure. We also point out that `Lilith` `SModelS` and `FermiDwarfs` are python packages; for the latest versions of these codes, a python3 installation is required in addition to the compilers mentioned above.

3.4 Module structure of main programs

Each model included in `micrOMEGAs` is accompanied with sample files for C programs which call `micrOMEGAs` routines, the *main.c* files. These files consist of several modules enclosed between the instructions

```
#ifdef XXXXX
.....
#endif
```

Each of these blocks contains some code for a specific problem

```
#define MASSES_INFO           //Displays information about mass spectrum
#define CONSTRAINTS           //Displays B->sgamma, Bs->mumu, etc
#define HIGGSBOUNDS           //Calls HiggsBounds to constrain the Higgs sector
#define HIGGSIGNALS           //Calls HiggsSignal to constrain the Higgs boson
#define SUPERISO              //calls SuperISO to compute flavour observables
#define LILITH                 //Calls LiLith to constrain the Higgs sector
#define SModelS               //Calls SModelS to constrain the new physics sector
#define OMEGA                  //Calculates the relic density
#define FREEZEIN              //Calculates the relic density in the freeze-in mechanism
#define INDIRECT_DETECTION     //Signals of DM annihilation in galactic halo
#define LoopGAMMA              //Gamma-Ray lines - available only in some models
#define RESET_FORMFACTORS      //Redefinition of Form Factors and other
                              //parameters
#define CDM_NUCLEON           //Calculates amplitudes and cross-sections
                              //for DM-nucleon collisions
```

```

#define CDM_NUCLEUS          //Calculates number of events for 1kg*day,
                             //recoil energy distribution for various nuclei
                             //and compares with experimental data.
#define NEUTRINO             //Calculates flux of solar neutrinos and
                             //the corresponding muon flux
#define DECAYS               //Calculates decay widths and branching ratios
#define CROSS_SECTIONS      //Calculates cross sections
#define CLEAN                //Removes intermediate files.

```

The flag

```

#define SHOWPLOTS           //switches on graphic facilities of micrOMEGAs.

```

All these modules are completely independent. The user can comment or uncomment any set of *define* instructions to suit his/her need.

3.5 Compilation of codes for specific models

After the compilation of micrOMEGAs one has to compile the executable to compute DM related observables in a specific model. To do this, go to the model directory, say MSSM, and launch

```
[g]make
```

This should generate the executable `main` using the `main.c` source file. In general

```
gmake main=filename.ext
```

generates the executable `filename` based on the source file *filename.ext*. For *ext* we support 2 options: `'c'`, `'cpp'` which correspond to C and C++ sources. `[g]make` called in the model directory automatically launches `[g]make` in the subdirectories `lib` and `work` to compile

`lib/aLib.a` – the library of auxiliary model functions, and

`work/work_aux.a` – the library of model particles, free and dependent parameters.

3.6 Command line parameters of main programs

The default versions of *main.c* programs need some arguments which have to be specified in command lines. If launched without arguments *main* explains which parameter are needed. As a rule *main* needs the name of a file containing the numerical values of the free parameters of the model. The structure of a file record should be

```
Name      Value # comment ( optional)
```

For instance, an Inert Doublet model (IDM) input file contains

```

Mh      125    # mass of SM Higgs
MHC     200    # mass of charged Higgs ~H+
MH3     200    # mass of odd Higgs ~H3
MHX     63.2   # mass of ~X particle

```

```

la2  0.01   # \lambda_2  coupling
laL  0.01   # 0.5*(\lambda_3+\lambda_4+\lambda_5)

```

In other cases, different inputs can be required. For example, in the MSSM with input parameters defined at the GUT scale, the parameters have to be provided in a command line. Launching `./main` will return

```

This program needs 4 parameters:
m0      common scalar mass at GUT scale
mhf     common gaugino mass at GUT scale
a0      trilinear soft breaking parameter at GUT scale
tb      tan(beta)
Auxiliary parameters are:
sgn +/-1, sign of Higgsino mass term (default 1)
Mtp     top quark pole mass
MbMb    Mb(Mb) scale independent b-quark mass
alfSMZ  strong coupling at MZ
Example: ./main 120 500 -350 10 1 173.1

```

4 Global Parameters and constants

The list of the global parameters and their default values are given in Tables 1 and 2. The numerical value for any of these parameters can be simply reset anywhere in the code. The numerical values of the scalar quark form factors can also be reset by the `calcScalarQuarkFF` routine presented below. Some physical values evaluated by `micrOMEGAs` also are presented as global variables, see Table 3.

All physical constants used in relic density calculations are defined in the file `include/micromegas_aux.h`, they are listed in Table 4.

5 The Inert Doublet Model : an example of BSM presentation in micrOMEGAs

The predictive power of `micrOMEGAs` rests on its tight coupling with the matrix-element generator `CalcHEP` [51]. Every physical quantity computed by `micrOMEGAs` – from decay widths to relic density integrals – ultimately originates from symbolic expressions generated in `CalcHEP`. For this reason, any extension of the Standard Model must first be expressed in the `CalcHEP` model format before it can be used in numerical studies.

A minimal model definition consists of four interrelated files,

```
vars1.mdl, func1.mdl, prtcls1.mdl, lgrng1.mdl,
```

which together encode the entire theoretical structure: the list of independent parameters, the relations among derived quantities, the spectrum of particles, and the set of interaction vertices. Although the syntax is deliberately simple, the logical separation between these files reflects the conceptual hierarchy of a field theory: input parameters determine all derived quantities, which in turn fix the particle properties and the Feynman rules that govern dynamics.

Table 1: Global input parameters of **micrOMEGAs**

Name	default value	units	comments
deltaY	0		Difference between DM/anti-DM abundances
K_dif	0.0112	kpc ² /Myr	The normalized diffusion coefficient
L_dif	4	kpc	Vertical size of the Galaxy diffusive halo
Delta_dif	0.7		Slope of the diffusion coefficient
Tau_dif	10 ¹⁶	s	Electron energy loss time
Vc_dif	0	km/s	Convective Galactic wind
Fermi_a	0.52	fm	nuclei surface thickness
Fermi_b	-0.6	fm	parameters to set the nuclei radius with
Fermi_c	1.23	fm	$R_A = cA^{1/3} + b$
Rsun	8.5	kpc	Distance from the Sun to the center of the Galaxy
Rdisk	20	kpc	Radius of the galactic diffusion disk
rhoDM	0.3	GeV/cm ³	Dark Matter density at Rsun
vEarth	232	km/s	Galaxy velocity of the Earth
vRot	220	km/s	Galaxy rotation velocity at Rsun
vEsc	544	km/s	Escape velocity at Rsun
etaSHMpp	0.2		η parameter of SHM++
betaSHMpp	0.9		β parameter of SHM++

 Table 2: Global parameters of **micrOMEGAs**: nucleon quark form factors

Proton		Neutron		comments
Name	value	Name	value	
ScalarFFPd	0.0191	ScalarFFNd	0.0273	Scalar form factor
ScalarFFPu	0.0153	ScalarFFNu	0.011	
ScalarFFPs	0.0447	ScalarFFNs	0.0447	
pVectorFFPd	-0.427	pVectorFFNd	0.842	Axial-vector form factor
pVectorFFPu	0.842	pVectorFFNu	-0.427	
pVectorFFPs	-0.085	pVectorFFNs	-0.085	
SigmaFFPd	-0.23	SigmaFFNd	0.84	Tensor form factor
SigmaFFPu	0.84	SigmaFFNu	-0.23	
SigmaFFPs	-0.046	SigmaFFNs	-0.046	

To illustrate how these ingredients cooperate, we use the *Inert Doublet Model* (IDM) as an example. The IDM provides one of the simplest frameworks in which an exact discrete symmetry stabilises a scalar dark matter particle, yet it retains close contact with Standard Model phenomenology.

5.1 Theoretical structure of the IDM

The model introduces a second $SU(2)$ doublet H_2 in addition to the Standard Model Higgs doublet H_1 . Its Lagrangian is

$$L = (SM \text{ terms}) + D^\mu H_2^* D_\mu H_2 - \mu^2 H_2^2 - \lambda_2 H_2^4 - \lambda_3 H_1^2 H_2^2 - \lambda_4 |H_1^* H_2|^2 - \lambda_5 \text{Re}[(H_1^* H_2)^2], \quad (1)$$

Table 3: Evaluated global variables

Name	units	comments	Evaluated by
Ncdm	integer	number of thermal sectors for DM particles.	sortOddParticles
CDM[k]	<i>character</i>	name of the lightest particles in each sector k=1...Ncdm	sortOddParticles
McdmN[k]	GeV	Mass of CDM[k]	sortOddParticles
Mcdm	GeV	minimal mass of odd particles	sortOddParticles
fracCDM[k]		fraction of CDM[k] in relic density.	darkOmega*
dmAsymm		Asymmetry between relic density of DM - $\overline{DM}$	darkOmega[FO]
Tstart, Tend	GeV	Temperature interval for solving the differential equation	darkOmega[2]

Table 4: Some useful constants included in **micrOMEGAs**

Name	Value	Units	Description
MPlanck	1.22091×10^{19}	GeV	Planck mass
EntropyNow	2.8912×10^9	m^{-3}	Present day entropy, s_0
RhoCrit100	10.537	GeVm^{-3}	ρ_c/h^2 or ρ for $H = 100\text{km/s/Mpc}$

with

$$H_1 = \begin{pmatrix} 0 \\ v + h/\sqrt{2} \end{pmatrix}, \quad H_2 = \begin{pmatrix} \tilde{H}^+ \\ (\tilde{X} + i\tilde{H}_3)/\sqrt{2} \end{pmatrix}. \quad (2)$$

Only even powers of H_2 appear, ensuring an exact Z_2 symmetry $H_2 \rightarrow -H_2$. This prevents mixing with H_1 and guarantees the stability of the lightest Z_2 -odd state – identified in **micrOMEGAs** as the dark matter particle. By convention, all odd fields must have names beginning with the tilde character ‘~’, which allows the code to recognise them automatically. See Section 2 for details.

5.2 Translation into CalcHEP syntax

The theoretical ingredients discussed above are distributed across the four **CalcHEP** model files, each fulfilling a distinct role in the numerical implementation. Tables 5–8 illustrate this correspondence step by step: the independent and derived parameters in **vars1.mdl** and **func1.mdl**, followed by the particle content and interaction vertices specified in **prtcls1.mdl** and **lgrng1.mdl**.

Model parameters. The model definition starts with the list of input and derived parameters. The independent quantities are defined in the file **vars1.mdl** and are shown in Table 5. They represent the fundamental inputs supplied by the user or fixed to Standard Model reference values. In the IDM, these include the electromagnetic coupling, the weak mixing angle, the Z -boson mass, and the masses of the inert scalars.

The dependent or constrained quantities are then specified in **func1.mdl** and summarised in Table 6. These relations express the derived parameters – such as the W -boson mass or the quartic couplings λ_i – in terms of the independent inputs. This separation be-

Name	Value	Comment
EE	0.31333	Electromagnetic coupling
SW	0.474	Sine of the Weinberg angle
MZ	91.187	Mass of the Z boson
MHX	111	Mass of inert scalar H_2 component
MH3	222	Mass of CP-odd scalar
MHC	333	Mass of charged scalar
LaL	0.01	Coupling in inert sector
...	...	...

Table 5: Independent input parameters defined in `vars1.mdl`.

tween inputs and functions ensures that parameter scans or theoretical consistency checks can be performed automatically without rewriting algebraic expressions in the model files.

Name	Expression
CW	$\sqrt{1-SW^2}$
MW	$MZ \cdot CW$
μ_2	$MHX^2 - LaL \cdot (2 \cdot MW / EE \cdot SW)^2$
la_3	$2 \cdot (MHC^2 - \mu_2) / (2 \cdot MW / EE \cdot SW)^2$
la_5	$(MHX^2 - MH3^2) / (2 \cdot MW / EE \cdot SW)^2$
...	...

Table 6: Derived parameters and algebraic relations defined in `func1.mdl`.

Particle content. The full particle inventory is given in Table 7, which corresponds to the file `prtcls1.mdl`. Each row specifies the quantum numbers and numerical identifiers of a physical state: the PDG code ensures interoperability with external programs, while the keyword `!wX` marks widths that `micrOMEGAs` should compute automatically. The three new inert scalars – $\sim X$, $\sim H_3$, and $\sim H_+$ – are explicitly introduced here alongside the Standard Model fields. Note that the symbol `!` before the width means that it will be computed automatically by CalcHEP from the interaction vertices. Symbol `G` in the `aux` column means that the particles will be treated in Feynman gauge and contribution of unphysical components of vector field has to be compensated by Faddeev-Popov and Goldstone ghosts.

Interaction vertices. The couplings that drive all physical processes are encoded in `lgrng1.mdl`, shown in Table 8. Each line corresponds to a term of the Lagrangian, expressed symbolically in the CalcHEP notation. Dirac matrices are denoted by `G`, while four-momenta are referenced as `m1.p2`, etc. For example, the entry involving $\{h, \sim X, \sim X\}$ represents the Higgs-portal interaction that controls both the annihilation rate of dark matter and its spin-independent scattering cross section.

See more details about implementation of BSM models in `micrOMEGAs` and generation of matrix elements in Sections 19, 18

Full Name	P	aP	PDG	spin2	mass	width	color	aux
photon	A	A	22	2	0	0	1	G
Z boson	Z	Z	23	2	MZ	!wZ	1	G
W boson	W+	W-	24	2	MW	!wW	1	G
Higgs	h	h	25	0	Mh	!wh	1	
odd Higgs	$\sim$ H3	$\sim$ H3	36	0	MH3	!wH3	1	
Charged Higgs	$\sim$ H+	$\sim$ H-	37	0	MHC	!wHC	1	
second Higgs	$\sim$ X	$\sim$ X	35	0	MHX	!wHX	1	
...	...	...	...	...	...	...	...	...

Table 7: Particle content of the IDM as implemented in `prtcls1.mdl`. The tilde identifies Z_2 -odd states recognised by `micrOMEGAs` as dark-sector fields.

P1	P2	P3	P4	Factor	Lagrangian term
A	W+	W-		-EE	$m3.p2*m1.m2 - m1.p2*m2.m3 - \dots$
A	$\sim$ H+	$\sim$ H-		EE	$m1.p3 - m1.p2$
h	$\sim$ X	$\sim$ X		$-2*MW*SW/EE$	$la3 + la4 + la5$
Z	Z	$\sim$ X	$\sim$ X	$EE^2(2*CW^2*SW^2)$	$m1.m2$
...	...	...	...	...	...

Table 8: Selected interaction vertices from `lgrng1.mdl`. These entries define the Feynman rules used in all subsequent cross-section calculations.

6 Setting of model parameters, spectrum calculation, parameter display

The independent parameters of a given model are specified in `work/models/vars1.mdl`. Three functions can be used to set the values of these parameters:

- `assignVal(name, val)`
- `assignValW(name, val)`

assign value *val* to parameter *name*. The function `assignVal` returns a non-zero value if it cannot recognize a parameter name while `assignValW` writes an error message.

- `readVar(fileName)`

reads parameters from a file. The file should contain two columns with the following format (see also Section 3.6)

name	value
------	-------

`readVar` returns zero when the file has been read successfully, a negative value when the file cannot be opened for reading and a positive value corresponding to the line where a wrong file record was found.

To create an input data file the user can choose the list of input parameters defined in `vars1.mdl` that need to be varied to produce a file in the appropriate format.

The constrained parameters of the model are stored in `work/models/func1.mdl`. Some of these parameters are treated as *public* parameters. The *public* parameters include by default all particle masses and all parameters whose calculation requires external functions (except simple mathematical functions like `sin`, `cos`, ...). The parameters needed

for the calculation of any *public* parameters in `work/models/func1.mdl` are also treated as *public*. It is possible to enlarge the list of *public* parameters. There are two ways to do this. One can type `*` before a parameter name to make it *public* or one can add a special record in `work/models/func1.mdl`

```
%Local! |
```

Then all parameters listed above this record become *public*.

The calculation of the particle spectrum and of all *public* model constraints is done with:

- `sortOddParticles(txt)`

This routine has to be called after a reassignment of any input model parameter, after changing the sets of particles in thermal equilibrium (section 8.5), and after defining the set of feeble particles (section 8.7). The routine calculates the constrained parameters of the model. This routine returns a non zero error code for a wrong set of parameters, for example parameters for which some constraint cannot be calculated. The name of the corresponding constraint is written in `txt`. This routine also defines the number of dark sectors containing particles in chemical equilibrium, `Ncdm`, and finds the name of the lightest particle in each sector `CDM[k]` ($k=1\dots N_{\text{cdm}}$) as well as the minimal mass in each sector, `McdmN[k]`. It also defines the mass of the lightest dark particle `Mcdm`, this particle can either be a WIMP or a FIMP.¹

- `qNumbers(pName, &spin2, &charge3, &cdim)`

returns the quantum numbers for the particle `pName`. Here `spin2` is twice the spin of the particle; `charge3` is three times the electric charge; `cdim` is the dimension of the representation of $SU(3)_c$, it can be 1, 3, -3 , 6, -6 or 8. The parameters `spin2`, `charge3`, `cdim` are variables of type `int`. The value returned is the PDG code. If `pName` does not correspond to any particle of the model then `qNumbers` returns zero.

- `pdg2name(nPDG)`

returns the name of the particle which PDG code is `nPDG`. If this particle does not exist in the model the return value is `NULL`.

- `antiParticle(pName)`

returns the name of the anti-particle for the particle `pName`.

- `pMass(pName)`

returns the numerical value of the particle mass.

Information about all particles is stored in array `ModelPrtcls` which contains `nModelParticles` elements. Each element of array is a `struct` which contains the fields:

```
char* name; // name of particle
char* aname; // name of anti-particle
int NPDG ; // PDG MC code
char* mass; // name of mass identifier
char* width; // name of width identifier
int spin2; // double spin
int cdim; // dimension of color representation
```

¹Note that `Mcdm1` and `Mcdm2` that were defined for two DM models are not defined anymore and have been replaced by `McdmN[1]` and `McdmN[2]`.

`int g; // number of degrees of freedom`

- `nextOdd(n, &pMass)`

returns the name and mass of the n^{th} odd particle assuming that particles are sorted according to increasing masses. For $n = 0$ the output specifies the name and the mass of the CDM candidate.

- `findVal(name,&val)`

finds the value of variable *name* and assigns it to parameter *val*. It returns a non-zero value if it cannot recognize a parameter name.

- `findValW(name)`

returns the value of variable *name* and writes an error message if it cannot recognize a parameter name.

The variables accessible by these last two commands are all free parameters and the constrained parameters of the model (in file `model/func1.mdl`) treated as *public*.

The following routines are used to display the value of the independent and the constrained *public* parameters:

- `printVar(FD)`

prints the numerical values of all independent and *public* constrained parameters into FD

- `printMasses(FD, sort)`

prints the masses of 'odd' particles (those whose names started with ~). If *sort* $\neq 0$ the masses are sorted so the mass of the CDM is given first.

- `printHiggs(FD, sort)`

prints the masses and widths of 'even' colorless scalars.

7 Thermodynamics of the Universe.

The thermodynamics of the Universe is determined by the effective numbers of degrees of freedom $h_{eff}(T)$ and $g_{eff}(T)$,

$$\rho_R(T) = \frac{\pi^2}{30} g_{eff}(T) T^4. \quad \text{and} \quad s(T) = \frac{2\pi^2}{45} h_{eff}(T) T^3. \quad (3)$$

These functions play a key role in the calculation of the DM relic density: $\rho_R(T)$ defines the Hubble rate H , Eq.6, and $s(T)$ allows to find the relation between the evolution of time and temperature after imposing entropy conservation

$$\frac{ds}{dt} = -3Hs \quad (4)$$

$$\frac{dt}{dT} = - \left(1 + \frac{1}{3} \frac{d \log(h_{eff}(T))}{d \log(T)} \right) \frac{1}{H(T)T} \quad (5)$$

These functions can be called with

- `gEff(T)`

which returns the effective number of degrees of freedom for the energy density of radiation at a bath temperature T , only SM particles are included.

- **hEff(T)**

which returns the effective number of degrees of freedom for the entropy density of radiation at a bath temperature T .

micrOMEGAs contains several files with tabulated data for **gEff** and **gEff**. They are stored in the folder **Data/hgEff/**. By default the tables for h_{eff}, g_{eff} correspond to the ones in Ref. [52] and can be found in the file **Data/hgEff/DHS.thg**. These default tables can be changed using

- **loadHeffGeff(char*fname)**

that looks for the file *fname* located in the directory **Data/hgEff** or in the current directory. This file should contain 3 columns for T , $h_{eff}(T)$ and $g_{eff}(T)$. A positive return value corresponds to the number of lines in the table. A negative return value indicates the line which creates a problem (e.g. wrong format), the routine returns zero when the file *fname* cannot be opened. The directory **Data/hgEff** also contains solutions described in Ref. [53], **LM.thg**, and in Ref. [54], **HP_B.thg**, **HP_C.thg**, as well as the tables used in DarkSUSY, **GG.thg**. The latter was used as default in previous versions of micrOMEGAs and does not include the contribution of the Higgs boson.

- **hEffLnDiff(T)**

returns the derivative of h_{eff} with respect to the *log* of the bath temperature, $\frac{d \log(h_{eff}(T))}{d \log(T)}$.

- **Hubble(T)**

returns the Hubble expansion rate in **GeV** units at a bath temperature $T[\text{GeV}]$. **Hubble** is defined via the density of the Universe and includes the contribution of radiation, dark matter, baryonic matter and dark energy,

$$H = \sqrt{\frac{8\pi\rho(T)}{3M_P^2}} \quad (6)$$

$$\rho(T) = \frac{\pi^2}{30}g_{eff}(T)T^4 + \mu_M \frac{2\pi^2}{45}h_{eff}(T)T^3 + \mu_{DE}^4 \quad (7)$$

where M_P is the Planck mass, $\mu_M = 0.519 \text{ eV}$, $\mu_{DE} = 2.24 \cdot 10^{-3} \text{ eV}$.

- **HubbleTime(T1,T2)**

which calculates the time interval in seconds during which the temperature of the Universe decreases from $T1[\text{GeV}]$ to $T2[\text{GeV}]$.

$$t = \int_{T_2}^{T_1} \left(-\frac{dt}{dT} \right) dT \quad (8)$$

The constant **T2_73K** gives the current temperature $T = 2.725\text{K}$. The age of the Universe calculated as **Hubble(10,T2_73K)** is 13.806 Gyr in agreement with the value quoted in the Particle Data Group [55] of 13.797(23) Gyr.

- **freeStreaming(p/m, T1,T2)**

calculates the length of the trajectory in **Mpc** units for a freely propagating particle between $T1[\text{GeV}]$ and $T2$. The parameter **p/m** characterizes the initial velocity of the particle $v = \frac{p/m}{\sqrt{1+(p/m)^2}}$. Usually the free streaming length, λ_{FS} , is obtained by the integral over time

$$\lambda_{FS} = \int_{t_2}^{t_1} \frac{v(t)}{a(t)} dt \quad (9)$$

where $a(t)$ is the scale factor, in particular $a = (s(T_{2.73K})/s(T))^{\frac{1}{3}}$. In the code we compute the free-streaming length as integral over T .

$$\lambda_{FS} = \int_{T_2}^{T_1} \left(1 + \left(\frac{a(T)m}{a(T_1)p} \right)^2 \right)^{-\frac{1}{2}} \frac{1}{a(t)} \left(-\frac{dt}{dT} \right) dT \quad (10)$$

See Section 8.6 for non-standard cosmology.

7.1 Modification of `geff`, `heff`

Note that entropy conservation follows from ²

$$\frac{d\rho}{dT} = T \frac{ds}{dT} \quad (11)$$

This relation is however not reproduced in the tables `LM.thg` and `DHS.thg` in the region of quark deconfinement. An improved solution that imposes entropy conservation is provided in the files `LMi.thg` and `DHSi.thg`. The correction can be found in the code `mdlIndep/improveGeff.c` which outputs a comparison between the two sets of tables.

New functions allow the user to include the contribution of new particles beyond the SM.

- `h1eff(M/T, mu/T, eta)`
- `g1eff(M/T, mu/T, eta)`

return the contribution of a single internal degree of freedom of a particle with mass `M` and chemical potential `mu` to the effective entropy (`hEff`) and energy (`gEff`) density functions. The argument `eta` specifies the particle statistics: `eta = 1` for bosons and `eta = -1` for fermions. To obtain the total contribution of a new particle species, the value returned by `h1eff` or `g1eff` must be multiplied by its internal degree of freedom g_i . By default, `micrOMEGAs` includes only Standard Model particles in `hEff` and `gEff`; these functions allow the user to add the contribution of additional BSM particles present in the thermal bath to the total effective degrees of freedom entering the computation of the Hubble rate and entropy density. To do this the user can create a new table in the format of the existing ones in the `Data/hgeff` directory and then use `loadHeffGeff("filename")`. The chemical potential is necessary to treat models with lepton asymmetry, otherwise the user can set $\mu/T = 0$.

The functions

- `p1eff(M/T, mu/T, eta)`
- `n1eff(M/T, mu/T, eta)`

can be used to calculate the pressure and number density of one degree of freedom

$$p = \frac{\pi^2}{90} p1eff(M/T, \mu/T, \eta) T^4 \quad (12)$$

$$n = n1eff(M/T, \mu/T, \eta) T^3 \quad (13)$$

²This equation is not valid for $T \leq 10$ MeV, where photons and neutrinos have different temperatures, or in case of lepton asymmetry.

These functions are normalized so that

$$h1eff(0,0,1) = g1eff(0,0,1) = p1eff(0,0,1) = 1 \quad (14)$$

$$n1eff(0,0,1) = \frac{\zeta(3)}{\pi^2} \quad (15)$$

8 Relic density calculation

8.1 Switches and auxiliary routines

micrOMEGAs can treat three different scenarios for the calculation of the relic density : *freeze-out*, *freeze-in* and modified cosmology denoted as *inflaton*. The scenario can be selected automatically by specifying a flag in the input file :

```
#! FreezeOut
#! FreezeIn
#! Inflaton
```

The first flag is applied by default and can be omitted. The selection can also be made by the user in the main.c file.

- **VWdecay, VZdecay**

Switches to turn on/off processes with off-shell gauge bosons in the final state for DM annihilation and particle decays. If **VW/VZdecay=1**, the 3-body final states will be computed for annihilation processes only while if **VW/VZdecay=2** they will be included in coannihilation processes as well. By default the switches are set to (**VW/VZdecay=1**).³ Note that micrOMEGAs calculates the width of each particle only once and stores the result in *Decay Table*. A second call to the function **pWidth** (whether an explicit call or within the computation of a cross section) will return the same result even if the user has changed the **VW/VZdecay** switch. We recommend to call

- **cleanDecayTable()**

after changing the switches to force micrOMEGAs to recalculate the widths taking into account the new value of **VW/VZdecay**. The **sortOddParticles** command which must be used to recompute the particle spectrum after changing the model parameters also clears the decay table.

- **useSLHAWidth**

Switch to determine how the particle widths are computed. If **=1** the particle widths stored in a SLHA file (SUSY Les Houches Accord [56]) are downloaded by micrOMEGAs. These widths then do not depend on the **VW/VZdecay** switches. If **=0** micrOMEGAs will calculate the widths, it will also do so if the switch is set to 1 and the widths are not provided in the SLHA file. By default this switch is set to 0.

- **improveCrossSection(n1,n2,n3,n4, Pcm,T, &cs)**

allows to substitute a new cross-section for a given process instead of the one calculated by micrOMEGAs at tree level. Here **n1,n2** are the PDG codes for particles in the initial state and **n3,n4** for those in the final state. **Pcm** is the center of mass momentum and **cs** is the cross-section in [GeV^{-2}]. The cross-section can be temperature dependent, **T** is

³Including the 3-body final states can significantly increase the execution time for the relic density computation.

the temperature in [GeV]. This function is called just after the calculation of the annihilation cross section in routines that calculates the relic density and indirect detection. **micrOMEGAs** calls this routine substituting for the last parameter the address of the memory where the calculated tree level cross section **cs** is stored. This function is useful if, for example, the user wants to include their loop improved cross-section calculation and/or the Sommerfeld effect. **micrOMEGAs** contains a dummy version of this routine disposed in **sources/improveCS.c** which does not modify the default cross section. This file also contains some commented out example of the code for the IDM model. To activate this facility the user has to write their own version of the **improveCrossSection** routine and place it in the directory **MODEL/lib**, then the *dummy* version will be ignored.

- **SYukawa(a,b)** and **SHulthen(a,b)**

These functions can be used to calculate Sommerfeld enhancement in the **improveCrossSection** routine for Yukawa and Hulthen potentials for s-channel scattering. The arguments are

$$a = \alpha/v \quad (16)$$

$$b = m_{med}/(m_r v) \quad (17)$$

where v is the relative velocity of colliding particles, m_{med} is the mass of the mediator, $m_r = \frac{m_1 m_2}{m_1 + m_2}$ - the reduced mass of colliding particles and $\alpha = \frac{e^2}{4\pi}$ where the coupling e is defined from the Lagrangian that describes interactions of a Dirac particle (f_D) or of a charged scalar field (ϕ) with a scalar (h) or a vector (V_μ) mediator,

$$\begin{aligned} \mathcal{L} &= eh\bar{f}_D f_D, & \mathcal{L} &= ev_\mu \bar{f}_D \gamma^\mu f_D \\ \mathcal{L} &= 2eM_\phi h\phi\phi^*, & \mathcal{L} &= ieV_\mu(\partial^\mu \phi h\phi^* - \phi\partial^\mu \phi^*) \end{aligned} \quad (18)$$

For Majorana particles $\mathcal{L}$ contains an extra factor $\frac{1}{2}$. The Sommerfeld factor for pseudo-scalar and axial-vector interactions is not available in **micrOMEGAs**, since it has been shown to be negligible [57]. The calculation of **SYukawa** is based on Eq.5.1-5.4 of [58]. Here we solve numerically the differential equation for DM elastic scattering in spherical coordinates and compute the wave function at zero to get the enhancement factor. For the Hulthen potential we use an analytical solution described in Eq. 5.6 of [59]. A demonstration of the Sommerfeld enhancement for both **SYukawa** and **SHulthen** can be found in the file **mdlIndep/Sommerfeld.c**.

8.2 Temperature interval and Error codes for routines calculating relic density

micrOMEGAs provides three routines, **darkOmega**, **darkOmega2**, **darkOmegaN** for calculating the relic density of one-component, two-component and N-component DM respectively by solving differential equations for the abundances

$$Y_i = n_i/\mathfrak{s} \quad (19)$$

where n_i is the number density of the i^{th} -component DM and $\mathfrak{s}$ is the entropy density. The equations contain the thermal equilibrium abundance $\bar{Y}_i = \frac{\bar{n}_i}{\mathfrak{s}}$ as well as $\chi\chi \rightarrow SM, SM$ annihilation cross sections $v\sigma(T)$. Decays of dark sector particles and processes such as $\chi, SM \rightarrow \chi', SM$ are taken into account only in **darkOmegaN**. Abundances are calculated at a temperature **Tend** that can be defined by the user. The default value is **Tend=10⁻³GeV**

since in general the evolution of the relic abundance stops at higher temperatures. The initial temperature for integration, `Tstart`, is set by the condition

$$Y(Tstart) - \bar{Y}(Tstart) \approx 0.1\bar{Y}(Tstart).$$

These functions are described in the following subsections.

`micrOMEGAs` provides as well the routines `darkOmegaTR`, `darkOmega2TR`, `darkOmegaNTR` which also calculate the DM relic density. However the input parameters for these routines specify the initial temperature and abundances. The initial temperature is assigned to the variable `Tstart`.

All the `darkOmega*` routines have an `&err` parameter which returns the following error code :

```
32 - no WIMP
64 - Tstart is not found. It means that one of the DM component was never
    in thermal equilibrium with SM particles.
128 - problem in solution of differential equation. It can appear if the
    equation is stiff because Tstart is very large.
```

The `darkOmega*` routines return `NAN` if one of these error appears.

For calculating $v\sigma(T)$ we use the program `simpson` to evaluate the integrals over scattering angle and energy of collisions. This program can return the following error codes

```
1 - NAN in integrand;
2 - too deep recursion;
4 - loss of precision.
```

which are passed to `&err`. In general, these codes can be treated as warnings, although it can be useful to check the calculation of the problematic integrals using e.g. the `gdb` debugging tools. More information on this tool can be found in section 20.1. The error code `err` is a binary code which can signal several problems simultaneously.

8.3 Calculation of relic density for one-component Dark Matter models

All routines to calculate the relic density in version 3 are available in further versions. For these routines, the difference between dark sectors is ignored and the dark matter is the lightest particle among all those whose names starts with a $\tilde{\cdot}$. These routines are intended for models with either a Z_2 or Z_3 discrete symmetry.

• `vSigmaA(T,fast,Beps)`, `vSigmaS(T,fast,Beps)`

calculates the thermally averaged cross section for DM annihilation times velocity at a temperature T [GeV], $\sigma_v = \langle v\sigma \rangle$ for DM annihilation (A) and semi-annihilation. (S),

$$\langle v\sigma^A \rangle_T = \frac{T}{8\pi^4\bar{n}(T)^2} \int ds \sqrt{s} K_1\left(\frac{\sqrt{s}}{T}\right) \sum_{\tilde{\alpha},\tilde{\beta}} p_{\tilde{\alpha}\tilde{\beta}}^2(s) g_{\tilde{\alpha}} g_{\tilde{\beta}} \sum_{x \geq y} \sigma_{\tilde{\alpha}\tilde{\beta} \rightarrow xy}(s) \quad (20)$$

$$\langle v\sigma^S \rangle_T = \frac{T}{8\pi^4\bar{n}(T)^2} \int ds \sqrt{s} K_1\left(\frac{\sqrt{s}}{T}\right) \sum_{\tilde{\alpha},\tilde{\beta}} p_{\tilde{\alpha}\tilde{\beta}}^2(s) g_{\tilde{\alpha}} g_{\tilde{\beta}} \sum_{x\tilde{\gamma}} \sigma_{\tilde{\alpha}\tilde{\beta} \rightarrow x\tilde{\gamma}}(s) \quad (21)$$

where

$$\bar{n}(T) = \frac{T}{2\pi^2} \sum_{\tilde{\alpha}} g_{\tilde{\alpha}} m_{\tilde{\alpha}}^2 K_2\left(\frac{m_{\tilde{\alpha}}}{T}\right), \quad (22)$$

is the equilibrium number density of DM particles. Here $\tilde{\alpha}, \tilde{\beta}, \tilde{\gamma}$ is used for **Odd** particles and x, y for **Even** particles. Here only $2 \rightarrow 2$ processes are included and $\sigma_{\tilde{\alpha}\tilde{\beta} \rightarrow x[\tilde{\gamma}/y]}$ is the cross section for the corresponding process averaged over the spins of incoming particles and summed over the spins of outgoing particles. K_1, K_2 are modified Bessel functions of the second kind, and $m_{\tilde{\alpha}}$ and $g_{\tilde{\alpha}}$ stand for the mass and the number of degrees of freedom of particle $\tilde{\alpha}$. Note, that if $\tilde{\alpha} \neq \tilde{\beta}$ then each $\sigma_{\tilde{\alpha}\tilde{\beta}}$ term will be presented twice. The value for $v\sigma$ is expressed in $[pb \cdot c]$. The parameter *Beps* defines the criteria for including coannihilation channels as for **darkOmega** described below. The *fast* = 1/0 option switches between the *fast/accurate* calculations.

Note, that if $\tilde{\alpha} \neq \tilde{\beta}$, both $\sigma_{\tilde{\alpha}\tilde{\beta}}$ and $\sigma_{\tilde{\beta}\tilde{\alpha}} = \sigma_{\tilde{\alpha}\tilde{\beta}}$ will contribute to the sum. Moreover, the sum over α in Eq.20,21,22 runs over each particle and anti-particle separately.

In the low temperature limit

$$\langle v\sigma^A \rangle_{T=0} = \frac{1}{\bar{n}(T)^2} \sum_{\tilde{\alpha}, \tilde{\beta}} \sum_{x \geq y} \bar{n}_{\tilde{\alpha}}(T) \bar{n}_{\tilde{\beta}}(T) \lim_{v \rightarrow 0} v \sigma_{\tilde{\alpha}\tilde{\beta} \rightarrow xy}(v) \quad (23)$$

where v is the relative velocity. When only one particle and its anti-particle contribute to DM annihilation, Eq.23 reads

$$\langle v\sigma^A \rangle_{T=0} = \frac{1}{2} \lim_{v \rightarrow 0} (v \sigma_{\tilde{\alpha}, \tilde{\alpha} \rightarrow SM, SM}(v) + v \sigma_{\tilde{\alpha}, \tilde{\alpha} \rightarrow SM, SM}(v)) \quad (24)$$

In many models, for example when DM is a Dirac fermion, there are no process $\tilde{\alpha}, \tilde{\alpha} \rightarrow SM, SM$ and our definition of $\langle v\sigma \rangle_T$ looks unnatural. However this definition has the advantage that it leads to a universal formula for the number of annihilation events in unit of space-time volume, regardless of the nature of DM,

$$\frac{dN}{dt d^3x} = \frac{1}{2} \langle v\sigma \rangle_T n^2 \quad (25)$$

where n is the number density of DM particles⁴.

After a call to **vSigmaA** or **vSigmaS**, the global array **vSigmaTCh** contains the contribution of different channels to **vSigma**. **vSigmaTCh[i].weight** specifies the relative weight of the i^{th} channel,

vSigmaTCh[i].prtcl[j] ($j=0, 4$) defines the particles names for the i^{th} channel.

The last record in **vSigmaTCh** array has zero weight and NULL particle names.

In terms of $v\sigma^A$ and $v\sigma^S$ the abundance equation reads

$$\frac{dY}{dt} = - \langle v\sigma^A \rangle_T (Y^2 - \bar{Y}^2) - \frac{\langle v\sigma^S \rangle_T}{2} (Y^2 - Y\bar{Y}) \quad (26)$$

• **darkOmega(&Xf, fast, Beps, &err)**

calculates the dark matter relic density Ωh^2 . This routine solves the differential evolution

⁴Eq.25 is valid if we assume a Maxwell-Boltzmann distribution, as is done for all routines that compute the relic density of WIMPs, the treatment of quantum statistics in the case of FIMPs is discussed in section 8.7.

equation using the Runge-Kutta method. $X_f = M_{\text{cdm}}/T_f$ characterizes the freeze-out temperature which is defined by the condition $Y(T_f) = 2.5\bar{Y}(T_f)$. For asymmetric DM this condition reads $2\sqrt{Y^+(T_f)Y^-(T_f)} = 2.5\bar{Y}(T_f)$. The value of X_f is given for information and is also used as an input for the routine that gives the relative contribution of each channel to Ωh^2 , see `printChannels` below. The `fast = 1` flag forces the fast calculation (for more details see Ref. [3]). This is the recommended option and gives an accuracy around 1%. The parameter `Beps` defines the criteria for including a given coannihilation channel in the computation of the thermally averaged cross-section, [3]. The recommended value is $B_{\text{eps}} = 10^{-4} - 10^{-6}$ whereas if $B_{\text{eps}} = 1$ only annihilation of the lightest odd particle is computed. Non-zero error code means that the temperature where thermal equilibrium between the DM and SM sectors is too large $M_{\text{cdm}}/T < 2$ or $T > 10^5 \text{ GeV}$.

`darkOmega` solves the differential equation for the abundance $Y(T)$ in two temperature intervals `[Tstart,Tf]` and `[Tf,Tend]` [2]. The temperatures are defined by the conditions $Y(T_{\text{start}}) \approx 1.1\bar{Y}(T_{\text{start}})$, $Y(T_f) \approx 10\bar{Y}(T_f)$ while `Tend` is defined by the user. In the second interval, `[Tf,Tend]`, the contribution of $\bar{Y}$ is neglected and the differential equation is integrated explicitly. The solution in the interval `[Tstart,Tf]` is tabulated and can be displayed via the function `YF(T)`. The equilibrium abundance can be accessed with the function `Yeq(T)`.

- `darkOmegaF0(&Xf, fast, Beps)`

calculates the dark matter relic density Ωh^2 using the freeze-out approximation.

- `printChannels(Xf,cut,Beps,prcnt,FD)`

writes into `FD` the contributions of different channels to $(\Omega h^2)^{-1}$. Here `Xf` is an input parameter which should be evaluated first in `darkOmega[F0]`. Only the channels whose relative contribution is larger than `cut` will be displayed. `Beps` plays the same role as in the `darkOmega[F0]` routine. If `prcnt` $\neq 0$ the contributions are given in percent. Note that for this specific purpose we use the freeze-out approximation.

- `oneChannel(Xf,Beps,p1,p2,p3,p4)`

calculates the relative contribution of the channel $p1, p2 \rightarrow p3, p4$ to $(\Omega h^2)^{-1}$. `p1,...,p4` are particle names. To sum over several channels one can write "*" instead of a particle name, e.g. "*" in place of `p1`.

- `omegaCh` is an array that contains the relative contribution and particle names for each annihilation channel. These array and function are similar to `vSigmaTCh` described above. The array `omegaCh` is filled after calling either `darkOmegaF0` or `printChannels`.

- `darkOmegaTR(Tstart,Ystart, fast, Beps)`

This function is similar to `darkOmega` except that the initial temperature (`Tstart`) and abundance (`Ystart`) that are needed for solving the evolution equation have to be provided by the user as arguments to the function.

There is an option to calculate the relic density in models with DM- $\overline{\text{DM}}$ asymmetry. In this case, we assume that the number difference of DM- $\overline{\text{DM}}$ is conserved in all reactions. Thus a small difference in initial abundances can lead to a large DM asymmetry after freeze-out as is the case for the baryon asymmetry.

- **deltaY**

describes the difference between the DM and anti-DM abundances for the models where the number of DM particles minus the number of anti-DM is conserved in decays and collisions. In such models **deltaY** is a constant during the thermal evolution of the Universe, see Ref. [7].

- **dmAsymm**

is defined by the equation

$$\Omega_{\pm} = \Omega \frac{1 \pm \text{dmAsymm}}{2}$$

and evaluated by **micrOMEGAs** while calculating the relic density with an initial asymmetry **deltaY**, see [7]. This parameter can also be reset after the relic density computation and will then be taken into account for direct and indirect detection rates.

8.3.1 darkOmegaExt

- **darkOmegaExt(&Xf, vs_a, vs_s, vs_3, vs_4, &err)**

calculates the dark matter relic density Ωh^2 using annihilation cross sections of two odd particles. The annihilation cross-sections are provided by external functions **vs_a**, **vs_s**, **vs_3**, **vs_4** which return **vSigma** in $[pb \cdot c]$ units as a function of temperature.

Here **vs_a** is the annihilation cross section into two SM particles as in Eq.20, while **vs_s** is the semi-annihilation cross section with production of one outgoing odd particle, Eq.21. **vs_3** and **vs_4** correspond to the annihilation cross section into 3 and 4 odd particles, these can be relevant for strongly interacting DM⁵. **vs_3(T)** and **vs_4(T)** must include an additional factor of $\exp(M_{\text{cdm}}/T)$ and $\exp(2M_{\text{cdm}}/T)$ respectively. These factors arise because the main processes are the reverse ones ($3 \rightarrow 2$ and $4 \rightarrow 2$) which are taken into account via the detailed balance equation. These contributions can be essential at low temperatures when the direct ones becomes zero because of Boltzmann suppression [60]. Note that some input functions may not be relevant, in this case the user should replace them with NULL.

darkOmegaExt solves the following abundance equation

$$\begin{aligned} \frac{dY}{dT} = & -v\sigma_a(Y^2 - Y_{eq}^2) - \frac{1}{2}v\sigma_s(Y^2 - Y_{eq}Y) \\ & + \frac{1}{2}v\sigma_3Y^2(e^{-\frac{M_{\text{cdm}}}{T}} - \frac{Y}{\tilde{Y}_{eq}}) + v\sigma_4Y^2(e^{-\frac{2M_{\text{cdm}}}{T}} - \frac{Y^2}{\tilde{Y}_{eq}^2}) \end{aligned} \quad (27)$$

where $\tilde{Y}_{eq} = Y_{eq}e^{\frac{M_{\text{cdm}}}{T}}$.

- **darkOmegaExtTR(&Xf, TR, YR, vs_a, vs_s, vs_3, vs_4, &err)**

This routine is similar to **darkOmegaTR**. It solves Eq. 27 in the interval $T \in [TR, T_{\text{end}}]$ with the condition $Y=YR$. Note that the function **darkOmegaExt** described above finds a starting point and calls **darkOmegaExtTR**. The evolution of the abundance can be accessed via function **YF(T)**.

At first, **darkOmegaExtTR** builds the interpolation for the external functions as required for a fast solution of the evolution equation. The result of the interpolation can be verified by functions **vs_aExt(T)**, **vs_sExt(T)**, **vs_3Ext(T)**, **vs_4Ext(T)**.

⁵Processes with 2 outgoing odd particles are omitted because they do not change the number of odd particles.

8.3.2 vSigmaCC

- **vSigmaCC(T, cc, mode)**

calculates the thermally averaged *cross section* $\times$ *velocity* for $2 \rightarrow 2$, $2 \rightarrow 3$, and $2 \rightarrow 4$ processes. T is the temperature in [GeV], cc is the address of the code for each process. This address can be obtained by the function **newProcess** presented in Section 18. The returned value is given in [c.pb]. Note that **vSigmaCC** takes into account only the first subprocess in the cc code, while the **newProcess** routines allows generation of code for multi processes.

To prepare the code for the **darkOmegaExt** routine, one needs $mode \neq 0$, say, $mode = 1$. If $mode \neq 0$, **vSigmaCC** calculates the contribution of a given process to the total annihilation cross section, see Eq.20,21. The incoming particles should belong to the odd sector. For $2 \rightarrow 2$ processes the result after summation over all subprocesses should be identical to the one obtained via **vSigma** above. For this mode, **vSigmaCC** includes combinatoric factors: 2 if $\tilde{\alpha} \neq \tilde{\beta}$, an additional factor 2 if the incoming state is not self-conjugated. For processes with 3 and 4 outgoing particles **vSigmaCC** checks the number of odd outgoing particles and adds factors $e^{\frac{M_{cdm}}{T}}$ and $e^{\frac{2M_{cdm}}{T}}$ when needed. This means that the cross sections prepared by **vSigmaCC** can be used directly in **darkOmegaExt**.

If $mode = 0$, **vSigmaCC** is defined by the integral

$$\langle v \sigma^{\tilde{\alpha}\tilde{\beta} \rightarrow X} \rangle_{T=0} = \frac{1}{2T m_{\tilde{\alpha}}^2 m_{\tilde{\beta}}^2 K_2(\frac{m_{\tilde{\alpha}}}{T}) K_2(\frac{m_{\tilde{\beta}}}{T})} \int ds \sqrt{s} K_1(\frac{\sqrt{s}}{T}) p_{cm}^2(s) \sigma^{\tilde{\alpha}\tilde{\beta} \rightarrow X}(p_{cm}(s))$$

where p_{cm} is the center of mass momentum of incoming particles. Note that

$$\lim_{T \rightarrow 0} \text{vSigmaCC}(T, cc, 0) = \lim_{p_{cm} \rightarrow 0} \sigma(p_{cm}) v_{rel}(p_{cm})$$

where $v_{rel}(p_{cm})$ is the relative velocity of incoming particles. The result of **vSigmaCC** can be different from that of **vSigma** described above when there is an important contribution from NLSP's to the total number density of DM particles.

One important application of **darkOmegaExt** is that it can be used to take into account off shell resonances in the calculation of the relic density. For this one has to first compute the corresponding $2 \rightarrow 3$ and/or $2 \rightarrow 4$ processes, this can be done with the functions presented below in the narrow width approximation:

- **vSigmaPlus24(proc22, T, &err)**

calculates the contribution to $v\sigma$ for a $2 \rightarrow 4$ process associated with the $2 \rightarrow 2$ process for which outgoing particles are off-shell. Here **proc22** is the name of the $2 \rightarrow 2$ process and T is the temperature. For each virtual particle, the decay channel with the largest branching fraction is chosen, the result is then divided by the corresponding branching fractions.

- **vSigmaPlus23(proc22, T, &err)**

calculates the contribution to $v\sigma$ for a $2 \rightarrow 3$ process associated with the $2 \rightarrow 2$ process (**proc22**) for which one of the outgoing particles can be off-shell. The contribution of the on-shell $2 \rightarrow 2$ process (for example $\chi\chi \rightarrow BB'$) is subtracted. First, the 3-body cross-section for the process corresponding to one real and one virtual final state (corresponding to the main decay channel of the virtual particle) is computed. To ensure the proper

behaviour near threshold one would need to compute the 4-body final state, rather we use a trick to approximate the 4-body result from the 3-body result after applying a K-factor. To obtain the K factor we assume that the 4-body matrix element is proportional to

$$|\mathcal{M}_4^2| \propto \left[m_{B'}^2 m_B^2 + \frac{1}{8} (s - m_{B'}^2 - m_B^2)^2 \right] \times \quad (28)$$

$$\frac{\Gamma_B}{((m_B^2 - m_B'^2)^2 - \Gamma_B^2 m_B'^2)} \frac{\Gamma_B'}{((m_{B'}^2 - m_B'^2)^2 - \Gamma_B'^2 m_B'^2)} \quad (29)$$

where $m_B, m_{B'}$ and $m_B', m_{B'}$ are respectively the virtual and pole masses of B and B', $\Gamma_B, \Gamma_{B'}$ the widths of the virtual particles, and s is taken in the range $2m_\chi < \sqrt{s} < m_B + m_{B'} + 10(\Gamma_B + \Gamma_{B'})$. We integrate this matrix element with $\Gamma_B = 0$ in order to simulate the 3-body result, then we integrate the same matrix element with the correct value for Γ_B . The ratio of these integrals gives the K factor. The same trick is applied automatically for calculating matrix elements with virtual W and Z if the flags `VWdecay` and `VZdecay` are activated. see Section 8.1 [7].

When first called, both `vSigmaPlus23` and `vSigmaPlus24` tabulate the cross sections of $2 \rightarrow 3$ and $2 \rightarrow 4$ processes respectively and keep results in memory for subsequent calls to calculate integrals over s . A call to `sortOddParticles()` cleans the tabulated cross sections.

To take into account the off-shell contribution in the computation of the relic density one has to create a new function which sums the contributions of `vSigmaA` and `vSigmaPlus23` and `vSigmaPlus24` and pass this function to `darkOmegaExt`. We have checked that both `vSigmaPlus23` and `vSigmaPlus24` give similar results.

The functions presented in this section are used to calculate the relic density of a single WIMP particle. If they are used for a model with multiple dark sectors, it will assume that all particles of the dark sectors are in chemical equilibrium with each other and will solve only for the relic density of the lightest particle of the dark sectors. The fractional contribution of heavier DM particles to the total relic density will be set to zero. A proper calculation of multi-component DM must rather rely on the routines described in sections 8.4 and 8.5. If the DM is feeble (FIMP), special routines have to be used to compute the relic density, see Section 8.7. The particles that are marked as *feeble* are just ignored by the routines described in this section.

8.4 Calculation of relic density for two-component Dark Matter models

- `darkOmega2(fast, Beps)`

Calculates Ωh^2 for either one- or two-components DM models. In the former case it should give the same result as `darkOmega`. The parameters `fast` and `Beps` have the same meaning as for the `darkOmega` routine. The returned value corresponds to the sum of the contribution of the two DM components to Ωh^2 .

The two-component DM are referred to as CDM[1] and CDM[2] and correspond to the lightest particle in each sector. `darkOmega2` also fills the elements of the array `fracCDM[1]`, `fracCDM[2]` which contains the mass fraction of CDM[1] and CDM[2] in

the total relic density, namely,

$$\text{fracCDM}[i] = \frac{\Omega_i}{\Omega_1 + \Omega_2} \quad (30)$$

These fractions are then used in routines which calculate the total signal from both DM components in direct and indirect detection experiments, `nucleusRecoil`, `calcSpectrum`, and `neutrinoFlux`. The user can change the global `fracCDM[i]` parameter before the calculation of these observables to take into account the fact that the value of the DM fraction in the Milky Way could be different than in the early Universe.

The routines that were described in section 8.3 are not available for two-component DM models. In particular the individual channel contribution to the relic density cannot be computed and DM asymmetry is ignored. After calling `darkOmega2` the user can check the cross sections for each class of reactions (but not for individual processes) which were tabulated during the calculation of the relic density. The functions

- `vsabcd F(T)`

computes the sum of the cross sections for each class of reactions ($a, b, c, d = 0, 1, 2$) tabulated during the calculation of the relic density. Here T is the temperature in [GeV] and the return value is $v\sigma$ in [c·pb]. These functions are defined in the interval $[T_{\text{start}}, T_{\text{end}}]$, where T_{start} is a global parameter defined by `darkOmega2`, $T_{\text{end}}=10^{-3}\text{GeV}$. Specifically the functions available are

<code>vs1100F</code>	<code>vs1110F</code>	<code>vs1120F</code>	<code>vs1112F</code>	<code>vs1122F</code>	<code>vs1210F</code>	<code>vs1211F</code>
<code>vs1220F</code>	<code>vs1222F</code>	<code>vs2200F</code>	<code>vs2210F</code>	<code>vs2220F</code>	<code>vs2211F</code>	<code>vs2221F</code>

The temperature dependence of the abundances can also be called by the user, the functions are named `Y1F(T)` and `Y2F(T)` and are defined only in the interval $T \in [T_{\text{end}}, T_{\text{start}}]$. The equilibrium abundances are accessible via the `Yeq1(T)`, `Yeq2(T)` functions and the deviation from equilibrium by the functions `dY1F(T) = Y1F(T) - Y1eq(T)` and `dY2F(T) = Y2F(T) - Y2eq(T)`.

- `darkOmega2TR(Tstart, Y1start, Y2start, fast, Beps)`

This function is similar to `darkOmega2` described above, however the initial temperature (`Tstart`) and abundances (`Y1start`, `Y2start`) that are needed for solving the evolution equations should be passed by the user as arguments to the function.

As was the case for routines for one component DM, `darkOmega2` ignores particles declared as feeble. However one can use `darkOmega2TR` for a model which contains both a FIMP and a WIMP by specifying e.g. `Y1start = Yeq` for the WIMP in sector 1 and `Y2start = 0` for the FIMP in sector 2. Note that the decay processes between particles of different sectors are not taken into account, for models, where these can be important we recommend rather to use `darkOmegaN` described in the next section.

8.5 Calculation of relic density for N-component DM taking into account coscattering processes.

In this section we present routines for the calculation of the relic density of N-component DM. The dark particles need to be divided into *sectors*, within each of which chemical equilibrium is observed. By default, this separation is defined by the number of `~` symbols in the beginning of the particle names. Thus, `~x1` and `~~x2` denote dark particles of two

different sectors. Usually, the sector assignment corresponds to the charge of the discrete symmetry responsible for DM stability, cf. section 2. However, in the absence of chemical equilibrium, the splitting into sectors needs to be done differently.

There are two types of processes which are responsible for chemical equilibrium: decays and cospattering (or DM conversion) processes. The latter corresponds to processes of the type

$$\chi_i, SM \rightarrow \chi'_i, SM' \quad (31)$$

In general the decay processes are responsible for chemical equilibrium at low temperatures, while cospattering processes maintain chemical equilibrium at high temperatures. The `darkOmegaN` routine calculates DM abundances taking into account both cospattering and decay processes which relate different sectors.

In the absence of chemical equilibrium, the splitting into sectors is done using the function

- `defThermalSet(n, particles_list)` which moves all particles mentioned in *particles_list* to sector *n*. All particles that were assigned to sector *n* before this command are returned to their default sectors specified by the number of ~ in the beginning of their names. Particles in the *particles_list* have to be separated by commas, and particle and anti-particle automatically belong to the same sector. By definition, sector 0 is the SM bath while sector -1 is used to define *feeble* particles which do not take part in freeze out. Such particles will be ignored when solving for the relic density. Sectors *n* > 0, are used for all other cases.

In general, `defThermalSet` can define a set which includes particles with different charges of the discrete symmetry group (different number of ~ symbols) — in particular the set could include Z_2 odd particles as well as SM particles. In this case the user must keep in mind that the abundance equations are solved for sectors *n* > 0 only. This entails that a Z_2 odd particle assigned to sector 0 will not be considered as potential DM candidate. The function returns an error code if *particles_list* contains a particle name which is not defined in the model.

- `printThermalSets()` prints the contents of all particle sets specified by `defChEset` on the screen.

To verify whether chemical equilibrium is reached in one sector, one can use

- `checkTE( n, T, mode, Beps)` which checks the condition for chemical equilibrium in the n^{th} sector at temperature *T*. If *mode*=0, then both decay and co-scattering are taken into account. If *mode*=1 (2), then only decay (co-scattering) processes are taken into account. `checkTE` returns the minimal value of $\Gamma/H(T)$ obtained after testing all possible subsets of particles in sector *n*. The particle assignment corresponding to the minimal value of $\Gamma/H(T)$ is printed on the screen. This value has to be $\gg X_f$ to have chemical equilibrium, when this condition is satisfied the correction to the abundance calculated assuming chemical equilibrium is approximately $\Delta Y/Y \approx X_f H/\Gamma$.

- `YdmNEq(T,  $\alpha$ )` calculates the thermal equilibrium abundance for particles of sector α , where α has to be presented by a text label. For instance, `YdmNEq(T, "1")`.

- `vSigmaN(T, channel)` calculates the thermally averaged cross section $\langle v\sigma \rangle$ in [pb·c] units. Here *channel* is a text code specifying the reaction, e.g. `vSigmaN(T, "1100")` for $1, 1 \leftrightarrow 0, 0$ processes. If *channel* starts with an exclamation mark, then `vSigmaN` returns the results of the previous call of `darkOmegaN`. Otherwise `vSigmaN` recalculates all needed cross sections using the parameters *fast*, *Beps* defined in previous call of `darkOmegaN` or the parameters defined by a call to

- `setFastBeps(fast, Beps)`

To find the contribution of different processes to `vSigmaN`, one can call

- `vSigmaNCh(T, channel, fast, Beps, &vsPb)` which returns an array of annihilation processes together with their relative contributions to the total annihilation cross section. The cross section is given by the return parameter `vsPb` in [pb c] units. The elements of the array are sorted according to weights and the last element has `weight=0`. The structure of this array is identical to `vSigmaTCh` which was defined for one-DM models, see 8.3. The input parameter `channel` is written in text format. The memory allocated by `outCh` can be cleaned after usage with the command `free(outCh)`. The following lines of code give an example on how to use this function

```
aChannel*outCh=vSigmaNCh(T, "1100", Beps, &vsPb);
for(int n=0;;n++)
{ if(outCh[n].weight==0) break;
  printf(" %.2E  %s %s -> %s %s\n", outCh[n].weight,
    outCh[n].prtcl[0], outCh[n].prtcl[1], outCh[n].prtcl[2], outCh[n].prtcl[3]);
}
free(outCh);
```

- `darkOmegaNTR(TR, Y, fast, Beps, &err)` solves the equation of the thermal evolution of abundances starting from the initial temperature `TR` and returns the total Ωh^2 as described in [19]. The array `Y` has to contain the initial abundances at the temperature `TR`. After completion, `Y[k]` contains the abundances of sector $k - 1$ at the temperature `Tend` defined by the user ⁶. The parameter `TR` is assigned to the global variable `Tstart`.
- `darkOmegaN(fast, Beps, &err)` calls `darkOmegaNTR` to solve the equations of the thermal evolution of abundances in the temperature interval `[Tend, Tstart]`. In each sector, the function looks for the temperature T_i where $Y_i(T_i) \approx Y_{eq}(T_i)$. The minimum value of T_i is assigned to `Tstart`. If `Tstart` is not found, then the error code 64 is generated and `darkOmegaN` returns NaN.
- `YdmN(T,  $\alpha$ )` presents the evolution of abundances for particles of sector α calculated by `darkOmegaN` or `darkOmegaNTR` for $T \in [Tend, Tstart]$.

For the above functions, `micrOMEGAS` provides the possibility to selectively exclude part of the terms in the evolution equation. This is realized via the string `ExcludedForNDM`, which can be assigned specific keywords. The keyword `"DMdecay"` excludes decay processes which contribute to the DM evolution, while the keyword `"1100"` excludes $1, 1 \leftrightarrow 0, 0$ processes. to exclude cospattering ($2, 0 \leftrightarrow 1, 0$ or $1, 0 \leftrightarrow 2, 0$) processes, set

```
ExcludedForNDM="2010";
```

To reset and include all channels one must use `ExcludedForNDM=NULL;`.

⁶By default `Tend=10-3GeV`, however when the decay contribution is important it is preferable to choose a smaller value such as `Tend=10-8GeV`.

Note that, for the computation of cospattering, the user defines which particles belong to sector 1 and sector 2. If all particles are in thermal equilibrium but one of them is nevertheless assigned to sector 2 which contains no other particle, the two abundance equations will be solved and should give the same result as the single abundance equation, that is $Y(\text{heavier particles}) = 0$ and $Y(\text{lightest particle}) = Y$ of the single equation.

Finally, in the cospattering phase kinetic equilibrium may be lost; it is the responsibility of the user to verify that this is not the case for the parameters considered, so that the momentum-integrated Boltzmann equations remain a good approximation.

8.6 Scenario with late decaying field

In the early Universe, a field ϕ ⁷ which decays into SM radiation will inject entropy in the SM bath and modify the evolution of the DM density. The dynamics of the background is driven by the set of Boltzmann equations for the inflaton energy density ρ_ϕ and the SM entropy density, [61]

$$\frac{d\rho_\phi}{dt} = -\Gamma\rho_\phi - 3H\rho_\phi \quad (32)$$

$$\frac{ds}{dt} = \Gamma\frac{\rho_\phi}{T} - 3Hs \quad (33)$$

where Γ is the width of the late decaying field ϕ and $s = \frac{2\pi^2}{45}h_{eff}(T)T^3$. The Hubble expansion rate is given by

$$H = \sqrt{\frac{8\pi}{3} \frac{\rho_R + \rho_\phi}{M_P^2}} = \sqrt{H_{SM}^2 + H_I^2} \quad (34)$$

and the SM energy density, ρ_R , by

$$\rho_R = \frac{\pi^2}{30}g_{eff}(T)T^4. \quad (35)$$

The function

•getInflDecay(HI, Gamma, &Trh, &Tmax, &aEnd)

solves Eqs. (32) and (33). The solution is stored in the tabulated functions **Ta(a)** and **Ha(a)**, which respectively return the temperature and the Hubble parameter as a function of the scale parameter $a \in [1, \text{aEnd}]$. Moreover the functions **rhoIa(a)** and **rhoSMa(a)** return the energy density of the inflaton and SM bath respectively. These functions are available to the user. Here, **HI** is the Hubble rate at the end of inflation when **HI** is generated by the inflaton only. **Gamma** is Γ_ϕ , the decay width of the inflaton. **Trh**, **Tmax** and **aEnd** are return parameters. They represent respectively the temperature when the energy density of inflation becomes equal to the energy density of SM particles, the maximal temperature reached, and the expansion factor at the temperature **Tend**. This temperature is defined by the user as a global parameter. The function **getInflDecay** returns 0 when it can find a solution to the differential equations or 128 otherwise.

⁷We will assimilate the field ϕ with the inflaton, however our treatment is general and apply to any late decaying field.

The function

• **aT(T)**

returns the scale parameter a at a given temperature $T < \text{Tmax}$, $\text{aT}(T) = \text{NaN}$ when T exceeds Tmax . In general for $T < \text{Tmax}$ there are two values of a corresponding to a given temperature. **aT** returns the largest of them which corresponds to the cooling time of the Universe.

The function

• **getInflDecayPlus(Hsm, HI, Gamma, alpha, omega, &Trh, &Tmax, &aEnd)**

is an extended version of **getInflDecay** above. Here **Hsm** is the SM bath contribution to the Hubble rate, Eq. 34, the parameters **alpha** and **omega** (α and ω) modify the evolution of the scalar field,

$$\frac{d\rho_\phi}{dt} = -\Gamma(a)\rho_\phi - 3(1+\omega)H\rho_\phi \quad (36)$$

and Γ now depends on the scale parameter a

$$\Gamma(a) = \Gamma a^{-\frac{8\alpha-3(1+\omega)}{2}} \quad (37)$$

The evolution of the Dark Matter density n is given by the equation

$$\frac{dn_\chi}{dt} + 3Hn_\chi = -\langle v\sigma \rangle (n_\chi^2 - \bar{n}_\chi^2) + b\rho_\phi \quad (38)$$

which can be conveniently rewritten in terms of $z_\chi = n_\chi a^3$

$$a \frac{dz_\chi}{da} = -\frac{1}{Ha^3} (\langle v\sigma \rangle (z_\chi^2 - \bar{z}_\chi^2) - b\rho_\phi a^6) \quad (39)$$

where

$$b = \frac{1}{m_\phi} \sum_k k Br(\phi \rightarrow k\chi) \quad (40)$$

and we assume $bm_\phi \ll 1$.

To solve for the relic density, we have two functions:

• **darkOmegaInfl(b, Beps, &Tfo, &err)**

solves the DM evolution Eq. (39) where **b** is the branching fraction for the decay of the inflaton into N DM particles divided by the inflaton mass in GeV, see Eq. (40). This function uses **Ta**, **rhoIa**, and **Ha** obtained by **getInflDecay** or **getInflDecayPlus**. The main return value of this routine is the DM relic density $\Omega_\chi h^2$. The auxiliary output parameters are *double* **Tfo** and *integer* **err**. The parameter **Tfo** returns the freeze-out temperature. Recall that in freeze-out the final relic density decreases when the annihilation cross section increases, otherwise **Tfo**=NaN.⁸ The error code **err** indicates whether there is a problem with the solution of the differential Eqs.(33) and (32), in which case the 8th bit of the error code **err** is 1, or with the solution of Eq. (39), then the 9th bit is 1. The lowest bits contain information about problems with numerical integration and should be treated as warnings. The parameter **Beps** has the same meaning as in other **darkOmega** routines; it determines the condition to include Boltzmann suppressed channels [2]. The DM abundance is stored in an array and is available through the functions **Za(a)**, **Ya(a)** while the equilibrium abundance is stored in **ZaEq(a)**, **YaEq(a)**.

⁸The C-function *isfinite* checks the number and returns 0 in the case of NaN.

- `darkOmegaInfl(b, Beps, &err)`

solves the DM evolution for N -component DM [62]. This function has the same argument and output as above except for `Tfo`, this parameter is not included because some components can behave as WIMPs while others are FIMPs. The DM abundance is stored via the functions `ZaN(a,ch)`, `YaN(a,ch)`, while the equilibrium abundance is stored in `ZaNEq(a,ch)`, `YaNEq(a,ch)` where `ch` is a text which specifies the channel, for example `ch="2"`. `b` represents an array of coefficients that describe the branching fraction of the inflaton decay into DM components, see Eq. (40). In particular, the first coefficient (that is, `b[0]`) corresponds to the branching fraction of ϕ into the first DM component CDM[1].

The user has the possibility to define the parameters that enter `getInflDecayPlus` in the input file. It can be done using the following records with numerical values, see example in `IDM/infl2.par`. The input parameter of the `darkOmegaInfl` function can be defined in the same manner.

```

#! HsmInfl ...
#! HInfl ...
#! GammaInfl ...
#! wInfl ...
#! alphaInfl ..
#! BrInfl ...

```

where `HsmInfl` and `HInfl` correspond to H_{SM} and H_I in Eq. 34, `GammaInfl` is Γ in Eq. 33, `wInfl`, `alphaInfl` are w, α in Eq. 37 and `BrInfl` is b in Eq. 40.

8.7 Calculation of relic density for freeze-in

Several routines are provided in `micrOMEGAs` to compute the DM abundance in freeze-in scenarios. These can be found in the file `sources/freezein.c`. The first line of this file contains the statement

```
//#define NOSTATISTICS
```

This statement can be uncommented for `micrOMEGAs` to compute the relic density assuming a Maxwell-Boltzman distribution. This option is faster.

The auxiliary functions that are needed for the computation of the factors from statistical quantum mechanics are

- `Stat2(P/T, xY, x1, x2, η1, η2)`,

returns the S function defined in Eq. (41), that takes into account particle statistical distributions for the decay of a mediator $Y \rightarrow a, b$ of fixed momentum P .

$$S(P/T, x_Y, x_a, x_b, \eta_a, \eta_b) = \frac{1}{2} \int_{-1}^1 dc_\theta \frac{e^{E_Y/T}}{(e^{E_a(c_\theta)/T} - \eta_a)(e^{E_b(c_\theta)/T} - \eta_b)} . \quad (41)$$

where $x_i \equiv m_i/T$, E_a, E_b are the energies of the outgoing particles and $\eta_i \equiv \pm e^{\mu_i/T}$ and μ_i is the chemical potential.

- `K1to2(x1, x2, x3, η1, η2, η3)`,

returns the $\tilde{K}_1$ function defined in Eq. (42), that takes into account particle statistical

distributions.

$$\tilde{K}_1(x_1, x_2, x_3, \eta_1, \eta_2, \eta_3) \equiv \frac{1}{(4\pi)^2 p_{\text{CM}} T} \int \prod_{i=1}^3 \left(\frac{d^3 p_i}{E_i} \frac{1}{e^{E_i/T} - \eta_i} \right) e^{E_1/T} \delta^4(P_1 - P_2 - P_3) \quad (42)$$

The code does *not* check whether or not a particle is in thermal equilibrium with the SM thermal bath and that it is the responsibility of the user to specify which particles belong to the bath, $\mathcal{B}$, or are out of equilibrium, $\mathcal{F}$. This can be done through the function

- `toFeebleList(particle_name)`

which assigns the particle `particle_name` to the list of feebly interacting ones (*i.e.* those which belong to $\mathcal{F}$). Feebly interacting particles can be odd or even. This function can be called several times to include more than one particle. All odd or even particles that are not in this list are assumed to be in thermal equilibrium with the SM and belong to $\mathcal{B}$. The treatment of the particles that belong to $\mathcal{F}$ for the computation of Ωh^2 within the freeze-in routines is described below. Calling `toFeebleList(NULL)` will reassign all particles to $\mathcal{B}$.

The actual computation of the freeze-in DM abundance can be performed with the help of three functions:

- `darkOmegaFiDecay(TR, bathParticle, feebleParticle)`

calculates the abundance of `feebleParticle` resulting from the decay of `bathParticle`. The equations used for the three different cases are described in [10].

For the freeze-in routines that compute $2 \rightarrow 2$ processes described below (`darkOmegaFi` and `darkOmegaFi22`, the effect of the thermal masses of particles in the t/u - channel can be simulated by introducing a cut on the Mandelstam variables t or u . The thermal masses, M_T , are defined by the user in the array `Tkappa`,

$$M_T = T \text{Tkappa}[\mathbf{k}] \quad (43)$$

where $\mathbf{k}$ is the internal particle number. This number can be obtained by

$$\mathbf{k} = \text{abs}(\text{pTabPos}(\text{pName})) - 1$$

The cut is applied on $t > t_{\text{max}} - M_T^2$ for a t-channel propagator and on $u > u_{\text{max}} - M_T^2$ for a u-channel propagator. By default `Tkappa[k]=0` for all particles.

- `darkOmegaFi22(TR, Process, feebleParticle, &err)`

calculates the DM abundance of `feebleParticle` taking into account only $2 \rightarrow 2$ `Process`. For example `"b,B ->~x1,~x1"` for the production of DM (here $\sim x1$) via $b\bar{b}$ scattering. This routine allows the user to extract the contribution of individual processes. `TR` is the reheating temperature. `err` is the returned error code, it has the following meaning

- 1: the requested processes does not exist
- 2: $2 \rightarrow 2$ process is expected
- 3: can not calculate local parameters // some constrain parameters can not be calculated.
- 4: $T_{\text{end}} \geq T_R$, or $T_{\text{end}} = 0$
- 5: one of the incoming particles belong to $\mathcal{F}$.
- 6: None of the outgoing particles are odd and feeble.

- 7: Lost of precision in temperature integrand
- 8: Pole in temperature integrand
- 9: NAN in temperature integrand
- 10: Lost of precision in sqrt(s) integrand
- 11: Pole in sqrt(s) integrand
- 12: NAN in sqrt(s) integrand
- 13: Lost of precision in angle integrand
- 14: Pole in angle integrand
- 15: NAN in angle integrand
- 16: lost of precision caused by diagram cancellation

- **darkOmegaFi(TR, feebleParticle &err)**

calculates the DM abundance after summing over all $2 \rightarrow 2$ processes involving particles in the bath $\mathcal{B}$ in the initial state and at least one particle in $\mathcal{F}$ in the final state. The routine checks the decay modes of all bath particles and if one of them has no decay modes into two other bath particles, the $2 \rightarrow 2$ processes involving this particle are removed from the summation and instead the contribution to the DM abundance computed from the routine **darkOmegaFiDecay** is included in the sum. This is done to avoid appearance of poles in the corresponding $2 \rightarrow 2$ cross-section. We recommend for such models to compute individual $2 \rightarrow 2$ processes with **darkOmegaFi22** described above. As before, we assume that all odd FIMPs will decay into the lightest one which is the DM. T_R has the same meaning as above. **err** is the returned error code, **err=1** if feeble particles have not been defined.

- **printChannelsFi(cut, prcnt, filename)**

writes into the file **filename** the contribution of different channels to Ωh^2 . The **cut** parameter specifies the lowest relative contribution to be printed. If **prcnt** $\neq 0$, the contributions are given in percent. The routine **darkOmegaFi** fills the array **omegaFiCh** which contains the contribution of different channels ($2 \rightarrow 2$ or $1 \rightarrow 2$) to Ωh^2 . **omegaFiCh[i].weight** specifies the relative weight of the *i*th channel, **omegaFiCh[i].prtel[j]** (*j*=0, 4) defines the particles names for the *i*th channel. The last record in the array **omegaFiCh** has zero weight and NULL particle names.

The temperature evolution of the abundances generated by these three routines can be obtained by calling the function **YFi(T)** in the interval $T \in [T_{\text{end}}, T_{\text{start}}]$ where **Tstart** is set internally by the three routines and is assigned to the variable **TR**.

Note that if no particle has been declared as being feebly interacting, the freeze-out routines **darkOmega**, **darkOmegaF0**, and **darkOmega2** [8] will work exactly like in previous versions of **micrOMEGAs**. A non-empty **feeblelist**, however, will affect these routines since **micrOMEGAs** will exclude all the particles in this list from the computation of the relic density via freeze-out. To compute the relic density of particles in **feeblelist** one has to use the **darkOmegaFi** function (or the other routines describe in this section) for each of these particles. Alternatively the relic density of feeble particles can be computed using the **darkOmega*TR** routines described in section 8.4,8.5 by setting the initial abundance of these particles to zero. Note that the result can differ from the one obtained by the

darkOmegaFi routines since these take into account the Maxwell-Boltzmann statistics. ⁹

9 Direct detection

9.1 Amplitudes for elastic scattering

- **nucleonAmplitudes(CDM,pAsi,pAsd,nAsi,nAsd)**

calculates the amplitudes for CDM-nucleon elastic scattering at zero momentum. **pAsi(nAsi)** are spin independent amplitudes for protons(neutrons) whereas **pAsd(nAsd)** are the corresponding spin dependent amplitudes. Each of these four parameters is an array of dimension 2. The zeroth (first) element of these arrays gives the χ -nucleon amplitudes whereas the second element gives $\bar{\chi}$ -nucleon amplitudes. Amplitudes (in GeV^{-2}) are normalized such that the total cross section for either χ or $\bar{\chi}$ cross sections is

$$\sigma_{tot} = \frac{4M_\chi^2 M_N^2}{\pi(M_\chi + M_N)^2} (|A^{SI}|^2 + 3|A^{SD}|^2) \quad (44)$$

nucleonAmplitudes depends implicitly on form factors which describe the quark contents in the nucleon. These form factors are global parameters (see Table 1 for default values)

$$TypeFFPq \quad TypeFFNq$$

where *Type* is either "Scalar", "pVector", or "Sigma", FFP and FFN denote proton and neutron and *q* specifies the quark, *d*, *u* or *s*. Heavy quark coefficients are calculated automatically. The form factors can be changed individually or recalculated all together with

- **calcScalarQuarkFF($m_u/m_d, m_s/m_d, \sigma_{\pi N}, \sigma_s$)**

computes the scalar coefficients for the quark content in the nucleon from the quark mass ratios $m_u/m_d, m_s/m_d$ as well as from $\sigma_{\pi N}$ and σ_s . The default values given in Table 2 are obtained for $\sigma_s = 42\text{MeV}, \sigma_{\pi N} = 34\text{MeV}, m_u/m_d = 0.56, m_s/m_d = 20.2$ [63]. The function **calcScalarQuarkFF(0.553, 18.9, 55., 243.5)** will reproduce the default values of the scalar quark form factors used in micrOMEGAs2.4 and earlier versions.

micrOMEGAs automatically takes into account loop contributions from box diagrams as calculated in [64] (DM spin 1/2 case) and [65] (DM spin 0 and 1 cases).

nucleonAmplitudes does not take into account an operator involving DM-DM-gluon-gluon interactions. It is by default assumed that such interactions are generated at one-loop from DM interactions with heavy colored particles and this loop-induced contribution of heavy coloured particles to DM-DM-gluon-gluon interactions ¹⁰ is taken into account automatically. Thus a direct contribution might lead to double counting. Note that to avoid double counting an Hgg vertex in the model file will also be ignored. To include the contribution of an additional operator that involves DM interactions with gluons, it is rather recommended to introduce a new auxiliary heavy color (dim=3) fermion in the model file, the direct detection amplitude that results from the effective operator $\alpha_s G_{\mu\nu} G^{\mu\nu} \bar{\chi} \chi$ can be represented by a term $-12\pi M_F \bar{F} F \bar{\chi} \chi$, where M_F is the fermion

⁹Note that in previous versions of micrOMEGAs the relic density of **darkOmegaFi** was rescaled when it was not the lightest particle in the dark sector to take into account the fact that the FIMP will eventually decay into the DM. In this version this rescaling has to be done by the user.

¹⁰Heavy quarks include c,b,t quarks

mass which should be chosen large enough to ensure that F does not contribute to other processes.

In general we assume that DM-nucleon interactions do not depend on the recoil energy. This assumption is not valid when DM-quark elastic scattering proceeds through a light mediator. In this case, one cannot make a direct comparison of Eq.44 with the limits on DM-nucleon cross section provided by experiments. `micrOMEGAs` treats differently two cases with a light mediator, the photon mediator which can induce milli-charge, dipole or quadruple interactions and other mediators. For the photon mediator, the infinite contribution to amplitudes is subtracted and `nucleonAmplitudes` returns an error code 4. If there is another mediator lighter than 100MeV which contributes to SI or SD amplitudes, the error code is 1 or 2. The contribution of the light mediators to DM scattering on nucleus will however be taken into account as described next.

9.2 Scattering on nuclei

• `nucleusRecoil(f,A,Z,J,Sxx,dNdE)`

This is the main routine of the direct detection module. The input parameters are:

- ◇ `f` - the DM velocity distribution normalised such that

$$\int_0^\infty f(v)dv = 1 \quad (45)$$

The units are km/s for v and s/km for $f(v)$.

- ◇ `A` - atomic number of nucleus;
- ◇ `Z` - number of protons in the nucleus;
- ◇ `J` - nucleus spin;
- ◇ `void Sxx(double p, double*S00,double*S01,double*S11)` - is a routine which calculates nucleus form factors for spin-dependent interactions (`S00,S01,S11`), it depends on the momentum transfer p in fm^{-1} . For nucleus with zero spin one has to substitute `NULL` for this parameter;
- ◇ The distribution over recoil energy is stored in the array `dNdE`

$$dNdE[n] = \frac{dN}{dE_n}$$

in units of (1/keV/kg/day) where

```
En=RE_START*RE_STEP**n;    0<=n<RE_DIM
#define RE_DIM      150    // dimension of recoil energy array
#define RE_START    1.E-2  // energy [keV] corresponding to zero'th element
#define RE_STEP     1.08   // factor for recoil energy grid
                          // E_{n+1}=E_n*RE_STEP
```

The `include/micromegas.h` file contains predefined values for charges and spins of atomic nucleus. They are called with $Z_{\{Name\}}$ and $J_{\{Name\}}\{atomic_number\}$ where $Name$ is the name of an atom. For example, for ^{73}Ge , a call to this routine will be:

`nucleusRecoil(Maxwell,73,Z_Ge,J_Ge73,SxxGe73,dNdE);`

The returned value of the function `nucleusRecoil` gives the number of events per day and per kilogram of detector material. The result depends implicitly on the global parameter `rhoDM`, the density of DM near the Earth. This routine is based on the spin-dependent and spin-independent cross sections, but also takes into account the modification of the recoil energy distribution caused by a light t- channel propagator. For a light mediator, the nucleus recoil energy distribution reads

$$\frac{dN_A}{dE} = \frac{M_{\text{med}}^4}{(M_{\text{med}}^2 + 2M_A E)^2} \frac{dN_A^{\text{std}}(M_{DM}, \sigma_0)}{dE} \quad (46)$$

where N_A^{std} is the standard expression for the number of recoil events for a point-like interaction with a DM- nucleon elastic scattering cross-section σ_0 , M_{med} is the mass of the t-channel mediator. The factor to correct the recoil energy distribution as in Eq.46 is introduced automatically in the code after an internal check for the existence of a light mediator. Note that the recoil energy distribution in a model with a light mediator is different from the one used in analysis of direct detection experiments, therefore it is necessary to recast the results of the DD experiment to determine whether a model is excluded or not, see Section 9.3.2.

For a complex WIMP and if DM has only one component, `nucleusRecoil` averages over χ and $\bar{\chi}$ taking into account the asymmetry between χ and $\bar{\chi}$. For models with 2 DM particles the result takes into account the relative contribution of each DM particle through the parameter `fracCDM2`.

- `dNdERecoil(E,dNdE)` interpolates the dNdE table.

9.2.1 Nucleus form factors

The form factors for the spin independent (SI) cross section are defined by the Fermi distribution

$$F_A(q) = \int e^{-qx} \rho_A(|x|) d^3x \quad (47)$$

$$\rho_A(r) = \frac{c_{\text{norm}}}{1 + \exp((r - R_A)/a)} \quad (48)$$

$$R_A = cA^{\frac{1}{3}} + b \quad (49)$$

where a, b, c are defined with the global parameters `Fermi_a`, `Fermi_b`, `Fermi_c`.

The spin dependent form factors collected in [66] are implemented in micrOMEGAs [5]:

```
SxxF19   SxxNa23   SxxNa23A   SxxAl27   SxxSi29   SxxSi29A
SxxK39   SxxGe73   SxxGe73A   SxxNb92   SxxTe125   SxxTe125A
SxxI127   SxxI127A   SxxXe129   SxxXe129A SxxXe129M
SxxXe131 SxxXe131A SxxXe131B SxxXe131Me
```

Here the characters after the atomic number are used to distinguish different implementations of the form factor for the same isotope. Recently we have added some form factors used in XENON1T and PICO-60 experiments. See Table 9.

Name	reference	ID
SxxF19EF SxxXe131EFT SxxXe129EFT	[67]	EFT
SxxF19SHELL SxxXe129SHELL SxxXe131SHELL	[68]	SHELL
SxxF19SHELLm SxxXe129SHELLm SxxXe131SHELLm	[68]	SHELLm

Table 9: Implemented form factors

The minimal and maximal values for the SD form factors, $S_{00}(q), S_{01}(q), S_{11}(q)$, are computed in Ref. [68] within the shell model. The ID **SHELL** corresponds to the average

$$S_{ab} = (S_{ab}^{min} + S_{ab}^{max})/2 \quad (50)$$

which are obtained from the minimum and maximum fitted values in Table VI in [68].

The ID **SHELLm** corresponds to the form factors which lead to the most robust exclusion. Since S_{ab}^{min} often leads to a negative value for the subdominant component of the form factor, and this has no physical meaning, we use rather the minimum value of the proton-only, S_p^{min} , and neutron-only, S_n^{min} , form factors also given in [68]. These correspond to the minimal form factor for the case when only one type of interaction (with proton or neutron) is included. With this we construct the nucleus form factors

$$\begin{aligned} S_{00} &= \frac{1}{4} \left(S_p^{min} + S_n^{min} \pm 2\sqrt{S_p^{min} S_n^{min}} \right) \\ S_{11} &= \frac{1}{4} \left(S_p^{min} + S_n^{min} \mp 2\sqrt{S_p^{min} S_n^{min}} \right) \\ S_{01} &= \frac{1}{2} (S_p^{min} - S_n^{min}) \end{aligned} \quad (51)$$

The sign in Eq.51 is chosen to reproduce the ratio $S_{00}(0)/S_{11}(0)$ for the central value of the form factors in Ref. [68].

For nuclei whose form factors are not known one can use the routine

• **nucleusRecoil0(f,A,Z,J,Sp,Sn,dNdE)**

which is similar to the function **nucleusRecoil** except that the spin dependent nuclei form factors are described by Gauss functions [5] whose values at zero momentum transfer are defined by the coefficients **Sp,Sn**. Predefined values for the coefficients **Sp,Sn** in the format

$$\begin{aligned} Sp_{-}\{Nucleus\ Name\}\{Atomic\ Number\} \\ Sn_{-}\{Nucleus\ Name\}\{Atomic\ Number\} \end{aligned}$$

are presented in the file `include/micromegas.h`. For example,

```
#define Sn_He3      0.552
#define Sn_017      0.5
```

9.2.2 Velocity distribution

Ignoring the direction of motion of DM particles and the small effect of DM acceleration by the gravitational field of the Sun, the DM velocity distribution in the vicinity of the direct detection experiment is given by

$$f(\mathbf{v}) = \int_{|\vec{v}| < \mathbf{vEsc}} d^3\vec{v} F_G(\vec{v} - \vec{v}_{Earth}) \delta(\mathbf{v} - |\vec{v}|) \quad (52)$$

where F_G is the DM velocity distribution in the frame of the galaxy, $\vec{v}_{Earth}$ is the velocity of the Earth in the Galaxy and $\mathbf{vEsc}$ is the maximal DM velocity in the Sun's orbit due to the finite gravitational potential of our Galaxy. $\mathbf{vEsc}$ and $\mathbf{vEarth} = |\vec{v}_{Earth}|$ are global parameters of micrOMEGAs.

The velocity distributions that are available in `micrOMEGAs` are the following

- **Maxwell(v)**

returns

$$F_G^M(\mathbf{v}) = c_{\text{norm}} \frac{1}{(2\pi \mathbf{vRot}^2)^{3/2}} \exp\left(-\frac{(\vec{v})^2}{\mathbf{vRot}^2}\right) \theta(\mathbf{vEsc} - |\vec{v}|) \quad (53)$$

which corresponds to the isothermal model. Here $\mathbf{vRot}$ is the orbital velocity of stars in the Milky Way, it is also a global parameter of micrOMEGAs. c_{norm} is the normalization factor,

$$c_{\text{norm}}^{-1} = \text{erf}\left(\frac{\mathbf{vEsc}}{\mathbf{vRot}}\right) - \frac{2}{\sqrt{\pi}} \frac{\mathbf{vEsc}}{\mathbf{vRot}} \exp\left(-\frac{\mathbf{vEsc}^2}{\mathbf{vRot}^2}\right) \quad (54)$$

- **SHMpp(v)**

returns the velocity distribution **SHM++** proposed in [69].

$$F_G(\vec{v}) = (1 - \eta) F_G^M(\mathbf{v}) + \eta F_G^S(\mathbf{v}) \quad (55)$$

This distribution consists of two components. The first, $F_G^M(\vec{v})$, is the standard Maxwell velocity distribution described above. The second component is the velocity distribution from the *Gaia* sausage [70, 71], it is not spherically symmetric and is defined by the anisotropy parameter β with

$$F_G^S(\vec{v}) = \frac{c_{\text{norm}}}{(2\pi)^{3/2} \Delta v_r \Delta v_\theta \Delta v_\phi} \exp\left(-\left(\frac{v_r}{\Delta v_r}\right)^2 - \left(\frac{v_\theta}{\Delta v_\theta}\right)^2 - \left(\frac{v_\phi}{\Delta v_\phi}\right)^2\right) \theta(\mathbf{vEsc} - |\vec{v}|) \quad (56)$$

where

$$\Delta v_r = \frac{\mathbf{vRot}}{\sqrt{1 - \frac{2}{3}\beta}}, \quad \Delta v_\phi = \Delta v_\theta = \frac{\mathbf{vRot} \sqrt{1 - \beta}}{\sqrt{1 - \frac{2}{3}\beta}} \quad (57)$$

and

$$c_{\text{norm}}^{-1} = \text{erf}\left(\frac{\mathbf{vEsc}}{\mathbf{vRot}}\right) - \left(\frac{1 - \beta}{\beta}\right)^{1/2} \exp\left(-\frac{\mathbf{vEsc}^2}{\mathbf{vRot}^2}\right) \text{erfi}\left(\frac{\mathbf{vEsc}}{\mathbf{vRot}} \frac{\beta^{1/2}}{(1 - \beta)^{1/2}}\right) \quad (58)$$

where erfi is the imaginary error function.

The central values and uncertainties of the **SHM++** parameters are

$$\begin{aligned}
\rho_{DM} &= 0.55 \pm 0.17 \text{ GeV/cm}^3 \\
v_{Rot} &= 233 \pm 3 \text{ km/s} \\
v_{Esc} &= 580 \pm 63 \text{ km/s} \\
\beta = \text{betaSHMpp} &= 0.9 \pm 0.05 \\
\eta = \text{etaSHMpp} &= 0.2 \pm 0.1
\end{aligned} \tag{59}$$

Note that these central values for the global parameters, **vRot**, **vEsc** and **rhoDM** are different from the ones in Table 1, thus the user has to change these parameters before using **SHMpp**.

9.3 Comparison with Direct Detection experiments

Examples on how to use the routines described below to impose constraints from direct detection experiments can be found in `mdlIndep/dd_exp.c` of **micrOMEGAs**.

9.3.1 Experimental data

The SI 90% DD limits tabulated from the results presented by LZ5T [72], XENON1T [73], DarkSide-50 [74], PICO-60 [75] and CRESST-III [76] are accessible through the following functions

[SI] name	region[GeV]	cite	[SD] name	region[GeV]	cite
XENON_NT	$3 < M_{DM} < 12$	[77]	PICO60_SDp	$3 < M_{DM}$	[75]
PandaXS2	$2 < M_{DM} < 6$	[78]	LZ5T_SDp	$9 < M_{DM}$	[72]
LZ5T	$9 < M_{DM}$	[72]	LZ5T_SDn	$9 < M_{DM}$	[72]
PandaX4T	$5 < M_{DM}$	[79]	XENON1T_SDp	$6 < M_{DM}$	[80]
XENON1T	$6 < M_{DM}$	[73]	XENON1T_SDn	$6 < M_{DM}$	[80]
DS50	$0.7 < M_{DM} < 15$	[74]	CRESST_III_SDn	$0.35 < M_{DM} < 12$	[76]
PICO60	$3 < M_{DM}$	[75]			
CRESST_III	$0.35 < M_{DM} < 12$	[76]			

These functions give the excluded cross sections in cm^2 units . For a DM mass outside the range specified the function returns NaN. For experiments that present an exclusion limit for masses larger than the masses of nuclei in the detector, we use the extrapolation

$$cs(M) = cs(M_{max}) \frac{M}{M_{max}}$$

We also have included the functions

- **Darwin_XLZD(M)** which presents the projections for DM exclusion with the future experiment **Darwin** [81] and
- **Neutrino_FloorXeSI(M)** which gives the neutrino floor limit for Xenon-like experiments [82].

9.3.2 Recasting the experimental limits with **micrOMEGAs**

For the cases of light mediators, multi-component DM or non-standard DM velocity distribution, the limits on the DM-nucleon cross-section obtained by direct detection experiments cannot be applied directly. It is therefore necessary to perform a recasting of each

DD experiment and recalculate the experimental limit corresponding to a non-standard recoil energy spectrum. The recasting for XENON1T, DS50, PICO60, and CRESST-III was done in [83], in this version we have added the recasting of LZ5T experiment. Fig. 1(left) demonstrates the quality of our recasting for SI interaction. Note that our exclusion limit is slightly weaker in the range 10-30 GeV.

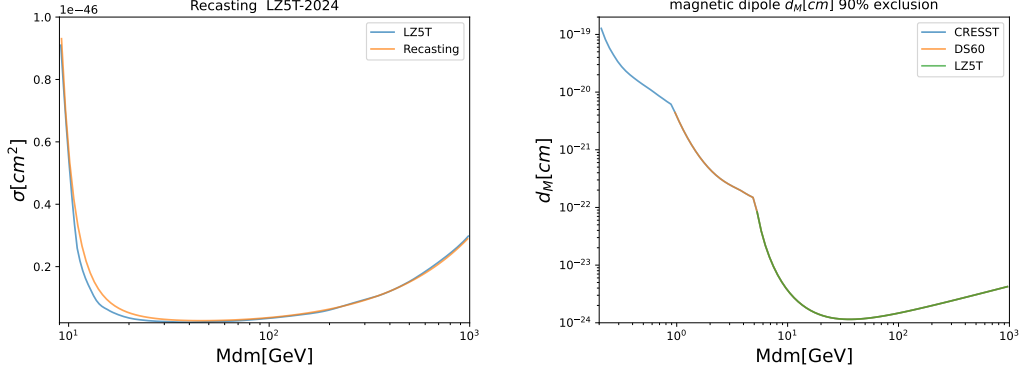


Figure 1: Left: recasting of LZ5T experiment. The orange curve is obtained by the `DD_factorCS` routine. Right: Exclusion of magnetic monopole by direct detection experiments.

- `DD_pvalCS(expCode, fv, σSIP, σSIN, σSDP, σSDN, &expName)`
calculates the value $\alpha = 1 - C.L.$ for a model with DM-nucleon cross sections $\sigma_{SI_P}, \sigma_{SI_N}, \sigma_{SD_P}, \sigma_{SD_N}$. Cross sections are specified in [pb] units. The return value 0.1 corresponds to a 90% exclusion. The `expCode` parameter can be any of the codes `LZ5T`, `XENON1T_2018`, `DarkSide_2018`, `CRESST_2019`, `PICO_2019` or their combination concatenated with the symbol `|`. There is also a predefined parameter that currently combines these experiments

`AllDDexp=LZ5T_2024|XENON1T_2018|DarkSide_2018|PICO_2019|CRESST_2019;`

The parameter `char* expName` is used to indicate the experiment that provides the best exclusion among those specified in `expCode`. The function `DD_pvalCS` calculates the exclusion for each experiment independently, returns the smallest α , and assigns the name of the corresponding experiment to `expName` if it is not `NULL`.

The f_v parameter specifies the DM velocity distribution in the detector frame. For example, one can use `Maxwell` or `SHMpp` which are included in `micrOMEGAs`, otherwise the user can define another distribution. The DM velocity distribution has to be normalized as in Eq. 45. The units are km/s for v and s/km for $f_v(v)$. `DD_pvalCS` implicitly depends on the global parameters `Mcdm` and `rhoDM` which specify the DM mass and DM local density respectively.

For XENON1T one can choose between three recasting, p_{eff}^q with $q = 0, 1, 2$, see Ref. [83]. The flag `Xe1TnEvents=q` allows to choose the corresponding recasting, otherwise and by default the code uses p_{eff}^1 . The three approaches agree within 5%. For PICO-60, the user can choose between the recasting based on Feldman-Cousins statistics, `PICO60Flag=0` which is the default value, or the one based on Neyman one side belt exclusion, `PICO60Flag=1`.

- `DD_factorCS(expCode, α, fv, σSIP, σSIN, σSDP, σSDN, &expName)`
returns the overall factor which should be applied to the cross sections, $\sigma_{SI_P}, \sigma_{SI_N}, \sigma_{SD_P}, \sigma_{SD_N}$

to reach the exclusion level α . All parameters are the same as in `DD_pvalCS` above.

- `*dNdEFact(Enr_kev, A)`

is the address of the function which modifies the nucleus recoil distribution for `DD_pvalCS` and `DD_factorCS` to take into account a t-channel propagator with small or zero mass. By default `dNdEFact=NULL` and this function does not contribute to the calculation of the direct detection cross sections. Otherwise it is taken as an additional factor in the nucleus recoil distribution, see Eq.46. The parameter `Enr_kev` is the recoil energy in [keV] units, `A` is the atomic number of the nucleus. This function should be defined by the user, an example is given in `mdlIndep/dd_exp.c`.

- `DD_pval(expCode,  $f_v$ , &expName)`

- `DD_factor(expCode,  $\alpha$ ,  $f_v$ , &expName)`

These functions are similar to `DD_pvalCS` and `DD_factorCS` described above but use the cross section calculated from the DM model under consideration in `micrOMEGAs`. The necessary corrections for a light mediator are implemented automatically, these functions do not use `dNdEFact`.

9.3.3 Direct detection limit for photon exchange processes.

As mentioned above, the contribution of a photon mediator is subtracted from the A^{SI} amplitude. The DM-nucleus amplitude is obtained as the sum A^{SI} and the electromagnetic amplitude. The latter is obtained using the DM, DM^*, γ vertex found in the file for the Lagrangian and the Coulomb N, N^*, γ vertex for the nuclei N . In Fig.1 we show the limit on the DM magnetic dipole obtained from a recasting of different experiments. The DM model with electric milli-charge, magnetic and electric dipole is presented in the `dipoleDM` model included in `micrOMEGAs`.

9.3.4 Setting spin-dependent form factors

The spin dependent form factors for XENON1T and PICO60 are defined via the parameters

`XENON1T_2018_sdXe129` `XENON1T_2018_sdXe131` `PICO_2019_sdF19`

The default values for these parameters correspond to the ones used by the experiments, namely

`SxxXe129SHELL` `SxxXe131SHELL` `SxxF19EFT`

They can be replaced by any form factor listed in Section 9.2.1 through direct assignment. Form factors can also be changed using the routine

- `setSpinDepFF(ExperimentID, setID)`

where the choice for `ExperimentID` is given in Section 9.3.2 and `setID` can be

`EFT` - corresponding to the form factors in [67]

`SHELL` - corresponding to the average form factors in [68], Eq. 50

`SHELLm` - corresponding to the minimal form factor of [68], Eq. 51. See section 9.2.1.

10 Indirect detection

10.1 Interpolation and display of spectra

Various spectra and fluxes of particles relevant for indirect detection are stored in arrays with **NZ** elements. The first (zeroth) element of the array contains the maximum energy E_{max} . As a rule E_{max} is the mass of the DM particle. The i^{th} element ($1 \leq i \leq NZ-1$) of the spectrum array contains the value of $E_i \frac{dN}{dE_i}$ where $E_i = E_{max} e^{Zi(i)}$, $Zi(i) = -7 \log(10) \left(\frac{i-1}{250}\right)^{1.5}$. Here E is the kinetic energy $E = \sqrt{p^2 + m^2} - m$. By default **NZ=250**, thus the array covers the energy interval $E_{max} \geq E > 1.101 \times 10^{-7} E_{max}$. The user can change the value of **NZ** defined in the file `include/micromegas.h` in order to treat smaller energies as described in section 10.2. To decode and interpolate the spectrum array one can use the following functions:

- **SpectdNdE(E,spectTab)**

interpolates the tabulated spectra and returns the particle distribution dN/dE where E is the energy in GeV. For a particle number distribution the returned value is given in GeV^{-1} while a particle flux is expressed in $(\text{sec cm}^2 \text{ sr GeV})^{-1}$.

- **eSpectdNdE(E,spectTab)**

returns $E \times \text{SpectdNdE}(E, \text{spectTab})$.

- **addSpectrum(SpectTab,toAdd)**

sums the spectra `toAdd` and `SpectTab` and writes the result in `SpectTab`. For example, this routine can be useful for summing spectra with different maximal energy.

- **spectrMult(SpectTab, func)**

allows to multiply the spectrum `SpecTab` by any energy dependent function `func`

- **spectrInt(Emin,Emax,SpecTab)**

integrates a spectrum/flux, `SpecTab` from `Emin` to `Emax`.

- **spectrInfo(Emin,Spec,&Etot)**

provides information on the spectra. The returned value and `Etot` corresponds respectively to

$$N_{tot} = \int_{E_{min}}^{E_{max}} \text{SpectdNdE}(E, \text{Spec}) dE = \text{spectrInt}(E_{min}, E_{max}, \text{Spec}) \quad (60)$$

$$E_{tot} = \int_{E_{min}}^{E_{max}} E \text{SpectdNdE}(E, \text{Spec}) dE \quad (61)$$

where the first element of the table `Spec` contains the value of E_{max} . Alternatively the user can directly integrate the spectra using standard simpson routines, for example

$$N_{tot} = \text{simpson_arg}(\text{SpectdNdE}, \text{Spec}, E_{min}, \text{Spec}[0], 0.01, \text{NULL}) \quad (62)$$

$$E_{tot} = \text{simpson_arg}(\text{eSpectdNdE}, \text{Spec}, E_{min}, \text{Spec}[0], 0.01, \text{NULL})$$

The spectrum can be displayed on the screen with
`displayPlot("Spectrum","E",E_min, Spec[0],0,1,"dNdE",0, SpectdNdE,Spec)`

- `fillSpect(dNdE, Emax, SpectArray)`

This routine converts the spectrum `dNdE` which is a function of energy E into an array `SpectArray` which contains NZ elements. The `Emax` parameter is assigned to `SpectArray[0]`.

- `FSRdNdE(E, p, m, q, spin2, med)`

computes the Final State Radiation spectrum, $\frac{dN}{dE_\gamma}$, for DM annihilation in charged particles with momentum $\mathbf{p}$ and mass m . The `spin2` parameter is 0 for bosons and 1 for fermions. The parameter `med=0` if the particle and antiparticle are produced via a scalar mediator and 1 for a vector mediator. E is the energy of the photon. Here we use formulas 4.8 - 4.11 of Ref. [84]. Note that `FSRdNdE` contains the photon spectrum generated by both particle and anti-particle.

To boost a spectrum one can use the command

- `boost( $\gamma$ , Emax, m, SpectTab)`

where γ is the boost parameter, `SpectTab` is an array of NZ lines which contains the initial spectrum for the particle of mass m . After calling the boost command the array `SpectTab` will be overwritten with the new spectrum. The first element of the resulting array is `SpectTab[0]=Emax` unless the maximal energy of the particle after boost, E , exceeds `Emax`, then `SpectTab[0]=E`.

10.2 Annihilation spectra

DM pair annihilation into SM particles will lead to stable particles

$$\gamma, e^+, p^-, \nu_e, \nu_\mu, \nu_\tau, D^- \quad (63)$$

The resulting spectra which take into account final state radiation, hadronisation and decays were calculated by Pythia and tabulated.

- `basicSpectra(Mass, pdg, outN, Spectr)`

computes the spectra of outgoing particles and writes the result in an array of dimension NZ , `Spectr`, `pdg` is the PDG code of the particles produced in the annihilation of a pair of WIMPs. `outN` specifies the outgoing particle,

$$\text{outN} = \{0, 1, 2, 3, 4, 5, 6\} \text{ for } \{\gamma, e^+, p^-, \nu_e, \nu_\mu, \nu_\tau, D^-\}$$

The `Mass` parameter defines the mass of the DM particle. `basicSpectra` returns an error code if the parameter `outN` (`err=1`) or `pdg` (`err=2`) is out of range. The spectrum depends on

- `SpectraFlag`

is a switch that allows to choose the tables that contain the spectra. The default value 0 corresponds to micrOMEGAs' spectra, while other values 1, 2, 3 activate spectra generated by external codes. Note that the spectra for the `Mass` parameter smaller than 5 GeV are available only in micrOMEGAs' tables while the antideuterons spectra are provided only within PPPC `SpectraFlag=2`.

SpectraFlag=0 corresponds to new tables for $\gamma, e^+, \bar{p}, \nu$ recently obtained for micrOMEGAs using Pythia8 for DM in the range $0.5\text{GeV} - 10\text{TeV}$.¹¹ These tables include also the spectra for polarized W's and Z's. To get the spectra generated by transverse and longitudinal W's substitute **pdgN= 24 +' T'** and **24 +' L'** correspondingly. In the same manner **pdgN= 23 +' T'** and **23 +' L'** provides the spectra produced by a polarized Z boson. We have also included the spectra corresponding to DM annihilation into $\pi^0, \pi^\pm, K^0, K^\pm$, and η mesons. The lower limit is defined by the particle mass.

SpectraFlag=1 corresponds to the spectra generated by *Pythia-8* [85, 86] for DM mass in the range $5\text{GeV} - 5\text{TeV}$, here polarization is not taken into account.

SpectraFlag=2 corresponds to the spectra generated with *PPPC* [87, 88] for DM mass larger than 5GeV . The spectra take into account polarized W and Z, as well as polarized leptons, *pgd= 11 +' R''* or *pgd= 11 +' L''*, the latter are however not supported in the current version of micrOMEGAs.

Finally **SpectraFlag=3** corresponds to the spectra provided in *CosmiXs* for DM in the range $5\text{GeV} - 100\text{TeV}$ [89]. These spectra are generated with PYTHIA and include electroweak corrections. These spectra also included longitudinal/transverse W and Z as well as left/right polarized charged leptons.

The *Pythia-8*, *PPPC*, and *CosmiXs* tables contain spectra that start at $x \gtrsim 10^{-9}$, while the default spectra **SpectraFlag=0** start at $x \gtrsim 10^{-7}$. To download the spectra including smaller energies one has to recompile micrOMEGAs after setting the parameter **NZ=295** in the file `include/micromegas.h`.

The maximal values of the x parameter in *Pythia-8* and *CosmiXs* tables are 0.94 and 0.9 respectively. For this reason, the spectral line for $DM, DM \rightarrow \gamma, \gamma$ which corresponds to a contribution $2\delta(1-x)$ in the photon spectra is not included, only the sub-dominant part of the photon spectra from photon radiation is included. In micrOMEGAs the δ -like contribution to photon spectra in the γ, γ channel are added to the *Pythia-8* and *CosmiXs* tables. In the same manner we add a $\delta(1-x)$ contribution to the neutrino spectra for the $\nu, \bar{\nu}$ channel. In *PPPC* the corresponding δ -like contributions are simulated by a linear function in the interval $x \in [0.89, 1]$.

The QCD uncertainties that can impact the annihilation spectra were evaluated in [86]. For each DM mass, annihilation channel and choice of final state, Ref. [86] have estimated three types of QCD uncertainties and provide tabulated spectra including these uncertainties. These tables are now incorporated in the code. We have defined three parameters that determines which uncertainties are taken into account,

- **DHad**: uncertainties on the flux from the variations of the hadronisation model parameters.
- **DScale**: shower uncertainties on the flux from the variation of the shower evolution variable by a factor of 2.
- **DcNS**: uncertainties from the variations of the non-singular terms of the DGLAP splitting kernels.

By default these parameters are set to 1 and all types of uncertainties are taken into account, however any one can be switched off by setting the corresponding parameter

¹¹These tables replace the original micrOMEGAs tables obtained with Pythia6, these can be accessed with **SpectraFlag=-1**

to zero. The total uncertainties $\Delta^2(E)_{min,max}$ are obtained by taking the sum of the square of the individual uncertainties. The global parameter, **spectUncert** determines whether or not uncertainties are included in the spectra. When **spectUncert=0**, the default spectra without uncertainties, $(dN/dE)_{wo}$, are used, **spectUncert=1** means that **basicSpectra** returns the maximal spectra corresponding to $(dN/dE)_{wo} + \Delta(E)_{max}$ while **spectUncert=-1** the minimal spectra corresponding to $(dN/dE)_{wo} - \Delta(E)_{min}$ are used. Note that although these uncertainties were calculated for the *Pythia-8* spectra, micrOMEGAs uses the same uncertainties for other spectra as well. One can access the spectra corresponding to the specified value of **spectUncert** by calling

- **spectraUncertainty(Mass,pdgN,outN,Spectr)**

which has the same parameters as .

The test code **mdlIndep/basicSpectra.c** contains an example of a call to **basicSpectra** and compares the four different spectra that are included in micrOMEGAs, checks energy conservation and shows uncertainties.

The functions presented below use the spectra for elementary processes obtained by **basicSpectra** and model dependent matrix elements.

- **decaySpectrum(pName,outN,spectTab)**

calculates the spectrum of the outgoing particle **outN** from the decay of particle **pName**. The result is written in an array of dimension **NZ**, **spectTab**, **outN** specifies the outgoing particle. **decaySpectrum** returns the error code 1 if **pName** is stable and 2 if $1 \rightarrow 2$ and $1 \rightarrow 3$ decay channels are not kinematically closed.

- **calcSpectrum(key,Sg,Se,Sp,Sne,Snm,Sn1,&err)**

calculates the spectra of DM annihilation at rest and returns σv in cm^3/s . For multicomponent DM the number of annihilation events in one cm^3 during one second at a distance **R** from the galactic center is given by

$$N(R) = \frac{1}{2} \sigma v \left(\frac{\rho(R)}{M_{cdm}^{eff}} \right)^2 \quad (64)$$

where

$$M_{cdm}^{eff} = \left(\sum_{i=1}^{N_{cdm}} \frac{fracCDM[i]}{M_{cdm} N[i]} \right)^{-1} \quad (65)$$

and $\rho(R)[GeV/cm^3]$ is the DM density. In the special case of one-component DM, $M_{cdm}^{eff} = M_{cdm}$ is the mass of DM.

The calculated spectra for γ , e^+ , $\bar{p}$, ν_e , ν_μ , ν_τ are stored in arrays of dimension **NZ** as described above: **Sg**, **Se**, **Sp**, **Sne**, **Snm**, **Sn1**. To remove the calculation of a given spectra, substitute **NULL** for the corresponding argument. **key** is a switch to include the polarisation of the W , Z bosons (**key=1**) or photon radiation (**key=2**). Note that final state photon radiation (FSR) is always included. When **key=2** the 3-body process $\chi\chi' \rightarrow XX + \gamma$ is computed for those subprocesses which either contain a light particle in the t-channel (of mass less than 1.2 M_{cdm}) or an outgoing W when $M_{cdm} > 500 GeV$. The FSR is then subtracted to avoid double counting. Only the electron/positron spectrum is modified with this switch. When **key=4** the contributions for each channel to the total annihilation rate are written on the screen. More than one option can be switched on simultaneously by adding the corresponding values for **key**. For example both the W polarization and

photon radiation effects are included if `key=3`. A problem in the spectrum calculation will produce a non zero error code, `err ≠ 0`. `calcSpectrum` uses `basicSpectra` to interpolate and sum spectra obtained by Pythia, thus it depends on the switch `SpectraFlag`.

- **vSigmaCh**

is an array that contains the relative contribution and particle names for each annihilation channel. It is similar to `vSigmaTCh` described in Section 6.2. Note that the list of particles contains five elements to allow to include gamma radiation. For 2->2 processes `vSigmaCh[n].prctl[4]=NULL`. The array `vSigmaCh` is filled by `calcSpectrum`.

- **calcSpectrumPlus(proc22, outP, Spect, &err)**

calculates DM annihilation into 3-body and 4-body final states via the exchange of virtual particles. These processes are not taken into account by `calcSpectrum`. Here `proc22` specifies the $2 \rightarrow 2$ process such as " $\chi, \chi' \rightarrow X, Y$ " which is kinematically forbidden at small relative velocity. `calcSpectrumPlus` includes all reactions with virtual `X` and `Y` particles. The parameter `outP` specifies the spectrum: 0 - photons, 1- positrons, 2- anti-protons, 3,4,5 - neutrinos. `Spect` is an array of size `NZ` which stores the calculated spectrum. The routine returns the value of $v\sigma$ in $[cm^3/s]$ units. If DM particles are not selfconjugated, the corresponding conjugated channel is added automatically.

10.3 Distribution of Dark Matter in Galaxy

The indirect DM detection signals depend on the DM density in our Galaxy. The DM density is given as the product of the local density at the Sun with the halo profile function

$$\rho(r) = \rho_{\odot} F_{halo}(r) \quad (66)$$

where we assume that

$$F_{halo}(R_{\odot}) = 1 \quad (67)$$

In `micrOMEGAs` $\rho_{\odot}$ is a global parameter `rhoDM` and $R_{\odot}$ is a global parameter `Rsun`. The default profile is the Zhao profile [90].

$$F_{halo}(r) = \left(\frac{R_{\odot}}{r}\right)^{\gamma} \left(\frac{r_c^{\alpha} + R_{\odot}^{\alpha}}{r_c^{\alpha} + r^{\alpha}}\right)^{\frac{\beta-\gamma}{\alpha}} \quad (68)$$

By default we use $\alpha = 1, \beta = 3, \gamma = 1$ which corresponds to the NFW profile and we set $r_c = 8.1[kpc]$ [91]. Note that we have modified the default value, it was $r_c = 20[kpc]$ in previous versions. The parameters of the Zhao profile can be reset by

- **setProfileZhao($\alpha, \beta, \gamma, r_c$)**

The function to set another density profile is

- **setHaloProfile(F_{halo})**

where $F_{halo}(r)$ is any function which depends on the distance from the galactic center, r , defined in $[kpc]$ units. For instance, `setHaloProfile(hProfileEinasto)` sets the Einasto profile

$$F_{halo}(r) = \exp \left[-\frac{2}{\alpha} \left(\left(\frac{r}{r_2}\right)^{\alpha} - \left(\frac{R_{\odot}}{r_2}\right)^{\alpha} \right) \right] \quad (69)$$

where by default $\alpha = 2.33$, $r_2 = 10.7$ [92], note that the default values in previous versions were $\alpha = 0.17$, $r_2 = R_{\text{sun}}$. The values can be changed by

- `setProfileEinasto( $\alpha, r_2$ )`.

The command `setHaloProfile(hProfileZhao)` sets back the Zhao profile.

Dark matter annihilation in the Galaxy depends on the average of the square of the DM density, $\langle \rho^2 \rangle$. This quantity can be significantly larger than $\langle \rho \rangle^2$ when clumps of DM are present [93]. In `micrOMEGAs`, we use a simple model where f_{cl} is a constant that characterizes the fraction of the total density due to clumps and where all clumps occupy the same volume V_{cl} and have a constant density ρ_{cl} . Assuming clumps do not overlap, we get

$$\langle \rho^2 \rangle = \rho^2 + f_{cl} \rho_{cl} \rho. \quad (70)$$

This simple description allows to demonstrate the main effect of clumps: far from the Galactic center the rate of DM annihilation falls as $\rho(r)$ rather than as $\rho(r)^2$. The parameters ρ_{cl} and f_{cl} have zero default values. The routine to change these values is

- `setClumpConst( $f_{cl}, \rho_{cl}$ )`

To be more general, one could assume that ρ_{cl} and f_{cl} depend on the distance from the galactic center. The effect of clumping is then described by the equation

$$\langle \rho^2 \rangle(r) = \rho(r)(\rho(r) + \rho_{clump}^{eff}(r)), \quad (71)$$

and the function

- `setRhoClumps( $\rho_{clump}^{eff}$ )`

allows to implement a more sophisticated clump structure. To return to the default treatment of clumps call `setRhoClumps(rhoClumpsConst)` or `setClumpConst`.

10.4 Photon signal

The photon flux does not depend on the diffusion model parameters but on the angle ϕ between the line of sight and the center of the galaxy as well as on the annihilation spectrum into photons

- `gammaFluxTab(fi, dfi, sigmav, Sg, Sobs)`

multiplies the annihilation photon spectrum with the integral over the line of sight and over the opening angle to give the photon flux. `fi` is the angle between the line of sight and the center of the galaxy, `dfi` is half the cone angle which characterizes the detector resolution (the solid angle is $2\pi(1 - \cos(dfi))$), `sigmav` is the annihilation cross section, `Sg` is the DM annihilation spectra. `Sobs` is the spectra observed in $1/(\text{GeV cm}^2 \text{ s})$ units.

The function `gammaFluxTab` can be used for the neutrino spectra as well.

- `gammaFluxTabGC(l, b, dl, db, sigmav, Sg, Sobs)`

is similar to `gammaFluxTab` but uses standard galactic coordinates. Here l is the galactic longitude (measured along the galactic equator from the galactic center, and b is the latitude (the angle above the galactic plane). Both l and b are given in radians. The relation between the angle `fi` used above and the galactic coordinates is $\text{fi} = \cos^{-1}(\cos(l) \cos(b))$. `gammaFluxTabGC` integrates the flux over a rectangle $[(l, b) - (l + dl, b + db)]$.

- `loopGamma(&vcs_gz, &vcs_gg)`

calculates σv for loop induced processes of DM pair annihilation into γZ and into $\gamma\gamma$. The result is given in $\frac{\text{cm}^3}{\text{s}}$. The function returns a non-zero value to signal a problem. This function is available only for MSSM [94], NMSSM [95], CPVMSSM and IDM. Note that

this function does not include non-perturbative effects that are in particular important when the mass of DM is much above the weak scale [96,97].

- **gammaFlux(fi,dfi,dSigmavdE)**

computes the photon flux for a given energy E and a differential cross section for photon production, $dSigmavdE$. For example, one can substitute $dSigmavdE=\sigma v SpectrdNdE(E,SpA)$ where σv and SpA are obtained by `calcSpectrum`. This function can also be used to compute the flux from a monochromatic gamma-ray line by substituting the cross section at fixed energy (in cm^3/s) instead of $dSigmavdE$, for example the cross sections obtained with the `loopGamma` function in the MSSM, NMSSM, CPVMSSM models (`vcsAA` and `vcsAZ`). In this case the flux of photons can be calculated with `gammaFlux(fi,dfi,2*vcsAA+vcsAZ)`.

- **gammaFluxGC(l,b,dl,db,vcs)**

is the analog of `gammaFlux` when using standard galactic coordinates.

10.5 Propagation of charged particles

The observed spectrum of charged particles strongly depends on their propagation in the Galactic Halo. The propagation depends on the global parameters

`K_dif`, `Delta_dif`, `L_dif`, `Rsun`, `Rdisk`

as well as

`Tau_dif (positrons)`, `Vc_dif (antiprotons)`

- **posiFluxTab(Emin,sigmav, Se, Sobs)**

computes the positron flux at the Earth. Here `sigmav` and `Se` are values obtained by `calcSpectrum`. `Sobs` is the positron spectrum after propagation. `Emin` is the energy cut to be defined by the user. Note that a low value for `Emin` increases the computation time. The format is the same as for the initial spectrum. The function `SpectrdNdE(E,Sobs)` described above can also be used for the interpolation, in this case the flux is returned in $(GeV\ s\ cm^2sr)^{-1}$.

- **pbarFlux(E,dSigmavdE)**

computes the antiproton flux for a given energy E and a differential cross section for antiproton production, $dSigmavdE$. For example, one can substitute $dSigmavdE=\sigma v SpectrdNdE(E,SpP)$

where σv and SpP are obtained by `calcSpectrum`. This function does not depend on the details of the particle physics model and allows to analyse the dependence on the parameters of the propagation model.

- **pbarFluxTab(Emin,sigmav, Sp, Sobs)**

computes the antiproton flux, this function works like `posiFluxTab`,

- **solarModulation(Phi, mass, stellarTab, earthTab)**

takes into account modification of the interstellar positron/antiproton flux caused by the electro-magnetic fields in the solar system. Here `Phi` is the effective Fisk potential in MeV, `mass` is the particle mass, `stellarTab` describes the interstellar flux, `earthTab` is the calculated particle flux in the Earth orbit.

Note that for `solarModulation` and for all `*FluxTab` routines one can use the same array for the spectrum before and after propagation.

11 Planck CMB constraint

The function

- `PlanckCMB(vSigma, SpA, SpE)`

determines whether the DM model under consideration is consistent with Planck measurements of the CMB anisotropies caused by the injection of electrons and photons coming from DM annihilation [98]. The spectra of DM annihilation into photons and positrons are provided in the arrays `SpA` and `SpE`. Here it is assumed that the DM relic density $\Omega h^2 = 0.12$, for multi-component DM the fraction of each DM component described by the `fracCDM` array is taken into account. We also assume that the input parameter `vSigma` represents the s-wave cross section in $[cm^3/s]$ units, this limit does not apply if the DM annihilation cross-section depends on velocity at small velocities. A return value more than 1 indicates that the model is excluded at 95% C.L. Note that if the DM model gives $\Omega h^2 < 0.12$ and one assumes that DM forms only a fraction ξ of the total DM in the Universe, then the value returned by `PlanckCDM` should be scaled by ξ^2 .

By default we assume that the annihilation is s-wave and that `vSigma` is the cross-section computed from `calcSpectrum`, the number of events will be computed using M_{cdm}^{eff} , as given in Eq.65. If a model of elementary particles is not loaded, and the user just defined the spectra assuming some specific annihilation channels, then the number of events is computed using the `Mcdm` parameter instead. This allows for a model independent test of the function `PlanckCMB`.

In the file `mdlIndep/CMB.c` we reproduce the top panels of Fig.3 and Fig.4 of Ref. [98], here we use the PPPC spectra.

12 Photons from Dwarf Spheroidal galaxies

- `DwarfSignal(vSigma, SpA)`

determines whether the photon spectra `SpA` is compatible with the 95% limits obtained from observations of Dwarf Spheroidal galaxies Carina, Fornax, LeoI, SegueI, Draco, LeoII, Sculptor, Sextans [36–38]. Here `vSigma` is the DM annihilation cross section at low velocities ($<v\sigma>$ in $[cm^3/s]$ units) as obtained by `calcSpectrum`. The return value more or equal to 1 indicates that the given model is excluded at 95% confidence level. The tested $<v\sigma>$ divided on the return value should be close to 95% excluded cross section. But this relation is approximate. In particular because return value is always in interval 0.01 – 100.

To compute the signal `vSigma` is divided by the squared mass M_{cdm}^{eff} , Eq.65. For a model independent test of this function the `Mcdm` parameter is used instead. For an example, see the file `mdlIndep/Dwarf.c` which reproduces the *combined* curve in [38], Fig.5 (Right).

13 Neutrino signal from the Sun and the Earth

This module only works for single component dark matter

After being captured, DM particles concentrate in the center of the Sun/Earth and then annihilate into Standard Model particles. These SM particles further decay producing neutrinos that can be observed at the Earth. The neutrino spectra originating

from different annihilation channels into SM particles and taking into account oscillations and Sun medium effects were computed both in `WimpSim` [99] and in `PPPC4DM $\nu$` [100]. We use the set of tables provided by these two groups as well as those from `DM $\nu$` [101] which were included in previous versions of `micrOMEGAs`. The new global parameter `WIMPSIM` allows to choose the neutrino spectra. The default value `WIMPSIM=0`¹² corresponds to the `PPPC4DM $\nu$` spectra while `WIMPSIM=1` corresponds to the `WimpSim` spectra and `WIMPSIM=-1` to the `DM $\nu$` spectra.

- `neutrinoFlux(f,forSun,nu, nu_bar)`

calculates the muon neutrino/anti-neutrino fluxes near the surface of the Earth. Here `f` is the DM velocity distribution normalized such that $\int_0^\infty v f(v) dv = 1$. The units are km/s for `v` and s^2/km^2 for `f(v)`. For example, one can use the same `Maxwell` function introduced for direct detection. This routine implicitly depends on the `WIMPSIM` switch.

If `forSun==0` then the flux of neutrinos from the Earth is calculated, otherwise this function computes the flux of neutrinos from the Sun. The calculated fluxes are stored in `nu` and `nu_bar` arrays of dimension `NZ=250`. The neutrino fluxes are expressed in $[1/Year/km^2]$.

- `muonUpward(nu,Nu,muon)`

calculates the muon flux which results from interactions of neutrinos with rocks below the detector. Here `nu` and `Nu` are input arrays containing the neutrino/anti-neutrino fluxes calculated by `neutrinoFlux`. `muon` is an array which stores the resulting sum of μ^+ , μ^- fluxes. `SpectdNdE(E,muon)` gives the differential muon flux in $[1/Year/km^2/GeV]$ units. The muon flux weakly depends on the propagation medium, e.g. rock or ice. The energy lost during propagation is described by the equation [102]

$$\frac{dE}{dx} = -(\alpha + \beta E)\rho \quad (72)$$

For propagation in ice (the switch `forRocks=0`), `micrOMEGAs` substitutes $\rho = 1.0 \text{ g/cm}^3$, $\alpha = 0.00262 \text{ GeVcm}^2/\text{g}$, $\beta = 3.5 \times 10^{-6} \text{ cm}^2/\text{g}$ [103], while for propagation in rocks, $\rho = 2.6 \text{ g/cm}^3$, $\alpha = 0.002 \text{ GeVcm}^2/\text{g}$, $\beta = 3.0 \times 10^{-6} \text{ cm}^2/\text{g}$ [102]. The result depends on the ratio α/β .

- `muonContained(nu,Nu,rho, muon)` calculates the flux of muons produced in a given detector volume. This function has the same parameters as `muonUpward` except that the outgoing array gives the differential muon flux resulting from neutrinos converted to muons in a km^3 volume given in $[1/Year/km^3/GeV]$ units. `rho` is the density of the detector in g/cm^3 .

- `atmNuFluxes(nuPdg,E,cos)`

returns the atmospheric neutrinos fluxes in $(m^2 \cdot sec \cdot sr \cdot GeV)^{-1}$ units as presented in [104]. Only $\nu_e, \bar{\nu}_e, \nu_\mu, \bar{\nu}_\mu$ are included corresponding to PDG codes, `nuPdg = 12,-12,14,-14`. Input parameters should be specified in the range, $-1 \leq \cos \leq 1$, $0.1 \leq E \leq 10^4 \text{ GeV}$. Ref. [104] provides several tables of calculated neutrino fluxes depending on disposition, solar activity, with and without mountains over the detector. By default we use the file `"grn-ally-20-01-mtn-solmax.d"` which gives the neutrino flux for a detector under the mountains in Gran Sasso at the time of large solar activity. To use other tables provided

¹²Since `PPPC4DM $\nu$` does not provide neutrino spectra produced at the center of the Earth, in this case and for `WIMPSIM=0` `micrOMEGAs` uses the `DM $\nu$` spectra.

in

<http://www-rccn.icrr.u-tokyo.ac.jp/mhonda/public/nflx2014/index.html>

the user has to copy the corresponding file in the micrOMEGAs directory `Data/nu_data` or in the current directory and write down the name of the file in the public variable `atmNuFile`. For example

```
atmNuFile="grn-ally-20-01-mtn-solmin.d";
```

- `atmNuFluxesI(nuPdg,E)`

returns $\frac{1}{2} \int_{-1}^1 atmNuFluxes(nuPdg, E, cs) dcs$

13.1 Comparison with IceCube results

These functions are described in [105] and allow to compare the predictions for the neutrino flux from DM captured in the Sun with results of IceCube22.

- `IC22nuAr(E)`

effective area in $[km^2]$ as a function of the neutrino energy, $A_{\nu_\mu}(E)$

- `IC22nuBarAr(E)`

effective area in $[km^2]$ as a function of the anti-neutrino energy, $A_{\bar{\nu}_\mu}(E)$.

- `IC22BGdCos(cs)`

angular distribution of the number of background events as a function of $\cos \phi$, $\frac{dN_{bg}}{d\cos \phi}$.

- `IC22sigma(E)`

neutrino angular resolution in radians as a function of energy.

- `exLevIC22( nu_flux, nuB_flux,&B)`

calculates the exclusion confidence level for number of signal events generated by given ν_μ and $\bar{\nu}_\mu$ fluxes, [105]. The fluxes are assumed to be in $[GeV km^2 Year]^{-1}$. This function uses the `IC22BGdCos(cs)` and `IC22sigma(E)` angular distribution for background and signal as well as the event files distributed by IceCube22 with $\phi < \phi_{cut} = 8^\circ$. The returned parameter `B` is a Bayesian factor representing the ratio of likelihood functions for the model with given fluxes and the model with null signal. See details in [105].

- `fluxFactorIC22(exLev, nu,nuBar)`

For given neutrino, `nu`, and anti-neutrino fluxes, `nuBar`, this function returns the factor that should be applied to the fluxes (neutrino-proton cross sections) to obtain a given exclusion level `exLev` in `exLevIC22`. This is used to obtain limits on the SD cross section for a given annihilation channel.

14 Relic density of sterile neutrinos.

- `darkOmegaNu(T0,Tinit, kL, sin22, LsInit, msKeV)`

calculates the relic density of sterile neutrinos. Here we assume that the sterile neutrino mixes with the SM left handed neutrino of generation `kL`. The parameter `sin22` = $\sin^2(\phi/2)$ defines the mixing angle. The parameter `LsInit` is the lepton asymmetry, L , in the `kL`'th generation at temperature `Tinit` normalized to the entropy density. Note that SM interaction does not change L/s . `msKeV` is the mass of the sterile neutrino in `keV`. The temperature `T0` is usually chosen to be equal to `0.01GeV` and `Tinit` has to be of the order of several `GeV`'s since the generation of the relic density of sterile neutrinos occurs at a temperature around 100-200 `MeV`.

Note that lepton asymmetry is changed by the resonance production of sterile neutrino. In general

$$Y = Y_{\nu_s} + Y_{\bar{\nu}_s} \approx Y_{\nu_s} - Y_{\bar{\nu}_s} = \frac{L(T_{init})}{s(T_{init})} - \frac{L(T_0)}{s(T_0)} \quad (73)$$

Thus the rate of DM generation follows the rate of disappearance of lepton asymmetry. The latter is represented by the function

- **LasymmS(T)**

which is given by $\frac{L(T)}{s(T)}$.

The resulting momentum distribution in terms of $\epsilon = p/T$ for $T \ll 1\text{MeV}$ is presented by the function

- **fNuSterile(ϵ , txt)**

where the text parameter **txt** allows one to specify the distribution of sterile neutrino, **+nus**, antineutrino, **-nus** or total one, **nus**. With the parameter **txt** one can define an additional factor of power of ϵ . For example, the number density of neutrino at temperature T is

$$\rho = \frac{4\pi}{(2\pi)^3} T^3 \int_0^\infty fNuSterile(\epsilon, "nus2") d\epsilon \quad (74)$$

Our calculation follows that of [106] and we have roughly a 15% numerical agreement with this paper. In particular, the momentum distribution **fNuSterile** can be compared with Fig. 10(b,d) and **LasymmS** can be compared with Fig. 10a. See the test code in `mdlIndep/nuS.c`.

15 Constraints from colliders

15.1 The Higgs sector

To obtain the limits on the Higgs sector for models with one or several Higgs bosons, the predictions for the signal strengths of the 125 GeV Higgs can be compared to the latest results of the LHC, for this an interface to the public code **HiggsSignals** [49] or **Lilith** [33–35] is provided. Moreover the exclusion limits obtained from Higgs searches in different channels at LEP, Tevatron and the LHC can be applied to other Higgses in the model using **HiggsBounds** [47].

15.1.1 Lilith

Lilith [33–35] is a Python library that is used to construct a global likelihood function $\mathcal{L}$ from the latest ATLAS and CMS results on the 125 GeV Higgs.¹³ The **Lilith** inputs are the set of reduced couplings of the 125 GeV state, *i.e.*, couplings normalized by the SM ones, as well as the branching ratios of Higgs decays to invisible, BR_{inv} , or to undetected non-SM final states, $\text{BR}_{\text{undetected}} = 1 - \text{BR}_{\text{inv}} - \sum \text{BR}(H \rightarrow \text{SM SM})$. By default, the automatic generation of the input file assumes that only DM contributes to

¹³**Lilith** can test Higgs bosons with masses within the [123, 128] GeV interval, a warning will be issued if no such state can be found. In the case where two or more states have masses within this interval, their signal strengths will be summed incoherently and an effective Higgs state will be tested against the LHC measurements.

the invisible width. Note that the reduced couplings of the 125 GeV Higgs are defined for all the models provided with `micrOMEGAs` with the exception of the $Z\gamma$ coupling. The latter is computed within `Lilith` assuming that only SM particles run in the loop. The file `include/Lilith.inc` (or `Lilith.inc_f`) contains the instructions to launch `Lilith` using a system call. The input file `Lilith_in.xml` for `Lilith` can be created by two commands

```
LilithMDL("Lilith_in.xml")
LilithMO("Lilith_in.xml")
```

both also return the number of neutral Higgs particles. `LilithMDL` requires that the reduced couplings be defined in `lib/lilith.c` of the model. These files are defined for all models provided with `micrOMEGAs` and should be written by the user for new models. On the other hand `LilithMO` generates automatically the input file required by `Lilith`. The functionality of `LilithMO` is described in Section 15.1.3. As input parameters, `Lilith` also requires the number of free parameters, `n_par`, and a reference likelihood point, `m2logL_reference`. Those are defined in the `main.c` file of each model and set to 0 by default.

The SLHA output file, `Lilith_out.slha`, consists of six entries which are respectively the log-likelihood evaluated at a parameter space point $\mathcal{P}$, $-2\log\mathcal{L}(\mathcal{P})$, the number of experimental degrees of freedom, n^{exp} , the reference likelihood point, the number of degrees of freedom, ndf , the p-value, p and the database version. For a detailed description of the various outputs and their interpretation see [33,34]. For example, one could flag or exclude points with too low p-value. In this context, a point with a p-value smaller than 0.3173, 0.0455, 0.0027 could be excluded at more than the 1σ , 2σ , 3σ levels, respectively.

15.1.2 HiggsBounds and HiggsSignals

Constraints on the properties of the 125 GeV Higgs boson can also be obtained with `HiggsSignals`. Moreover exclusion limits provided by the experimental LHC and Tevatron collaborations on additional Higgs bosons are obtained through an interface to `HiggsBounds` [107]. These codes are no longer distributed with `micrOMEGAs` but are downloaded and compiled when required¹⁴. The file `include/hBandS.inc` contains the instructions to call both `HiggsBounds` and `HiggsSignals`, in particular the options for running `HiggsSignals` are set in this file by fixing the `Dataset`, `Method` and `PDF` parameters. Moreover the interface uses the effective coupling option for specifying the input see [47] for more details. Two functions can be used to generate the input SLHA file, `HB.in`

```
hbBlocksMDL("HB.in",&NchHiggs)
hbBlocksMO("HB.in",&NchHiggs)
```

Both return the number of neutral Higgses and `NchHiggs` gives the number of charged Higgs particles. The function `hbBlocksMDL` is located in `lib/hbBlocks.c` for each MSSM-like model and contains the appropriate definition of the reduced couplings. The function `hbBlocksMO` generates automatically the required input file for any model and is thus very convenient to use for new models. The content of this function is described in

¹⁴For compilation one needs Fortran f90 compiler, for instance `gfortran`, and `cmake`

Section 15.1.3. Note that the PDG numbers which will be considered by `HiggsBounds` for Higgs-like particles are only 25, 35, 36, 37, 45, and 46. Moreover `HiggsBounds` will check the contribution to the Higgs invisible width only for particles with PDG codes 1000022, 1000012, 1000014, 1000016, 1000017, 1000018 or 1000019. Also note that the theoretical uncertainty on the mass of the Higgs boson should be specified in the SLHA BLOCK `DMASS`, see the example given in `main.[c/F]` to set the uncertainty at 2 GeV.

The complete outputs of `HiggsBounds` and `HiggsSignals` are stored in the files `HB.out` and `HS.out` respectively and can be accessed and read by the user using the `slhaval` function [108]. The screen output of `micrOMEGAs` contains the following information

```
HiggsBounds(version number)
id    result  obsratio  channel
```

where three channels are listed, `result` = 0, 1, -1 denotes respectively whether a parameter point is excluded at 95% CL, not excluded, or invalid; `obsratio` gives the ratio of the theoretical expectation relative to the observed value for the channel specified in `channel`. The first channel in this list is the one with the highest expected sensitivity and is the one that determines the overall exclusion, see [47], other channels are to be interpreted by the user.

The `HiggsSignals` output displayed on the screen is simply

```
HiggsSignals(version number)
Nobservables chi^2  pval
```

where `Nobservables` gives the number of observables used in the fit, `chi^2` the associated χ^2 and `pval` the p-value. The interpretation of these values is left to the user, see [49] for a detailed description.

15.1.3 Automatic generation of interface files

The functions `LiLithM0` and `hbBlocksM0` need information about numerical values of couplings included in Higgs interactions with SM particles. We use the tools presented in Section 18.6 to find them.

All the QCD-neutral scalars belonging to the even sector (not designated with $\sim$) are considered as Higgs particles. For each of these, `micrOMEGAs` calculates the couplings to SM fermions and massive bosons and writes down into the interface file the ratio of these couplings to the corresponding SM Higgs coupling. Note that the couplings of the Higgs to SM fermions can significantly depend on the QCD scale; `micrOMEGAs` assumes that the quark masses entering the vertices are obtained at the same scale in both the SM and the new model, thus the scale dependence in the reduced couplings to fermions disappears. The loop-induced couplings of the Higgs to gluons and photons are calculated by the `LiLithM0/hbBlocksM0` routines whether or not the Hgg and $H\gamma\gamma$ vertices are already implemented in the Lagrangian. This includes NLO-QCD corrections and is performed as described in [7, 109].

The various branching ratios and widths of the Higgs required by `HiggsBounds`, `HiggsSignals` or `LiLith` are also written automatically in the interface file. When these values are provided in an SLHA file, they will be used. Otherwise `LiLithM0/hbBlocksM0` check the exis-

tence of $H \rightarrow gg$ and $H \rightarrow \gamma\gamma$ in the table of decays generated from the model Lagrangian. If found, the branching ratios and total widths are written in the interface file without comparing with the internal calculations. If not found, then `LiLithMO/hbBlocksMO` add these channels and recompute the total widths and all branching ratios.

15.2 Searches for New particles

15.2.1 SModelS

LHC limits on new (odd) particles can be obtained using **SModelS** [41–45], a code which tests Beyond the Standard Model (BSM) predictions against Simplified Model Spectra (SMS) results from ATLAS and CMS searches for new physics. The basic working principle of **SModelS** is to decompose a full input model (consisting of particle content, masses, production cross sections and decay widths) of a given BSM scenario into SMS components with their corresponding signal weights. These are then used to evaluate the constraints from a large database of experimental results. The outcome is presented as the ratio of predicted over excluded cross section, so-called r -values, for each experimental analysis in the database that provides a constraint. For efficiency-map type experimental results, the output also reports likelihoods. The (user-defined) combination of likelihoods from uncorrelated analyses is also possible [45, 110].

The **SModelS** approach provides a fast way to cast BSM predictions for the LHC in a model-independent framework, which can be directly confronted with the relevant experimental constraints. The underlying assumption is that differences in the event kinematics (e.g. from different production mechanisms or from the spin of the BSM particle) do not significantly affect the signal selection efficiencies. The majority of experimental results in the **SModelS** database are from searches for promptly decaying SUSY particles, but there are also a number of ‘exotics’ results and a growing number of results from searches for long-lived particles.

micrOMEGAs is now interfaced to version 3.1.0 of **SModelS**, which, among other novelties, features an improved treatment of width-dependent constraints. Most importantly, this new version can handle arbitrary simplified model topologies, without the need of an imposed $\mathcal{Z}_2$ -like symmetry. It is thus capable of treating also resonant production of new particles, RPV signatures, etc., in addition to the pair-production of new particles followed by linear cascade decays, which is typical for models with a $\mathcal{Z}_2$ -like symmetry.

For running **SModelS**, **micrOMEGAs** automatically generates an SLHA-type input file `smodels.slha` which contains

- QNUMBER blocks for the BSM particles as specification of the model;
- MASS block for masses of BSM particles;
- DECAY blocks for decay widths and branchings for BSM and Higgs particles;
- XSECTION blocks which contains cross sections of LHC processes with production of BSM particles.¹⁵

¹⁵At present, cross sections are computed automatically for $pp \rightarrow \text{BSM BSM}$ production. Resonant (s -channel) or associated production of one new particle, i.e. $pp \rightarrow \text{BSM}$ or $pp \rightarrow \text{BSM SM}$, which is relevant for models without a $\mathcal{Z}_2$ -like symmetry, will be added in a future release of the **micrOMEGAs-SModelS** interface.

This input file, per default called `smodels.slha`, is located in the same directory as `main.c` and is written by `micrOMEGAs` by calling the function

```
smodels(Pcm, nf, csMinFb, ExcludeBSM, fileName, version, wrt)
```

where `Pcm` is the proton beam energy in GeV and `nf` is the number of parton flavors used to compute the production cross sections of the odd-sector particles. (Note that u , d , $\bar{u}$, $\bar{d}$ and gluons are always included while s , c , and b quarks are included for `nf` = 3, 4, 5 respectively.). The new version of `SModelS` also considers $2 \rightarrow 1$ reactions

```
proton,proton -> BSM
```

All $2 \rightarrow 2$ and $2 \rightarrow 1$ reactions involving BSM particles are computed by `micrOMEGAs`, the user can however simplify this task by providing the list of particles that do not need to be considered in the parameter `ExcludeBSM`. In this list, particle names must be separated by commas or spaces. The corresponding antiparticles are taken into account automatically.

By default, `micrOMEGAs` uses the `cteq66` [111] structure functions (set in the `hCollider()` function, see section 18). `csMinFb` defines the minimum production cross section in pb for odd particles; processes with lower cross sections are not added to the SLHA file passed to `SModelS`, here denoted by `fileName`. Finally, `wrt` is a steering flag for the screen output; if `wrt` $\neq$ 0 the computed cross sections will be also written on the screen. The call of `smodels()` is per default done in the file `micromegas_X/include/SMODELS.inc`

For this functionality, the user first needs to specify which LHC energies should be considered. This is done by setting in `main.c` the switch

```
LHCrun = LHC8 and/or LHC13
```

for Run 1 (8 TeV), Run 2 (13 TeV) or both. This determines whether 8 TeV or 13 TeV cross sections or both will be appended to the SLHA file, which in turn determines which simplified model results will be considered from the `SModelS` database.

The run parameters for `SModelS` can be set in the file

```
micromegas_X/include/smodels_parameters.ini.
```

This concerns for example the parameter `sigmacut`, which is the cutoff cross section for topologies to be considered in the decomposition. Also the database version to be used is set in this parameters file; the default is `path = official`, which means that the official database pickle file corresponding to the `SModelS` code version will be downloaded and used. An interesting alternative may be to set `path = latest`, in which case always the latest available database will be used.

The `SModelS` parameters `combineSRs` and `combineAnas` can now be defined in `main.c`.

- `combineSRs != 0` turns on the combination of signal regions on when a covariance matrix or full JSON likelihood is available, see [43, 112].¹⁶

¹⁶Signal region combination can significantly increase the sensitivity of an analysis. However, since it can require much more CPU time, considerably increasing the run time, it is turned off by default.

- `combineAnas` = [list of analysis IDs] is a text parameter, which lists the analyses to be combined in a global likelihood. This switch works only for analyses with efficiency-map results. All the analyses are assumed to be fully uncorrelated, so use with caution!

By default, `char*combineAnas=NULL;`. For example, the following can be set by the user `char*combineAnas= "ATLAS-SUSY-2018-41,ATLAS-SUSY-2019-08,CMS-SUS-21-002";` For detailed information on the different options and parameters available, see the [online SModelS manual](#).

The output is written in SLHA-type format to `smodels.slha.smodelsslha` (or an alternative name selected by the user in `SModelS.inc`). This output consists of the following blocks:

- **SModelS_Settings**, which lists the SModelS code and database versions as well as input parameters for the decomposition;
- **SModelS_Exclusion**, which contains as the first line the status information if a point is excluded (1), not excluded (0), or not tested (−1), the latter being reported when no matching SMS results are found. This is followed by the list of experimental results. If `expandedOutput = True` (default), all results are printed. Otherwise, if the model is excluded, all results with r -value greater than one are shown and if the point is not excluded, only the result with the highest r -value is displayed;
- **SModelS_CombinedAnas** containing information about the combination of results, if `combineAnas` is defined (see above);
- **SModelS_Coverage**, which, if `testCoverage` is set to True (default: False) in the parameters file, lists the cross sections of missing topologies with prompt and/or displaced decays, and of topologies outside the grids of the experimental results

If a point is excluded (status 1), this is followed by a list of all results with $r > 1$, starting from the highest r -value. We remind the user that the SModelS r -values are defined as the ratio of the predicted theory cross section and the corresponding experimental upper limit at 95% confidence level. For each of these results, the SMS topology identifier as entry 0 (so-called Tx-name, see the [online SMS dictionary](#) for an explanation of the terminology), the r -value as entry 1, a measure of condition violation as entry 2, and the analysis identifier as entry 3 are listed. If the point is not excluded (status 0), the result with the highest r -value is given instead to show whether a point is close to the exclusion limit or not.

Last but not least, in order to exploit decay channels involving a SM-like Higgs for which the experimental collaborations assumed SM branching ratios for the h with the mass fixed to $m_h = 125$ GeV, micrOMEGAs checks whether neutral scalar particles with a mass in the range 123–128 GeV have branching ratios to WW^* , ZZ^* , $\tau\tau$, $b\bar{b}$ within 15% of those of a SM Higgs of the same mass. The corresponding particle will be identified as a SM Higgs through PDG code 25 in the `smodels.slha` file. Note that, for SModelS, PDG code 25 is reserved for a SM-like Higgs and should not be assigned generically. Thus, micrOMEGAs re-assigns the PDG codes when needed. Namely the particle which satisfies the conditions above is assigned the code 25, while a particle with code 25 that does not satisfy one of these conditions is re-assigned the code 25252525.

15.3 Other constraints

Limits from searches for a new massive Abelian gauge boson at the LHC, from LEP on an invisible Z as well as limits on light neutralinos from LEP are provided through the functions:

- **Zinvisible()**

returns 1 and prints a WARNING if the invisible width of the Z boson of the Standard Model is larger than 0.5 MeV ([113]) and returns 0 if this constraint is fulfilled. This routine can be used in any model with one DM where the Z boson is uniquely defined by its PDG=23 and whether the neutral LSP is its own antiparticle or not.

- **ZpLimCMS(ZpName)**

returns value more than 1 if its interaction with quarks and leptons is excluded at 95% confidence level (C.L.) according to the CMS analysis of dilepton resonance production based on 140 fb⁻¹ of Run 2 data [114]. The argument **ZpName** is the name of the Z' particle in the Beyond-Standard-Model (BSM) extension under investigation, as defined in the corresponding **CalcHEP** model file.

$$L = Z'_\mu \bar{q} \gamma^\mu (g_v + g_a \gamma^5) q \quad (75)$$

$$c_q = (g_v^2 + g_a^2) (Br(Z' \rightarrow e^+, e^-) + Br(Z' \rightarrow \mu^+, \mu^-)) \quad (76)$$

The couplings and branching ratios are obtained using tools presented in Sec. 18.6 and 18.5. The $[c_d, c_u]$ defined in CMS experiment are disposed on a straight line. We find the boundary points of this line $[cdMax(M_{Z'}), 0]$ and $[0, cuMax(M_{Z'})]$ ¹⁷. Then **ZpLimCMS** returns $c_d/cdMax(M_{Z'}) + c_u/cuMax(M_{Z'})$

- **LspNlsp_LEP(&nCS)**

checks the compatibility with the upper limit obtained at LEP [115] on the cross section for the production of neutralinos $\sigma(e^+e^- \rightarrow \tilde{\chi}_1^0 \tilde{\chi}_i^0)$, $i \neq 1$, when the heavier neutralino decays into quark pairs and the LSP, $\tilde{\chi}_i^0 \rightarrow \tilde{\chi}_1^0 q \bar{q}$. The function calculates $nCS = \sigma \times BR = \sum_i \sigma(e^+e^- \rightarrow \tilde{\chi}_1^0 \tilde{\chi}_i^0) \times BR(\tilde{\chi}_i^0 \rightarrow \tilde{\chi}_1^0 q \bar{q}) / cs_{Lim}$ where $cs_{Lim} = 0.1$ pb if $m_{NLSP} > 100$ GeV and 0.5 pb otherwise. The function returns 1 if $nCS \geq 1$ and 0 otherwise. This function can also be applied for non-SUSY models which feature the same signature, in this case the function will compute the cross section for production of the LSP and any other neutral particle from the odd sector which can decay into the LSP and a Z boson.

16 Additional routines for specific models

The models included in **micrOMEGAs** contain some specific routines which we describe here for the sake of completeness. The current distribution includes the following models: **MSSM**, **NMSSM**, **CPVMSSM**, **IDM** (inert doublet model), **LHM** (little Higgs model), **Z3IDM** (Inert doublet model with Z_3 symmetry), **Z4IDSM** (Inert doublet and singlet model with Z_4 symmetry), **Z5M** (Two scalar singlets model with Z_4 symmetry), **RDM** (scalar leptoquark and

¹⁷See section `#ifdef find_cu_cd_max` in `ZpPortal/main.c`

two singlet fermions), STFM (singlet-triplet fermionic model) as well as three models which serve as examples for freeze-in and for which there are currently no additional routines (SingletDM, ZpPortal, LLL_scalar).

Some of these models contain a special routine for reading the input parameters:

- `readVarMSSM`, `readVarNMSSM`, `readVarCPVMSSM`, `readVarlHiggs`, `readVarRHM`.

These routines are similar to the general `readVar` routine described in Section 6 but they write a warning when a parameter is not found in the input file and display the default values for these parameters.

The supersymmetric models contain several additional routines to calculate the spectrum and compute various constraints on the parameter space of the models. Some functions are common to the `MSSM`, `NMSSM`, `CPVMSSM`, `UMSSM` models:

- `o1Contents(FD)`

prints the neutralino LSP components as the $\tilde{B}$, $\tilde{W}$, $\tilde{h}_1$, $\tilde{h}_2$ fractions. For the `NMSSM` the fifth component is the singlino fraction $\tilde{S}$ and for the `UMSSM` the sixth component is the bino' fraction $\tilde{B}'$. The sum of the squares of the LSP components should add up to 1.

16.1 The MSSM

The `MSSM` has a long list of low scale independent model parameters, those are specified in the SLHA file [56, 116]. They are directly implemented as parameters of the model. `micrOMEGAs` can either read an SLHA file computed externally or generate this file in the framework of popular SUSY scenarios, these are specified by one of the instructions

```
#define SUGRA
#define SUGRANUH
#define AMSB
#define EWSB
```

If none of these instructions are given then `micrOMEGAs` reads the external SLHA file which should be passed as an argument when executing the *main* routine. To generate an SLHA file, `micrOMEGAs` uses either `SuSpect`, `SPHENO`, or `SoftSusy`. These codes, if need be, solve the RGE equations and calculate the masses of Higgs and SUSY particles including one-loop corrections. Note that `SuSpect` is included within the `micrOMEGAs` distribution while `SPHENO` or `SOFTSUSY` will be downloaded automatically when needed (see section 3.4). To specify the spectrum calculator the user has to define the parameter `RGE` in *main.c/main.cpp*. For instance

```
#define RGE suspect
```

Other possibilities for `RGE` are `softSusy`, `spheno` and `tree`. The option `tree` will generate the needed SLHA file for the spectrum using EWSB input parameters and tree-level formulas. One-loop corrections to the Higgs masses and to the effective Higgs potential are based on [117]. In this case, the realization of the `MSSM` is completely gauge invariant. This option can be useful to check the effect of loop corrections.

For `EWSB` scenarios the input parameters are the soft parameters, the names of these parameters are given in the `MSSM/mssm[1/2].par` files. The user can assign new values to these parameters by means of `assignVal` or `readVarMSSM`.

- **spectEwsbMSSM()**

calculates the masses of Higgs and supersymmetric particles in the MSSM including one-loop corrections starting from weak scale input parameters.

In these functions *spect* is defined by RGE.

For other MSSM scenarios, the parameters at the electroweak symmetry breaking scale are derived from an input at high scale. The same codes **suspect**, **spheno**, or **softSusy** are used for this. The corresponding routines are:

- **spectSUGRA(tb, MG1, MG2, MG3, A1, At, Ab, signMu, MHu, MHd, M11, M13, Mr1, Mr3, Mq1, Mq3, Mu1, Mu3, Md1, Md3)**

assumes that all input parameters except **tb** and **signMu** are defined at the GUT scale. The SUGRA/CMSSM scenario is a special case of this general routine.

- **spectSUGRAnuh(tb, MG1, MG2, MG3, A1, At, Ab, M11, M13, Mr1, Mr3, Mq1, Mq3, Mu1, Mu3, Md1, Md3, mu, MA)**

realizes a SUGRA scenario with non universal Higgs parameters. Here the **Mhu**, **Mhd** parameters in the Higgs potential are replaced with the **mu** parameter defined at the EWSB scale and **MA**, the pole mass of the CP-odd Higgs. The **signMu** parameter is omitted because **mu** is defined explicitly.

- **spectAMSB(am0, m32, tb, sng)**

does the same as above within the AMSB model.

We have an option to directly read a SLHA input file, this uses the function

- **lesHinput(file_name)**

which returns a non-zero number in case of problem.

The routines for computing constraints are (see details in [3]).

- **deltarho()**

calculates the $\Delta\rho$ parameter in the MSSM. It contains for example the stop/sbottom contributions, as well as the two-loop QCD corrections due to gluon exchange and the correction due to gluino exchange in the heavy gluino limit.

- **bsgnlo(&SMbsg)**

returns the value of the branching ratio for $b \rightarrow s\gamma$, see Appendix A. We have included some new contributions beyond the leading order that are especially important for high $\tan\beta$. **SMbsg** gives the SM contribution.

- **bsmumu()**

returns the value of the branching ratio $B_s \rightarrow \mu^+\mu^-$ in the MSSM. It includes the loop contributions due to chargino, sneutrino, stop and Higgs exchange. The Δm_b effect relevant for high $\tan\beta$ is taken into account. Code is based on [118].

- **btaunu()**

computes the ratio between the MSSM and SM branching fractions for $\bar{B}^+ \rightarrow \tau^+\nu_\tau$.

- **gmuon()**

returns the value of the supersymmetric contribution to the anomalous magnetic moment of the muon.

- **R123()**

computes the ratio of the MSSM to SM value for R_{l23} in $K^+ \rightarrow \mu\nu$ due to a charged higgs contribution, see Eq.70 in [7].

- **dtaunu(&dmmunu)**

computes the branching ratio for $D_s^+ \rightarrow \tau^+\nu_\tau$. **dmmunu** gives the branching ratio for $D_s^+ \rightarrow \mu^+\nu_\mu$

- **masslimits()**

returns a positive value and prints a WARNING when the choice of parameters conflicts with a direct accelerator limits on sparticle masses from LEP. The constraint on the light Higgs mass from the LHC is included.

- **treeMSSM()**

This function transforms a loop improved model defined in an SLHA file into a tree level one. The masses and mixing angles are stored in the file **tree.slha**. This option can be useful to check the effect of loop induced masses and mixings. This option guarantees gauge invariance for all processes.

16.2 The NMSSM

As in the MSSM there are specific routines to compute the parameters of the model as specified in SLHA. The spectrum calculator is **NMSPEC** [11] in the **NMSSMTools_6.1.0** package [27].

- **nmssmEWSB()**

calculates the masses of Higgs and supersymmetric particles in the NMSSM starting from the weak scale input parameters [28]. These can be downloaded by the **readVarNMSSM** routine.

- **nmssmSUGRA(m0,mhf,a0,tb,sgn,Lambda,aLambda,aKappa)**

calculates the parameters of the NMSSM starting from the input parameters of the CMSSM.

The routines for computing constraints are taken from NMSSMTools (see details in [4]).

- **bsgnlo(&M,&P), bsmumu(&M,&P), btaunu(&M,&P), gmuon(&M,&P)**

are the same as in the MSSM case. Here the output parameters M and P give information on the lower/upper experimental limits [119]

- **deltaMd(&M,&P),deltaMs(&M,&P)**

compute the supersymmetric contribution to the $B_{d,s}^0 - \overline{B}_{d,s}^0$ mass differences, ΔM_d and ΔM_s .

- **NMHwarn(FD)**

is similar to **masslimits_** except that it also checks the constraints on the Higgs masses, returns the number of warnings and writes down warnings in the file FD.

16.3 The CPVMSSM

The independent parameters of the model include, in addition to some standard model parameters, only the weak scale soft SUSY parameters. The independent parameters are listed in **CPVMSSM/work/models/vars1.mdl**. Masses, mixing matrices and parameters of the effective Higgs potential are read directly from CPsuperH [13, 120], together with the masses and the mixing matrices of the neutralinos, charginos and third generation sfermions. Masses of the first two generations of sfermions are evaluated (at tree-level) within **micrOMEGAs** in terms of the independent parameters of the model.

The routines for computing constraints are taken from CPsuperH, [29]

- **bsgnlo(), bsmumu(), btaunu(), gmuon()**

are the same as in the MSSM case.

- `deltaMd()`, `deltaMs()`

are the same as in the NMSSM case.

- `Bd11()`

computes the supersymmetric contribution to the branching fractions for $B_d \rightarrow \tau^+ \tau^-$ in the CPVMSSM.

- `ABsg()`

computes the supersymmetric contribution to the asymmetry for $B \rightarrow X_s \gamma$.

- `EDMe1()`, `EDMmu()`, `EDMTl()`

return the value of the electric dipole moment of the electron, d_e , the muon, d_μ , and of Thallium, d_{Tl} in units of ecm .

17 Tools for model independent analysis

A model independent calculation of the DM observables is also available. After specifying the DM mass, the cross sections for DM spin dependent and spin independent scattering on proton and neutron, the DM annihilation cross section times velocity at rest and the relative contribution of each annihilation channel to the total DM annihilation cross section, one can compute the direct detection rate on various nuclei, the fluxes for photons, neutrinos and antimatter resulting from DM annihilation in the galaxy and the neutrino/muon fluxes in neutrino telescopes. All the routines presented here depend implicitly on the global parameter `Mcdm` except for `basicSpectra` and `basicNuSpectra`. They also explicitly depend on spin-independent and spin-dependent cross sections of DM scattering on proton and neutron: `csIp`, `csIn`, `csDp`, `csDn`. These cross sections have to be specified in [pb]units. A negative value for one of these cross sections is interpreted as a destructive interference between the proton and neutron amplitudes.

These routines do not take into account the multi-component structure of DM and, in particular, possible differences between DM and anti-DM. For multi-component DM the user has to perform a summation over the different DM components.

The routines available include:

- `nucleusRecoilCS( $f_v$ , A, Z, J, Sxx, csIp, csIn, csDp, csDn, dNdE)`

is similar to `nucleusRecoil`, see Section 9.2.

- `nucleusRecoil0CS( $f_v$ , A, Z, J, Sp, Sn, csIp, csIn, csDp, csDn, dNdE)` is the corresponding modification of `nucleusRecoil0`.

- `DD_pvalCS(Exp, $f_v$, Sxx, csIp, csIn, csDp, csDn, \&expName)`

and

- `DD_factorCS(Exp, pval, $f_v$, Sxx, csIp, csIn, csDp, csDn, \&expName)`

are similar to `DD_pval`, `DD_factor` described in Section 9.3.

`DD_pvalCS`, `DD_factorCS` have an option to handle a low mass mediator which modifies the recoil energy distribution, see Eq.46.

- `(*dNdEfact)(Enr, MA)`

is the address of the function which modifies the nucleus recoil distribution to take into account a t-channel propagator with small or zero mass as described in Section 9.3.

For indirect detection, we also provide a tool for model independent studies

- **basicSpectra(Mass, pdgN, outN, Spectr)**

computes the spectra of outgoing particles and writes the result in an array of dimension `NZ, Spectr`. `pdgN` is the PDG code of the particles produced in the annihilation of a pair of WIMPs. To get the spectra generated by transverse and longitudinal W 's substitute `pdgN= 24 +' T'` and `24 +' L'` correspondingly. In the same manner `pdgN= 23 +' T'` and `23 +' L'` provides the spectra produced by a polarized Z boson. `outN` specifies the outgoing particle,

$$\text{outN} = \{0, 1, 2, 3, 4, 5\} \text{ for } \{\gamma, e^+, p^-, \nu_e, \nu_\mu, \nu_\tau\}$$

The `Mass` parameter defines the mass of the DM particle. Note that the propagation routines for e^+, p^-, γ can be used after this routine as usual. Note that the result of **basicSpectra** are not valid for $M_{\text{cdm}} < 2\text{GeV}$ as explained in the description of **calcSpectrum**.

To get indirect detection signals one can substitute the obtained spectra in the `[photon/posi/pbar]FluxTab` routines.

- **captureCS(f, forSun, Mass, csIp, csIn, csDp, csDn)**

calculates the number of DM particles captured per second assuming the cross sections for spin-independent and spin-dependent interactions with protons and neutrons `csIp`, `csIn`, `csDp`, `csDn` are given as input parameters (in [pb]). A negative value for one of the cross sections is interpreted as a destructive interference between the proton and neutron amplitudes. The first two parameters have the same meaning as in the **neutrinoFlux** routine Section 13. The result depends implicitly on the global parameters `rhoDM` and `Mcdm` in Table 1.

- **basicNuSpectra(forSun, Mass, pdg, pol, nu_tab, nuB_tab)**

calculates the ν_μ and $\bar{\nu}_\mu$ spectra corresponding to the pair annihilation of DM in the center of the Sun/Earth into a particle-antiparticle pair with PDG code `pdg`. `Mass` is the DM mass. Note that this routine depends implicitly on the global parameter `WIMPSIM` (1,0,-1) which selects the neutrino spectra computed by **WimpSim** [99], **PPPC4DM ν** [100] and **DM ν** [101]. The parameter `pol` selects the spectra for polarized particles available in **PPPC4DM ν** . `pol=-1(1)` corresponds to longitudinal (transverse) polarisation of vector bosons or to left-handed (right-handed) polarisation of fermions, `pol=0` is used for unpolarized spectra. When polarized spectra are not available, the unpolarized ones are generated irrespective of the value of `pol`. The parameter `outN` is 1 for muon neutrino and -1 for anti-neutrino. The resulting spectrum is stored in the arrays `nu_tab` and `nuB_tab` with `NZ=250` elements.

The files `*.c` in the directory **mdlIndep** contain examples of the calculation of the direct detection, indirect detection and neutrino telescope signals using the routines described in this section. The numerical input data corresponds to 'MSSM/mssmh.dat'. The file **mdlIndep/mySigmaV.c** contains an example of the computation of the relic density for a constant cross-section, for this one needs to define

- **constSigmaVpbc**

which is the address of a variable which contains $\langle v\sigma \rangle [\text{pb}]$ that will be used in the routines **darkOmega**, **darkOmegaTR**, **darkOmegaInf1**. By default this parameter is `NULL`.

18 Squared Matrix Elements, cross sections, decay widths.

All squared matrix elements needed to compute cross sections and decay widths are generated with CalcHEP, which is included in micrOMEGAs.

18.1 Squared matrix elements with CalcHEP.

There are two procedures for generating the code for matrix elements, the first specifies only the process while the second allows to use composite names and to impose some restrictions:

- `numout*cc=newProcess(procName)`

compiles the codes for any $2 \rightarrow 2$ or $1 \rightarrow 2$ reaction. The result of the compilation is stored in the shared library in the directory `work/so-generated/`. The name of the library is generated automatically. The library is loaded automatically and the `newProcess` routine returns the address of the loaded code (`cc`)¹⁸. The second call to `newProcess` with the same `procName` does not lead to recompilation of the library as long as the model is not changed. Moreover the library and its name depend on the `ForceUG` flag. A multichannel process can be generated using $N \times x$, for example, `e,E->2*x`. If the process can not be compiled, then a NULL address is returned. Note that it is also possible to compute processes with polarized massless beams, for example for a polarized electron beam use `e%` to designate the initial electron.

- `cc=getMEcode(twidth, Gauge, procName,excludeVirtual,excludeOut, libName)`

compiles the codes for any $2 \rightarrow 2$ or $1 \rightarrow 2$ reaction while allowing to exclude some set of virtual and outgoing particles listed in `excludeVirtual` and `excludeOut` respectively. The `twidth` parameter allows (0) or forbids (1) t-channel widths in the generated matrix elements. The `Gauge` parameter specifies unitary or Feynman gauge. The library name has to be defined by the user. Note, that this name is included in the names of different procedures and thus cannot contain $\pm$ and other symbols not allowed for names of C-routines. The `procName` argument simulates input process in CalcHEP GUI mode. If `procName` contains words which do not correspond to particle names, these words are treated as names of composite particles and GUI session of CalcHEP requires their decoding, see Fig.2 To support this facility `procName` uses the '{' symbol to separate input lines which in GUI mode is realized by pressing the `Enter` key. To organize the input process corresponding to Fig.2 we need

```
procname=Proton,Proton -> LL,LL- {u,U,d,D{e,E,m,M
```

- `passParameters(cc)`

passes independent and constrained parameters to the code `cc` and forces it to calculate internal constraints. If calculation of some internal constrained parameters leads to NAN,

¹⁸The `struct numout` is described in `CalcHEP_src/c_source/dynamicME/include/dynamic_cs.h` and `CalcHEP_src/include/num_out.h`

```

Enter process: Proton,Proton -> LL,LL
composite 'Proton' consists of: u,U,d,D
composite 'LL' consists of: e,E,m,M
Exclude diagrams with

```

Figure 2

then non-zero error code is returned. The integration functions presented in sections 18.3, 18.4 call the `passParameters` routine automatically.

18.2 Testing the content of the code for matrix elements.

- `procInfo1(address,&ntot,&nin,&nout)`

provides information about the total number of subprocesses (`ntot`) stored in the library specified by `address` as well as the number of incoming (`nin`) and outgoing (`nout`) particles for these subprocesses. Typically, for collisions (decays), `nin=2(1)` and `nout=2,3`. `NULL` can be substitute if this information is not needed.

- `procInfo2(address,nsup,N,M)`

fills an array of particle names `N` and an array of particle masses `M` for the subprocess `nsup` ($1 \leq nsup \leq ntot$). These arrays have size $nin + nout$ and the elements are listed in the same order as in `CalcHEP` starting with the initial state, see the example in `MSSM/main.c`.

- `address->interface->pinf(nsub, np, &mass, &pdg)`

returns directly the name of np^{th} particle in subprocess `nsub`. It also returns auxilliary information about the particle: its mass¹⁹ and PDG code.

18.3 Integration of matrix elements.

The squared matrix element corresponding to the compiled code `cc` can be calculated using the function

- `cc->interface->sqme(nSub,GG,pvect,NULL,&err)`

where

- `GG` is the value of the strong coupling
- `pvect` is an array of incoming and outgoing momenta. Elements of `pvect` have type `REAL`.
- The energy of n^{th} particle is stored in `pvect[4*(n-1)] > 0`. So, the conservation law reads $\sum_{n \in in} p_n = \sum_{n \in out} p_n$
- Momenta have to be presented in `GeV` units. Then return value is presented in $GeV^{8-2(nin+nout)}$.
- `cs22(cc,nsup,P,c1,c2,&err)`

calculates the cross-section for a given $2 \rightarrow 2$ process, `nsup`, with center of mass momentum $P(\text{GeV})$. All model parameters except the strong coupling `GG` can be specified with the functions `findVal[W]/assignVal[W]` described in Section 6. The strong coupling `GG` is defined via the scale parameter `GGscale`. The differential cross-section is integrated

¹⁹Parameter `mass` must be of type of `REAL` which is defined in `CalcHEP_src/interface/nType.h`

within the range $c1 < \cos \theta < c2$. θ is the angle between $\vec{p}_1$ and $\vec{p}_3$ in the center-of-mass frame. Here $\vec{p}_1$ ($\vec{p}_3$) denote respectively the momentum of the first initial(final) particle. *err* contains a non zero error code if *nsub* exceeds the maximum value for the number of subprocesses (given by the argument *ntot* in the routine `procInfo1`). To set the polarization of the initial massless beam, define `Helicity[i]` where $i = 0, 1$ for the 1st and 2nd particles respectively. The helicity is defined as the projection of the particle spin on the direction of motion. It ranges from $[-1, 1]$ for spin 1 particles and from $[-0.5, 0.5]$ for spin 1/2 particles. By definition a left handed particle has a positive helicity.

- `hCollider(Pcm, pp, nf, Qren, Qfac, pName1, pName2, Tmin, wrt)` calculates the cross section for particle production at hadron colliders. Here *Pcm* is the beam energy in the center-of-mass frame. *pp* is 1(-1) for *pp*(*p* $\bar{p}$) collisions, $nf \leq 5$ defines the number of quark flavors taken into account. The parameters *Qren* and *Qfac* define the renormalisation and factorization scales respectively. *pName1* and *pName2* are the names of outgoing particles. If $T_{\min} \leq 0$ then `hCollider` calculates the total cross section for the 2-body final state process. Otherwise it calculates the cross section for

`proton, [a]proton -> pName1, pName2, jet`

where $T_{\min} > 0$ defines the cut on the jet transverse momentum. The jet content is defined by the parameter *nf*. If $Q_{\text{fac}} \leq 0$, then running $\hat{s}$ is used for the factorization scale. If $Q_{\text{ren}} \leq 0$, $\hat{s}$ is used for the renormalization scale for a $2 \rightarrow 2$ process and p_T of the jet is used for the renormalization scale for a process with a jet in the final state. The last argument in the `hCollider` routine allows to switch on/off (*wrt*=1/0) the printing of the contribution of individual channels to the total cross section. The value returned is the total cross section in [pb].

One of the arguments *pName1*, *pName2* can be NULL. Then the cross section for $2 \rightarrow 1$ or $2 \rightarrow 1 + \text{jet}$ process will be calculated.

By default `hCollider` uses the `cteq66` [111] structure function built-in the `micrOMEGAs` code. One can set any other parton distribution stored in `micrOMEGAs/CalcHEP_src/pdTables` (PDT-distributions) or by linking the LHAPDF library [121]. The list of PDT structure functions can be obtained with the command

- `PDTList()`

and one of these can be activated by

- `setPDT(name)`

To work with other PDF's available in LHAPDF one should recompile `micrOMEGAs` with a parameter which defines the path to the LHAPDF library:

`make LHAPDF=Path_to_LHAPDF_library.` ²⁰

`micrOMEGAs` assumes the existence of a file `$LHAPDF/lib/libLHAPDF.so`

The list of available LHAPDF distributions can be obtained with the command

- `LHAPDFList()`

and one of these can be activated by

- `setLHAPDF(name, nset)`

where *nset* specifies the subset number. Note, that if a wrong input is provided, `setLHAPDF` terminates the execution.

²⁰The path will be stored in `FlagsForSh` and `FlagsForMake` files and subsequent call of `make` without parameters in the `micrOMEGAs` directory will not clean the corresponding records.

Alternatively, one can convert a PDF distribution to the PDT format with the command `lhpdf2pdt` disposed in *CalcHEP_src/bin*.

`./lhpdf2pdt directory_with_PDF_distribution`

the generated `.pdt` file has to be placed in the folder `/CalcHEP_src/pdTables`.

The parton densities are defined by the function

- `parton_distr(pdg,x,q)`

where `pdg` is the PDG code of a particle. Note that `parton_distr` defines the parton number density and contains a factor $1/x$ with respect to the definition used in LHAPDF.

18.4 CalcHEP Monte Carlo session

The user has the possibility to run a CalcHEP Monte Carlo session replacing CalcHEP CUI interface by direct calls to C-routines. Before starting a MC session the user can specify the cuts for phase space and histograms.

- `addCut(key, Parameter, min, max)`

restricts the region of integration by the condition

`min < Parameter < max , key >= 0`
`Parameter < min or max < Parameter, key < 0`

The nomenclature for `Parameter` can be found in [51] Sect. 4.3. For instance,

`addCut(1, "M(m,M)", 500, 600),`

applies a cut on the invariant mass of μ^+ , μ^- pairs. The task to fill a histogram during a MC integration is set by the instruction

- `addHistogram(Parameter, min, max)`

The MC session is initiated by the command

- `vegasGrid * vegPtr=initMCsession(cc, nSubProc, P1,P2,STF1,STF2)`

where `nSubProc` is the number of the subprocess in the code `cc`; `P1,P2` are momenta of incoming particles; `STR1, STR2` are structure functions in CalcHEP notation(see above). For example:

`vegPtr=initMCsession(cc,1,6.8E3,6.8E3, "PDT:cteq66(proton)","PDT:cteq66(proton)")`

In case of wrongly defined process, structure functions, cuts or histograms `initMCsession` returns `NULL` or terminates the program.

Integration itself is launched by the command

- `vegas_int(vegPtr, Ncall, alpha, Nproc, &res, &dres)`

where `Ncall` is the number of calls of the squared matrix element, `NProc` is the number of processors used for parallel calculation, `res` and `dres` are respectively the cross section obtained by the MC integration and its uncertainty in [pb] units. The scale $Q = \sqrt{s}$ is used both for the QCD coupling and for structure functions. After each session `vegas` improves the integration grid. In general the first calls of `vegas_int` have a large integration uncertainty which decreases step by step in further calls. The parameter `alpha` defines the rate of grid improvement. In the case `alpha=0` the integration grid is stable. The recommended value is `alpha=1.5`. We recommend to exclude the contributions of the few first `vegas` sessions from histogram statistics. It can be done by the command

- `clearHist()`

In the end of MC calculations one can save histograms in the file

- `saveHistograms(fileName)`,

remove cuts and histograms and clean the integration grid by the commands

- `delCuts()`
- `delHistograms()`
- `vegas_finish(vegPtr)`.

The code

```
CalcHEP_src/bin/show_distr    fileName
```

allows to visualize the histograms generated.

It is to sum several distributions obtained in different sessions using

```
CalcHEP_src/bin/sum_distr fileName1 fileName2 .. > fileSum
```

One can find an example of a Monte Carlo session in `mdlIndep/MCsession.c`.

18.5 Decays

The calculation of particle widths, decay channels and branching fractions can be done by the following functions

- `pWidth(pName,&address)`

returns directly the width for particle `pName` and the returned parameter `address` gives an address where information about the branchings is stored, see below the description of routines which use this parameter. By default `pWidth` works in the following manner (note that this can be modified for each particle by the `pWidthPref` command described below):

For models which read a SLHA parameter file, the values of the widths and branchings are taken from the SLHA file when `useSLHAWidth=1` .

If `useSLHAWidth=0` or the decay of a given particle is not presented in the SLHA file, then `micrOMEGAs` applies its own calculation of width. If the 1->2 decay channels are kinematically accessible then by default only these channels are included in the width. If not, `pWidth` compiles all open 1->3 channels and use these for computing the width. If all 1->3 channels are kinematically forbidden, `micrOMEGAs` compiles 1->4 channels. If $VW_{decay}(VZ_{decay}) \neq 0$, then `micrOMEGAs` also computes the 1->3 processes with virtual $W(Z)$ and adds these to the list of 1->2 decay channels. Note that 1->3 decay channels with a virtual W will be computed even if the mass of the decaying particle exceeds the threshold for 1->2 decays by several GeV's. This is done to ensure a proper matching of 1->2 and 1->3 processes.

The routine

- `pWidthPref(pName, pref)` changes the mode of the `pWidth` calculation individually for the particle `pName`. Here `pref=4` restores the default `pWidth` mode. Otherwise the `useSLHAWidth` flag is ignored and for `pref=0,1,2,3` we have

- 0 - widths are calculated using processes with minimal number of outgoing particles.
- 1 - widths are calculated using processes with minimal and next to minimal number of outgoing particles excluding processes with s-channel resonances to avoid double counting.
- 2- widths are read from the SLHA file - if the SLHA file does not contain widths, they are calculated as in `pref=0` case.

- 3- widths are read from the SLHA file - if the SLHA file does not contain widths, they are calculated as in `pref=1` case.
- `printTxtList(address,FD)`
lists the decays and their branching fractions and writes them in a file. `address` is the address returned by `pWidth`.
- `findBr(address,pattern)`
finds the branching fraction for a specific decay channel specified in `pattern`, a string containing the particle names in the CalcHEP notation. The names are separated by commas or spaces and can be specified in any order. If the `pattern` contains "*", then `findBr` sums all branching fractions which contain particles included in the `pattern`.
- `printPartialWidth(width,address,FD)`
prints the branching fractions and partial widths for each decay channel of a particle in the file `FD`. Here `width` and `address` are the result of the function `pWidth` described above.
- `slhaDecayPrint(pname,delVirt,FD)`
uses `pWidth` described above to calculate the width and branching ratios of particle `pname` and writes down the result in SLHA format. The return value is the PDG particles code. In case of problem, for instance wrong particle names, this function returns zero. This function first computes $1 \rightarrow 2$ decays. If such decays are kinematically forbidden then $1 \rightarrow 3$ decay channels are computed. Decays via virtual W/Z bosons will be listed via their decay products when `delVirt` $\neq$ 0.

18.6 Extraction of couplings for vertices.

The user might need information about the couplings entering a given vertex. It can be done by two commands, the first to get the symbolic expression entering the vertex, the second to extract the numerical values.

- `lVert* VVV =getVertex(pN1,pN2,nP3,pN4);`
extracts the symbolic expression in the four- vertex where `pNi` are the names of particles in the vertex. For three-leg vertices set `pN4=NULL`. The command returns the address of the structure which contains the following information

```
VVV->nTerms      - the number of terms
VVV->SymbVert[k] - the symbolic formula for the terms, 0<=k<VVV->nTerms
```

For example, to get information on the $E e Z$ vertex which in `lgrng1.mdl` is written as

E	e	Z		EE/(4*CW*SW)	4*SW^2*G(m3)-G(m3)*(1-G5)
---	---	---	--	--------------	---------------------------

we use the command

```
lVert* VVV=getVertex("E","e","Z",NULL);
```

which will return

```
VVV->GGpower=0
VVV->nTerms=2
VVV->SymbVert[0]="G(m3)"
VVV->SymbVert[1]="G5*G(m3)"
```

Note that the particle ordering in `getVertex` does not have to match the one in the Lagrangian and the Lorentz index corresponds to the position of the vector particles in the call of `getVertex`, not to their position in the Lagrangian table.

- `getNumCoeff(VVV,coeff);`

provides the numerical coefficients for all `SymbVert`, here the numerical array `coeff` should contain at least `VVV->nTerms` elements. Note that `GGpower` - the power of strong coupling included in the vertex - does not contribute to the numerical coefficient.

19 Implementation of a new model

It is possible to implement a new particle physics model in `micrOMEGAs`. For this the model must be specified in the `CalcHEP` format. `micrOMEGAs` then relies on `CalcHEP` to generate the libraries for all matrix elements entering DM calculations. Below we describe the main steps and tools for implementing a new model.

19.1 Main steps

- The command `./newProject MODEL` launched from the root `micrOMEGAs` directory creates the directory `MODEL`. This directory and the subdirectories contain all files needed to run `micrOMEGAs` with the exception of the new model files.
- The new model files in the `CalcHEP` format should then be included in the subdirectory `MODEL/work/models`. The files needed are `vars1.mdl`, `func1.mdl`, `prtcls1.mdl`, `lgrng1.mdl`, `extlib1.mdl`. For more details on the format and content of model files see [1].
- For odd particles and for the Higgs sector it is recommended to use the widths that are (automatically) calculated internally by `CalcHEP/micrOMEGAs`. For this one has to add the `'!`' symbol before the definition of the particle's width in the file `prtcls1.mdl`, for example

Full	name		P		aP		PDG			2*spin		mass		width		color	
Higgs	1		h1		h1		25			0		Mh1		!wh1		1	

- Some models contain external functions, if this is the case they have to be compiled and stored in the `MODEL/lib/aLib.a` library. These functions should be written in C and both functions and their arguments have to be of type `double`. The library `aLib.a` can also contain some functions which are called directly from the `main` program. The `MODEL/Makefile` automatically launches `make` in the `lib` directory and compiles the external functions provided the prototypes of these external functions are specified in `MODEL/lib/pmodel.h`. The user can of course rewrite his/her own `lib/Makefile` if need be.

If the new `aLib.a` library needs some other libraries, their names should be added to the `SSS` variable defined in `MODEL/Makefile`.

The `MODEL` directory contains samples of *main* programs, `main.c` and `main.cpp`. In these sample main programs it is assumed that the input parameters are provided in a separate file. In this case the program can be launched with the command:

```
./main data1.par
```

Note that for the direct detection module all quarks must be massive. However the cross sections do not depend significantly on the exact numerical values for the masses of light quarks.

19.2 Tools for model files generation

For models with a large number of parameters and various types of fields/particles such as the MSSM, it is more convenient to use an automatic tool to implement the model. Such tools are `LanHEP` [31], `FeynRules` [122], and `SARAH` [123]. `LanHEP` is included in `micrOMEGAs` and can be found in `Packages/LanHEP`. It can be compiled by calling `make` which generates the executable file `lhep`. Generation of model files in `CalcHEP`/`micrOMEGAs` notations is launched by the command

```
lhep -ca source_file
```

Most of the models included in `micrOMEGAs` are accompanied with the `LanHEP` sources that can be found in the `work/lanhep` directory. The user can find instructions for writing `LanHEP` source files in the directory `Packages/LanHEP/manuals`

19.3 Automatic width calculation

Automatic width calculation can be implemented by inserting the `'!'` symbol before the name of the particle width in the `CalcHEP` particle table (file `prtcls1.mdl`). In this case the width parameter should not be defined as a free or constrained parameter. Actually the `pWidth` function described in section 18 is used for width calculation in this case. We recommend to use the automatic width calculation for all particles from the 'odd' sector and for Higgs particles. For models which use `SLHA` parameter transfer (Section 19.6), the automatic width option will use the widths computed internally unless the user chooses to use those contained in the `SLHA` file by setting `useSLHAwidth=1`. See details in Section 18.5.

19.4 One-loop scalar vertices

The loop-induced couplings of scalars and pseudoscalars to gluons and photons can be calculated internally by `micrOMEGAs` with the use of the functions described below. First, effective vertices must be defined in the model file. These vertices correspond to the Lagrangian

$$\mathcal{L} = \lambda h F_{\mu\nu} F^{\mu\nu} + \lambda_5 h F_{\mu\nu} \tilde{F}^{\mu\nu} \quad (77)$$

where the first term is the effective operator for the CP-even part while the second term is for the CP-odd part. Similar operators can be defined for the couplings to gluons with $F^{\mu\nu}$ replaced by $G^{\mu\nu}$. In the `CalcHEP` notation, the vertices will appear in the file `lgrng1.mdl` as

```
A      |A      |h      |      |-4*LAAh      |p1.p2*m1.m2-m2.p1*m1.p2
```

A |A |h | | -4*LAA5h |eps(p1,m1,p2,m2)

where LAAh, LAA5h correspond respectively to λ, λ_5 in Eq. 77. These coefficients are computed internally by micrOMEGAs with the functions

•lAAhiggs(Mass,Name)

which calculates the coefficient of the loop-induced Scalar - photon - photon vertex based on the generic contributions of scalars/fermions/vector bosons given in Ref. [124]. Here Name specifies the scalar and Mass its mass. To do this, micrOMEGAs searches the model file to find the couplings of the scalar to all possible coloured scalars/fermions or vector bosons that can enter the one-loop process.

•lAA5higgs(Mass,Name)

which calculates the coefficient of the loop-induced Pseudoscalar - photon - photon vertex based on the results in Ref. [124] where Name specifies the pseudoscalar and Mass its mass.

As explained in [7, 51], an overall QCD correction factor is included for coloured particles in the loop. HDECAY [109] is used to generate this factor and is therefore tuned for the SM Higgs.

Similar effective operators can be defined for hgg vertices. The vertex describing the couplings of the Higgs to gluon pairs is defined

G |G |h | | -4*LGGh*RQCDh |p1.p2*m1.m2-m2.p1*m1.p2

where RQCDh is the NLO QCD correction corresponding to the radiation of gluons for the SM Higgs. Other NLO QCD vertex corrections for heavy quarks, light quarks and coloured scalars are included in the coefficient of the vertex as described in [7, 51]. As for the photon, the coefficients including NLO QCD vertex corrections are computed internally by micrOMEGAs with the functions

•lGGhiggs(Mass,Name)

which calculates the coefficient of the loop-induced Scalar - gluon - gluon vertex based on the formulae in Ref. [124] where Name specifies the scalar and Mass its mass.

•lGG5higgs(Mass,Name)

which calculates the coefficient of the loop-induced Pseudoscalar - gluon - gluon vertex based on the formulae in Ref. [124] where Name specifies which scalar and Mass its mass.

The coefficient of the vertices should be defined in func1.mdl, for example for the scalar vertices

LAAh | -cabs(lAAhiggs(Mh, "h"))

LGGh | -cabs(lGGhiggs(Mh, "h"))

and the overall QCD corrections are defined with

aQCDh | alphaQCD(Mh)/acos(-1)

RQCDh | sqrt(1+149/12*aQCDh+68.6482*aQCDh^2-212.447*aQCDh^3) |

where $\alpha_{\text{QCD}}(M_h)$ is the strong coupling, $\alpha_s(m_h)$ evaluated at the Higgs mass. Note that this construction is tuned for Higgs decays. In particular the RQCDh factor is caused by hadronization of gluons. For $g, g \rightarrow H$ production, other NLO factors need to be introduced by the user. The difference between NLO production and decays is ignored in micrOMEGAs. The scale entering the definition of α_s can be adapted to the process under consideration.

The four functions for loop-induced vertices were not available in micrOMEGAs' versions before v5.1, hence most of the models implemented use the functions introduced in [7] where the user had to give by hand the names of all particles that could contribute to the triangle diagrams. The IDM model is an example where this newer and simpler implementation of the loop-induced vertices can be found.

Note that in the interface to Lilith and HiggsBounds, the loop induced couplings to the Higgs are calculated by the LiLithMO/hbBlocksMO routines whether or not the Hgg and $H\gamma\gamma$ vertices are already implemented in the Lagrangian, as mentioned in section 15.1.

19.5 QCD functions

Here we describe some QCD functions which can be useful for the implementation of a new model.

- `initQCD(alfsMZ,McMc,MbMb,Mtp)`

This function initializes the parameters needed for the functions listed below. It has to be called before any of these functions. The input parameters are the QCD coupling at the Z scale, $\alpha_s(M_Z)$, the quark masses, $m_c(m_c)$, $m_b(m_b)$ and $m_t(pole)$.

- `alphaQCD(Q)`

calculates the running α_s at the scale Q in the $\overline{MS}$ scheme. The calculation is done using the N^4LO (5 loops) formula in [125, 126]. For $\alpha_{n_{df}-1}(\alpha_{n_f})$ at flavour thresholds $m_c(m_c)$, $m_b(m_b)$, $m_t(pole)$ we use formulas [127].

- `MtRun(Q)`, `MbRun(Q)`, `McRun(Q)`

calculates top, bottom and charm quarks running masses evaluated at N^4LO [?].

- `MtEff(Q)`, `MbEff(Q)`, `McEff(Q)`,

calculates effective top, bottom and charm quark masses using [125]

$$\begin{aligned} M_{eff}^2(Q) &= M(Q)^2 \left[1 + 5.67a + (35.94 - 1.36n_f)a^2 \right. \\ &\quad \left. + (164.14 - n_f(25.77 - 0.259n_f))a^3 \right] \end{aligned} \quad (78)$$

where $a = \alpha_s(Q)/\pi$, $M(Q)$ and $\alpha_s(Q)$ are the quark masses and running strong coupling in the $\overline{MS}$ -scheme. In micrOMEGAs, we use the effective quark masses calculated at the scale $Q = 2M_{cdm}$. In some special cases one needs a precise treatment of the light quarks masses. The function

- `MqRun(M2GeV, Q)`

returns the running quark mass defined at a scale of 2 GeV. The corresponding effective mass needed for the Higgs decay width is given by

- `Mqeff(M2GeV, Q)`

19.6 SLHA reader

Very often the calculation of the particle spectra for specific models is done by some external program which writes down the particle masses, mixing angles and other model parameters in a file with the so-called **SLHA** format [56, 116]. The micrOMEGAs program contains routines for reading files in the SLHA format. Such routines can be very useful for the implementation of new models.

In general a SLHA file contains several pieces of information which are called blocks. A block is characterized by its name and, sometimes, by its energy scale. Each block contains

the values of several physical parameters characterized by a *key*. The key consists in a sequence of integer numbers. For example:

```
BLOCK MASS    # Mass spectrum
#  PDG Code    mass          particle
      25      1.15137179E+02  # lightest neutral scalar
      37      1.48428409E+03  # charged Higgs
```

```
BLOCK NMIX    # Neutralino Mixing Matrix
  1  1      9.98499129E-01  # Zn11
  1  2     -1.54392008E-02  # Zn12
```

```
BLOCK Au Q= 4.42653237E+02  # The trilinear couplings
  1  1     -8.22783075E+02  # A_u(Q) DRbar
  2  2     -8.22783075E+02  # A_c(Q) DRbar
```

- `slhaRead(filename,mode)`

downloads all or part of the data from the file `filename`. `mode` is an integer which determines which part of the data should be read from the file, `mode= 1*m1+2*m2+4*m4` where

```
m1 = 0/1 - overwrites all/keeps old data
m2 = 0/1 - reads DECAY /does not read  DECAY
m4 = 0/1 - reads BLOCK/does not read  BLOCK
```

For example `mode=2` (`m1=0,m2=1`) is an instruction to overwrite all previous data and read only the information stored in the BLOCK sections of `filename`. In the same manner `mode=3` is an instruction to add information from DECAY to the data obtained previously. `slhaRead` returns the values:

```
0 - successful reading
-1 - can not open the file
-2 - error in spectrum calculator
-3 - no data
n>0 - wrong file format at line n
```

- `slhaValExists(BlockName, keylength, key1, key2,...)`

checks the existence of specific data in a given block. `BlockName` can be substituted with any case spelling. The `keylength` parameter defines the length of the key set `{key1,key2,...}`. For example `slhaValExists("Nmix",2,1,2)` will return 1 if the neutralino mass mixing element `Zn12` is given in the file and 0 otherwise.

- `slhaVal(BlockName,Q, keylength, key1, key2,.....)`

is the main routine which allows to extract the numerical values of parameters. `BlockName` and `keylength` are defined above. The parameter `Q` defines the scale dependence. This parameter is relevant only for the blocks that contain scale dependent parameters, it will be ignored for other blocks, for example those that give the particle pole masses. In general a SLHA file can contain several blocks with the same name but different scales (the scale is specified after the name of the block). `slhaVal` uses the following algorithm to read the scale dependent parameters. If `Q` is less(greater) than all the scales used in the different blocks for a given parameter `slhaVal` returns the value corresponding to

the minimum(maximum) scale contained in the file. Otherwise `slhaVal` reads the values corresponding to the two scales Q_1 and Q_2 just below and above Q and performs a linear interpolation with respect to $\log(Q)$ to evaluate the returned values.

Recently it was proposed to use an extension of the SLHA interface to transfer Flavour Physics data [128]. Unfortunately the structure of the new blocks is such that they cannot be read with the `slhaVal` routine. We have added two new routines for reading such data

- `slhaValFormat(BlockName, Q, format)`

where the *format* string allows to specify data which one would like to extract from the given block `BlockName`. For instance, to get the $b \rightarrow s\gamma$ branching ratio from the block

Block FOBS # Flavour observables

```
# ParentPDG type value      q    NDA ID1 ID2 ID3 ... comment
   5      1  2.95061156e-04  0     2   3  22      # BR(b->s gamma)
  521     4  8.35442304e-02  0     2 313  22      # Delta0(B->K* gamma)
  531     1  3.24270419e-09  0     2  13 -13      # BR(B_s->mu+ mu-)
...

```

one has to use the command `slhaValFormat("FOBS", 0., "5 1 %E 0 2 3 22")`. In this command the *format* string is specified in C-style. The same routine can be used to read HiggsBound SLHA output.

A block can also contain a textual information. For example, in HIGGSBOUNDS a block contains the following records,

Block HiggsBoundsResults

```
#CHANNELTYPE 1: channel with the highest statistical sensitivity
  1      1      328      # channel id number
  1      2      1      # HBresult
  1      3  0.72692779334500290  # obsratio
  1      4      1      # ncombined
  1      5 ||(p p)->h+..., h=1 where h is SM-like (CMS-PAS-HIG-12-008)|| # text description of channel

```

In particular, the last record contains the name of the channel which gives the strongest constraint on the Higgs. To extract the name of this channel one can use the new function

- `slhaSTRFormat("HiggsBoundsResults", "1 5 || %[^\n]||", channel);`

which will write the channel name in the text parameter *channel*.

- `slhaWarnings(fileName)`

writes into the file the warnings or error message stored in the SPINFO block and returns the number of warnings. If `FD=NULL` the warnings are not written in a file.

- `slhaWrite(fileName)`

writes down the information stored by `readSLHA` into the file. This function can be used for testing purposes.

SLHA also describes the format of the information about particle decay widths. Even though `micrOMEGAs` also performs the width calculation, one might choose to read the information from the SLHA file.

- `slhaDecayExists(pNum)`

checks whether information about the decay of particle `pNum` exists in the SLHA file. `pNum` is the particle PDG code. This function returns the number of decay channels. In particular zero means that the SLHA file contains information only about the total width, not on branching ratios while -1 means that even the total width is not given.

- `slhaWidth(pNum)`

returns the value of particle width.

- `slhaBranch(pNum, N, nCh)`

returns the branching ratio of particle `pNum` into the `N`-th decay channel. Here $0 < N \leq \text{slhaDecayExists}(pNum)$. The array `nCh` is an output which specifies the PDG numbers of the decay products, the list is terminated by zero.

The functions `slhaValExists`, `slhaVal`, `slhaDecayExists`, `slhaWidth` can be used directly in CalcHEP model files, see an example in `MSSM/work/models/func3.mdl`. Note that in this example the call to `slhaRead` is done within the function `suspectSUGRAc`.

19.6.1 Writing an SLHA input file

We have included in the `micrOMEGAs` package some routines which allow to write an SLHA input file and launch the spectrum generator via the CalcHEP *constraints* menu. This way a new model can be implemented without the use of external libraries. The routines are called from `func1.mdl`, see example below.

- `openAppend(fileName)`

deletes the input file `fileName` and stores its name. This file will then be filled with the function `aPrintf`.

- `aPrintf(format, ...)`

opens the file `fileName` and writes at the end of the file the input parameters needed in the SLHA format or in any other format understood by the spectrum calculator. The arguments of `aPrintf` are similar to the arguments of the standard `printf` function.

- `System(command, ...)` generates a command line which is launched by the standard `system` C-function. The parameter *command* works here like a format string and can contain `%s`, `%d` elements. These are replaced by the next parameters of the `System` call.

For example to write directly the SLHA model file needed by `SuSpect` to compute the spectrum in a CMSSM(SUGRA) model, one needs to add the following sequence in the `func1.mdl` model file.

```
open |openAppend("suspect2_lha.in")
input1|aPrintf("Block MODSEL # Select model\n 1 1 # SUGRA\n")
input2|aPrintf("Block SMINPUTS\n 5 %E#mb(mb)\n 6 %E#mt(pole)\n",MbMb,Mtp)
input3|aPrintf("BLOCK MINPAR\n 1 %E #m0\n 2 %E #m1/2\n ",Mzero,Mhalf)
input4|aPrintf("3 %E #tb\n 4 %E #sign(mu)\n 5 %E #A0\n",tb,sgn,A0)
sys |System("./suspect2.exe")
rd |slhaRead("suspect2_lha.out",0)
```

It is possible to cancel the execution of a program launched with `System` if it runs for too long. For this we have introduced two global parameters `sysTimeLim` and `sysTimeQuant`. `sysTimeLim` sets a time limit in milliseconds for `System` execution, if `sysTimeLim==0` (the default value) the execution time is not checked. The time interval between checks of the status of the program launched with `System` is specified by the parameter `sysTimeQuant`, the default value is set to 10. Note that it is preferable not too use too large a value for `sysTimeQuant` as it defines the lower time limit for a system call.

The function prototypes are available in
`CalcHEP_src/c_source/SLHApplus/include/SLHApplus.h`

19.7 Routines for diagonalisation

Very often in a new model one has to diagonalize mass matrices. Here we present some numerical routines for diagonalizing matrices. Our code is based on the `jacobi` routine provided in [129]. To use the same routine for a matrix of arbitrary size, we use a C option that allows to write routines with an arbitrary number of arguments.

- `initDiagonal()` should be called once before any other `rDiagonal(A)` routine described below. `initDiagonal()` assigns zero value to the internal counter of eigenvalues and rotation matrices. Returns zero.

- `rDiagonal(d,M11,M12,..M1d,M22,M23...Mdd)`

diagonalizes a symmetric matrix of dimension `d`. The $d(d+1)/2$ matrix elements, `Mij` ($i \leq j$), are given as arguments. The function returns an integer number `id` which serves as an identifier of eigenvalues vector and rotation matrix.

- `MassArray(id, i)`

returns the eigenvalues m_i ordered according to their absolute values.

- `MixMatrix(id,i,j)`

returns the rotation matrix R_{ij} where

$$M_{ij} = \sum_k R_{ki} m_k R_{kj}$$

A non-symmetric matrix, for example the chargino mass matrix in the MSSM, is diagonalized by two rotation matrices,

$$M_{ij} = \sum_k U_{ki} m_k V_{kj}.$$

- `rDiagonalA(d,M11,M12..M1d,M21,M22...Mdd)`

diagonalizes a non-symmetric matrix, the d^2 matrix elements, `Mij`, are given as arguments. The eigenvalues and the V rotation matrix are calculated as above with `MassArray` and `MixMatrix`.

- `MixMatrixU(id,i,j)`

returns the rotation matrix U_{ij} .

One can find example of work of diagonalization routines in `mdlIndep/diagonal.c`. The function prototypes can be found in

`CalcHEP_src/c_source/SLHApplus/include/SLHApplus.h`

20 Mathematical tools

Some mathematical tools used by `micrOMEGAs` are available only in C format. Prototypes of these functions can be found in

`include/micromegas_aux.h`

20.1 Integration

- `simpson(F, x1, x2, eps,&err)`

performs numerical integration of the function $F(x)$ in the interval $[x1, x2]$. `simpson` tries to reach relative precision `eps` or absolute precision $\frac{eps}{10} \int_{x1}^{x2} |F(x)| dx$. For one interval

simpson compares the results of 3, 5 and 9 points formulas. If they do not agree within the required precision, **simpson** splits the interval in two and applies the same procedure recursively. The smallest interval is $|x_2 - x_1|/2^{25-\log(\epsilon)}$.

A non-zero error code *err* means

- 1: NAN in integrand: **simpson** replaces NAN by zero and continues integration
- 2: Too deep recursion: one needs the smallest minimal interval to reach the required precision. For instance, these two error codes will show up in the integration of $\int dx/|x|^{0.8}$ with a precision $\epsilon = 10^{-3}$.
- 4: Lost of precision: If in a small interval $\approx |x_1 - x_2|/2^{10}$ the derivative of the function changes sign more than twice, it is treated as a lost of precision in the integrand and **simpson** decreases precision to $\epsilon=0.01$ and treats uncertainties of each interval as a random numbers.

In case of several error messages, *err* contains their sum. If the last argument of **simpson** is *NULL*, the error messages are displayed on the screen. If the integrand has a discontinuity or a pole, the requirement of reaching a fixed relative precision can not be satisfied. Thus we also stop the recursion if the integration uncertainty on the interval is small as compared to the current estimation of $\frac{\epsilon}{10} \int_{x_1}^{x_2} |F(x)| dx$. Since the current estimation of this integral can be significantly smaller than the final one, we keep the sum of errors obtained at the end of the recursion chains and compare it with the final estimation of the total integral. If the sum of errors is small, then the error 2 is not generated.

We have implemented a debugging tool for the **simpson** code based on **gdb**. To initiate it the user has to launch the `./main` code under **gdb** and set a breakpoint

```
gdb ./main
break verifySimpson
r inputParameters           // to launch code
```

The execution of the program will stop when **simpson** generates an error code. With the **bt** command one can see the sequence of calls leading to the problematic call of **simpson**. After

```
n           // next
```

command one reaches the cycle which is executed until the variable `show` $\neq$ 0. The user should see on the screen

```
293 while(show) {drawP(f, par, x1, x2,ans,nErr);
```

By default `show=0` and the cycle is not executed, but it can be changed by the **set** command

```
set show=1
n
```

and the user will see a plot of the function $F(x)$ on the screen. When the user closes the window with the plot, he/she will reach the next line of code

```
294 continue; }
```

Here the user can leave the cycle using the commands

```
set show=0
c                // continue
```

or verify a narrower range of the function $F(x)$ by redefining the variables **x1** and **x2**.

The file `mdlIndep/simpson.c` contains several examples which allow to test how this routine works.

- `gauss(F,x1,x2,N)`

performs Gauss N-point integration for $N < 8$.

- `peterston21(F, x1, x2,&aErr)`

performs numerical integration of the function $F(x)$ in the interval $[x1, x2]$ using Gauss-Kronrod-Peterson formula with 21 points. `aErr` is a parameter which returns the accuracy of the calculation using 10 points subset.

20.2 Solution of differential equations.

- `odeint(Y, Dim, x1, x2, eps,h1, deriv)`

solves a system of `Dim` differential equations in the interval $[x1, x2]$. The `Dim` component array `Y` contains the starting variables at `x1` as an input and is replaced by the resulting values at `x2` as an output. `eps` determines the precision of the calculation and `h1` gives an estimation of step of integration. The function `deriv` calculates $Y'_i = dY_i/dx$ with the call `deriv(x,Y,Y')`. The Runge-Kutta method is used, see details in [129].

- `stiff ( first, x1, x2, Dim, Y, Yscal, eps, &htry,derivs)`

- `stifbs( first, x1, x2, Dim, Y, Yscal, eps, &htry,derivs)`

these two functions solve stiff differential equations. Both routines are slightly adapted codes from [129]. Here the parameters `x1`, `x2`, `Dim`, `Y` have the same meaning as in the routine `odeint` above. The parameter `first` should be set to one for the first call to the routines with a given number of equations `Dim` and to zero for subsequent calls. The flag `first` is used for memory allocation. If `Yscal=NULL` the parameter `eps` defines the absolute precision of calculation ($\delta Y_i < eps$). Otherwise, the precision is defined by the condition $\delta Y_i < eps Y_{scale_i}$. The parameter `htry` defines the initial step of integration and contains the last step of integration used during calculations. The function `derivs` evaluates the differential equation $F = dY/dx$ and its partial derivatives:

$$\text{derivs}(x, Y, F, h, \text{dFdx}, \text{dFdY}) \text{ where } \text{dFdY}[i*\text{Dim}+j] = \frac{dF_i}{dY_j}$$

This routine can be called with parameters `dFdx=NULL` and `dFdY=NULL`. The parameter `h` presents current step of integration and can be used for numerical evaluation of `dFdx`.

20.3 Interpolation

- `polint3(x,Dim,X,Y)`

performs cubic interpolation for *Dim*-dimension arrays `X,Y`. Similar functions, `polint1` performs linear interpolations.

- `spline(x, y, dim, y2)`

- `splint(x, y, y2, dim, double x0, &y0)`

`spline` constructs cubic spline and `splint` calculates spline interpolation `y0` for a given point `x0`. Here `x` and `y` are a grid of function arguments and function values $y_i = Y(x_i)$. The function `spline` fills an array of second derivatives `y2` which is used by `splint`.

- `buildInterpolation(F,x1,x2,eps,delt, &Dim,&X,&Y)`

constructs a cubic interpolation of the function `F` in the interval $[x1, x2]$. `eps` controls the

precision of interpolation. If $eps < 0$ the absolute precision is fixed, otherwise a relative precision is required. The $delt$ parameter limits the distance between interpolation points: $|x_i - x_{i+1}| < delt|x_2 - x_1|$. The function checks that after removing any grid point, the function at that point can be reproduced with a precision eps using only the other points. It means that the expected precision of interpolation is about $eps/16$. Dim gives the number of points in the constructed grid. X and Y are variables of the `double*` type. The function allocates memory for the Dim array for each of these parameters. $X[i]$ contains the x-grid while $Y[i] = F(X[i])$.

20.4 Functions with additional parameters

We use routines that work with functions of type `double F(double x,void*arg)`, where the structure of `arg` is open, here `arg` presents the memory address where these parameters are stored. This is useful for example for integrating a function $f(x,y)$ over x , y is then passed as 'arg'. The functions available are `simpson_arg`, `peterson21_arg`, `buildInterpolation_arg`. The additional argument is the address `arg` which is specified just after the function name.

The recent versions of C-compiler may refuse the substitution of a function if the type of the argument does not correspond. For example, if there is a problem for the function

```
double myF(double x, double *par)
```

to be used in `simpson_arg`, we recommend to use a type conversion:

```
simpson_arg( (funcArg) myF, ....)
```

20.5 Plots

• `displayPlot(title,xName,xMin,xMax,lScale, N, ...)`

displays several curves/histograms on one plot. Here `title` contains some text, `xName` is the name of a variable, `xMin`, `xMax` are the lower and upper limits. If `lScale` $\neq 0$, a logarithmic scale is used for the x axis.

N is the number of curves/histograms to display, It should not exceed 15. After the parameter N , `displayPlot` expects $N \times 4$ parameters, where each tetrad can contain

text label	dimension of array	array of data	array of error
text label	dimension of array	array of data	NULL
text label	0	double *f(double x)	NULL
text label	0	double *f(double x, void *arg)	arg

where the first line is used for an histogram with error bars, the second line for a tabulated function, the third for a function $f(x)$ and the fourth for a function $f(x,arg)$ which also depends on some arguments contained in the structure `arg`. For a linear scale `lScale=0`, the arrays of data and errors should correspond to a grid

$$x_i = xMin + (i + 0.5)(xMax - xMin)/Dim ,$$

where $i = 0, \dots, Dim - 1$. For logarithmic scale

$$x_i = xMin \cdot \left(\frac{xMax}{xMin} \right)^{\frac{i+0.5}{Dim}}$$

`displayPlot` uses the X11 font presented in the file `CalcHEP_src/calchep.ini`. One can change the font by editing the file or via key signals: `Ctrl±` increases/decreases font size, while `Ctrl<` - changes the font saving its size.

20.6 Bessel functions

- `bessI0(x)`, `bessI1(x)`, `bessK0(x)`, `bessK1(x)`, `bessK2(x)`

Bessel functions I_0 , I_1 , K_1 , K_2 .

- $K1pol(x) = K_1(\frac{1}{x})e^{\frac{1}{x}}\sqrt{\frac{2}{\pi x}}$

- $K2pol(x) = K_2(\frac{1}{x})e^{\frac{1}{x}}\sqrt{\frac{2}{\pi x}}$

`micrOMEGAs` uses these functions for calculating the relic density. For small values of $x = T/M_{cdm}$, where these functions can be presented as polynomials in x . $K1pol(0) = K2pol(0) = 1$.

20.7 Statistics

- `FeldmanCousins(n0, b, cl)`

is the Feldman-Cousins [130] function for a Poisson distribution. Here `n0` is the observed number of events, `b` - the expected background, `cl` < 1 - the requested confidence level. This function sets the upper limit on the number of signal events compatible with `n0` and `cl`.

- `ch2pval(ndf, chi2)`

returns the `p-value` assuming a χ^2 distribution with `ndf` degrees of freedom (expected χ^2) and `chi2` is the observed χ^2 .

$$ch2pval(k, q) = \int_q^\infty \frac{1}{2^k \Gamma(\frac{k}{2})} Q^{\frac{k}{2}-1} e^{-\frac{Q}{2}} dQ$$

A An updated routine for $b \rightarrow s\gamma$ in the MSSM

The calculation of $b \rightarrow s\gamma$ was described in micromegas1.3 [3]. The branching fraction reads

$$B(\bar{B} \rightarrow X_s \gamma) = B(\bar{B} \rightarrow X_c e \bar{\nu}) \left| \frac{V_{ts}^* V_{tb}}{V_{cb}} \right|^2 \frac{6\alpha_{em}}{\pi f(z_0)} K_{NLO}(\delta) \quad (79)$$

where $\alpha_{em} = 1/137.036$, the factor K_{NLO} involves the photon energy cut-off parameter δ and $f(z_0) = 0.542 - 2.23(\sqrt{z_0} - 0.29)$ depends on $z_0 = (m_c/m_b)^2$ defined in terms of pole masses. In the code the standard model and Higgs contribution at NLO were included as well as the leading order SUSY contributions. However in the last few years the NNLO standard model contribution has been computed [131] and shown to lead to large corrections, shifting the standard model value by over 10%. It was also argued that the NNLO SM result could be reproduced from the NLO calculation by appropriately choosing the scale for the c-quark mass [132, 133].

In this improved version of the `bsgnlo` routine, we have changed the default value for the parameter $z_1 = (m_c/m_b)^2$ where m_c is the $\overline{MS}$ running charm mass $m_c(m_b)$. Taking $z_1 = 0.29$ allows to reproduce the NNLO result. It is therefore no longer necessary to apply a shift to the `micrOMEGAs` output of $b \rightarrow s\gamma$ to reproduce the SM value.

We have also updated the default values for the experimentally determined quantities in Eq. 79, see Table 10, and we have replaced the factor $f(z_0)$ by C_{sl} where

$$C_{sl} = \left| \frac{V_{ub}}{V_{cb}} \right|^2 \frac{\Gamma(\bar{B} \rightarrow X_c e \bar{\nu})}{\Gamma(\bar{B} \rightarrow X_u e \bar{\nu})} \quad (80)$$

accounts for the m_c dependence in $\bar{B} \rightarrow X_c e \bar{\nu}$.

$B(\bar{B} \rightarrow X_c e \bar{\nu})$	0.1064 [55]
C_{sl}	0.546 [133]
$ V_{ts}^* V_{tb}/V_{cb} ^2$	0.9613 [55]
A	0.808
λ	0.2253
$\bar{\rho}$	0.132
$\bar{\eta}$	0.341
m_b/m_s	50
$\lambda_2 \approx \frac{1}{4}(m_{B^*}^2 - m_B^2)$	0.12 GeV ² [134]
$\alpha_s(M_Z)$	0.1189

Table 10: Default values in micrOMEGAs

The CKM matrix elements in the Wolfenstein parametrisation given in Table 10 are used to compute the central value of $ckmf$ at order λ^4 ,

$$ckmf = \left| \frac{V_{ts}^* V_{tb}}{V_{cb}} \right|^2 = 1 + \lambda^2(2\bar{\rho} - 1) + \lambda^4(\bar{\rho}^2 + \bar{\eta}^2 - A^2) \quad (81)$$

With these default values the NLO- improved SM contribution is $B(\bar{B} \rightarrow X_s \gamma)|_{\text{SM}} = 3.27 \times 10^{-4}$ which corresponds to the result of Gambino and Giordano [133] after correcting for the slightly different CKM parameter used ($ckmf = 0.963$).

We have performed a comparison with superIso which includes the NNLO SM calculation for 10^5 randomly generated MSSM scenarios. The results are presented in Fig. 3 after applying a correction factor in superISO to account for the different value for the overall factor $F = B(\bar{B} \rightarrow X_c e \bar{\nu}) \left| \frac{V_{ts}^* V_{tb}}{V_{cb}} \right|^2 / C_{sl}$. The ratio of $F_{\text{micro}}/F_{\text{ISO}} = 0.942$. The two codes agree within 5% most of the time.

References

- [1] A. Pukhov, “CalcHEP 2.3: MSSM, structure functions, event generation, batches, and generation of matrix elements for other packages,” [arXiv:hep-ph/0412191 \[hep-ph\]](#).
- [2] G. Belanger, F. Boudjema, A. Pukhov, and A. Semenov, “MicrOMEGAs: A Program for calculating the relic density in the MSSM,” *Comput. Phys. Commun.* **149** (2002) 103–120, [arXiv:hep-ph/0112278 \[hep-ph\]](#).

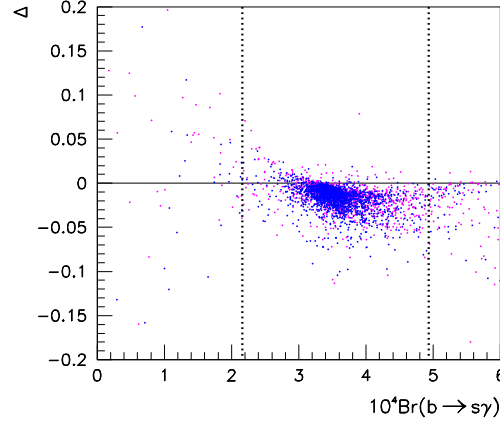


Figure 3: Relative difference for $B(\bar{B} \rightarrow s\gamma)$ between micromegas2.4 and superIso3.1. the vertical lines show the 3σ experimentally measured value.

- [3] G. Belanger, F. Boudjema, A. Pukhov, and A. Semenov, “micrOMEGAs: Version 1.3” *Comput. Phys. Commun.* **174** (2006) 577–604, [arXiv:hep-ph/0405253 \[hep-ph\]](#).
- [4] G. Belanger, F. Boudjema, A. Pukhov, and A. Semenov, “MicrOMEGAs 2.0: A Program to calculate the relic density of dark matter in a generic model,” *Comput. Phys. Commun.* **176** (2007) 367–382, [arXiv:hep-ph/0607059 \[hep-ph\]](#).
- [5] G. Belanger, F. Boudjema, A. Pukhov, and A. Semenov, “Dark matter direct detection rate in a generic model with micrOMEGAs 2.2” *Comput. Phys. Commun.* **180** (2009) 747–767, [arXiv:0803.2360 \[hep-ph\]](#).
- [6] G. Belanger, F. Boudjema, P. Brun, A. Pukhov, S. Rosier-Lees, P. Salati, and A. Semenov, “Indirect search for dark matter with micrOMEGAs2.4” *Comput. Phys. Commun.* **182** (2011) 842–856, [arXiv:1004.1092 \[hep-ph\]](#).
- [7] G. Belanger, F. Boudjema, A. Pukhov, and A. Semenov, “micrOMEGAs.3: A program for calculating dark matter observables,” *Comput. Phys. Commun.* **185** (2014) 960–985, [arXiv:1305.0237 \[hep-ph\]](#).
- [8] G. Bélanger, F. Boudjema, A. Pukhov, and A. Semenov, “micrOMEGAs4.1: two dark matter candidates,” *Comput. Phys. Commun.* **192** (2015) 322–329, [arXiv:1407.6129 \[hep-ph\]](#).
- [9] D. Barducci, G. Belanger, J. Bernon, F. Boudjema, J. Da Silva, S. Kraml, U. Laa, and A. Pukhov, “Collider limits on new physics within micrOMEGAs4.3” *Comput. Phys. Commun.* **222** (2018) 327–338, [arXiv:1606.03834 \[hep-ph\]](#).
- [10] G. Bélanger, F. Boudjema, A. Goudelis, A. Pukhov, and B. Zaldivar, “micrOMEGAs5.0 : Freeze-in,” *Comput. Phys. Commun.* **231** (2018) 173–186, [arXiv:1801.03509 \[hep-ph\]](#).

- [11] U. Ellwanger and C. Hugonie, “NMSPEC: A Fortran code for the sparticle and Higgs masses in the NMSSM with GUT scale boundary conditions,” *Comput. Phys. Commun.* **177** (2007) 399–407, [arXiv:hep-ph/0612134 \[hep-ph\]](#).
- [12] G. Belanger, F. Boudjema, C. Hugonie, A. Pukhov, and A. Semenov, “Relic density of dark matter in the NMSSM,” *JCAP* **0509** (2005) 001, [arXiv:hep-ph/0505142 \[hep-ph\]](#).
- [13] J. S. Lee, A. Pilaftsis, M. Carena, S. Y. Choi, M. Drees, J. R. Ellis, and C. E. M. Wagner, “CPsuperH: A Computational tool for Higgs phenomenology in the minimal supersymmetric standard model with explicit CP violation,” *Comput. Phys. Commun.* **156** (2004) 283–317, [arXiv:hep-ph/0307377 \[hep-ph\]](#).
- [14] G. Belanger, F. Boudjema, S. Kraml, A. Pukhov, and A. Semenov, “Relic density of neutralino dark matter in the MSSM with CP violation,” *Phys. Rev.* **D73** (2006) 115007, [arXiv:hep-ph/0604150 \[hep-ph\]](#).
- [15] R. Barbieri, L. J. Hall, and V. S. Rychkov, “Improved naturalness with a heavy Higgs: An Alternative road to LHC physics,” *Phys. Rev.* **D74** (2006) 015007, [arXiv:hep-ph/0603188 \[hep-ph\]](#).
- [16] A. Belyaev, C.-R. Chen, K. Tobe, and C. P. Yuan, “Phenomenology of littlest Higgs model with T^- parity: including effects of T^- odd fermions,” *Phys. Rev.* **D74** (2006) 115020, [arXiv:hep-ph/0609179 \[hep-ph\]](#).
- [17] G. Belanger *et al.*, “Leptoquark manoeuvres in the dark: a simultaneous solution of the dark matter problem and the $R_{D^{(*)}}$ anomalies,” *JHEP* **02** (2022) 042, [arXiv:2111.08027 \[hep-ph\]](#).
- [18] J. McDonald, “Thermally generated gauge singlet scalars as selfinteracting dark matter,” *Phys. Rev. Lett.* **88** (2002) 091304, [arXiv:hep-ph/0106249 \[hep-ph\]](#).
- [19] G. Alguero, G. Belanger, S. Kraml, and A. Pukhov, “Co-scattering in micrOMEGAs: A case study for the singlet-triplet dark matter model,” *SciPost Phys.* **13** (2022) 124, [arXiv:2207.10536 \[hep-ph\]](#).
- [20] G. Belanger, K. Kannike, A. Pukhov, and M. Raidal, “Impact of semi-annihilations on dark matter phenomenology - an example of Z_N symmetric scalar dark matter,” *JCAP* **1204** (2012) 010, [arXiv:1202.2962 \[hep-ph\]](#).
- [21] G. Bélanger, K. Kannike, A. Pukhov, and M. Raidal, “Minimal semi-annihilating Z_N scalar dark matter,” *JCAP* **1406** (2014) 021, [arXiv:1403.4960 \[hep-ph\]](#).
- [22] G. Bélanger, A. Pukhov, C. E. Yaguna, and O. Zapata, “The Z_5 model of two-component dark matter,” *JHEP* **09** (2020) 030, [arXiv:2006.14922 \[hep-ph\]](#).
- [23] G. Belanger, A. Pukhov, and G. Servant, “Dirac Neutrino Dark Matter,” *JCAP* **0801** (2008) 009, [arXiv:0706.0526 \[hep-ph\]](#).

- [24] J. Da Silva, *Supersymmetric Dark Matter candidates in light of constraints from collider and astroparticle observables*. PhD thesis, Annecy, LAPTH, 2013.
[arXiv:1312.0257 \[hep-ph\]](#).
<https://inspirehep.net/record/1266989/files/arXiv:1312.0257.pdf>.
- [25] G. Belanger, J. Da Silva, U. Laa, and A. Pukhov, “Probing U(1) extensions of the MSSM at the LHC Run I and in dark matter searches,” *JHEP* **09** (2015) 151, [arXiv:1505.06243 \[hep-ph\]](#).
- [26] A. Djouadi, J.-L. Kneur, and G. Moultaka, “SuSpect: A Fortran code for the supersymmetric and Higgs particle spectrum in the MSSM,” *Comput. Phys. Commun.* **176** (2007) 426–455, [arXiv:hep-ph/0211331](#).
- [27] <https://www.lupm.univ-montp2.fr/users/nmssm/>.
- [28] U. Ellwanger and C. Hugonie, “NMHDECAY 2.0: An Updated program for sparticle masses, Higgs masses, couplings and decay widths in the NMSSM,” *Comput. Phys. Commun.* **175** (2006) 290–303, [arXiv:hep-ph/0508022 \[hep-ph\]](#).
- [29] <http://www.hep.man.ac.uk/u/jslee/CPsuperH.html>.
- [30] T. Hahn and M. Perez-Victoria, “Automatized one loop calculations in four-dimensions and D-dimensions,” *Comput. Phys. Commun.* **118** (1999) 153–165, [arXiv:hep-ph/9807565](#).
- [31] A. Semenov, “LanHEP — A package for automatic generation of Feynman rules from the Lagrangian. Version 3.2,” *Comput. Phys. Commun.* **201** (2016) 167–170, [arXiv:1412.5016 \[physics.comp-ph\]](#).
- [32] S. Yellin, “Extending the optimum interval method,” 2007.
<https://arxiv.org/abs/0709.2701>.
- [33] J. Bernon and B. Dumont, “Lilith: a tool for constraining new physics from Higgs measurements,” *Eur. Phys. J.* **C75** no. 9, (2015) 440, [arXiv:1502.04138 \[hep-ph\]](#).
- [34] S. Kraml, T. Q. Loc, D. T. Nhung, and L. D. Ninh, “Constraining new physics from Higgs measurements with Lilith: update to LHC Run 2 results,” *SciPost Phys.* **7** no. 4, (2019) 052, [arXiv:1908.03952 \[hep-ph\]](#).
- [35] M. Bertrand, S. Kraml, T. Q. Loc, D. T. Nhung, and L. D. Ninh, “Constraining new physics from Higgs measurements with Lilith-2,” in *Tools for High Energy Physics and Cosmology*. 12, 2020. [arXiv:2012.11408 \[hep-ph\]](#).
- [36] A. Albert, B. Anderson, K. Bechtol, A. Drlica-Wagner, M. Meyer, M. Sánchez-Conde, L. Strigari, M. Wood, T. M. C. Abbott, F. B. Abdalla, A. Benoit-Lévy, G. M. Bernstein, R. A. Bernstein, E. Bertin, D. Brooks, D. L. Burke, A. C. Rosell, M. C. Kind, J. Carretero, M. Crocce, C. E. Cunha, C. B. D’Andrea, L. N. da Costa, S. Desai, H. T. Diehl, J. P. Dietrich, P. Doel, T. F. Eifler, A. E. Evrard, A. F. Neto, D. A. Finley, B. Flaugh, P. Fosalba, J. Frieman, D. W. Gerdes, D. A. Goldstein, D. Gruen, R. A. Gruendl,

- K. Honscheid, D. J. James, S. Kent, K. Kuehn, N. Kuropatkin, O. Lahav, T. S. Li, M. A. G. Maia, M. March, J. L. Marshall, P. Martini, C. J. Miller, R. Miquel, E. Neilsen, B. Nord, R. Ogando, A. A. Plazas, K. Reil, A. K. Romer, E. S. Rykoff, E. Sanchez, B. Santiago, M. Schubnell, I. Sevilla-Noarbe, R. C. Smith, M. Soares-Santos, F. Sobreira, E. Suchyta, M. E. C. Swanson, G. Tarle, V. Vikram, A. R. Walker, and R. H. Wechsler, “Searching for dark matter annihilation in recently discovered milky way satellites with fermi-lat,” *The Astrophysical Journal* **834** no. 2, (Jan., 2017) 110, [arXiv:1611.03184](#). <http://dx.doi.org/10.3847/1538-4357/834/2/110>.
- [37] V. Bonnivard, C. Combet, M. Daniel, S. Funk, A. Geringer-Sameth, J. A. Hinton, D. Maurin, J. I. Read, S. Sarkar, M. G. Walker, and M. I. Wilkinson, “Dark matter annihilation and decay in dwarf spheroidal galaxies: the classical and ultrafaint dsphs,” *Monthly Notices of the Royal Astronomical Society* **453** no. 1, (Aug., 2015) 849–867, [arXiv:1504.02048](#). <http://dx.doi.org/10.1093/mnras/stv1601>.
- [38] A. Alvarez, F. Calore, A. Genina, J. Read, P. D. Serpico, and B. Zaldivar, “Dark matter constraints from dwarf galaxies with data-driven j-factors,” *Journal of Cosmology and Astroparticle Physics* **2020** no. 09, (Sept., 2020) 004–004, [arXiv:2002.01229](#). <http://dx.doi.org/10.1088/1475-7516/2020/09/004>.
- [39] B. C. Allanach, “SOFTSUSY: a program for calculating supersymmetric spectra,” *Comput. Phys. Commun.* **143** (2002) 305–331, [arXiv:hep-ph/0104145](#) [hep-ph].
- [40] W. Porod and F. Staub, “SPHeno 3.1: Extensions including flavour, CP-phases and models beyond the MSSM,” *Comput. Phys. Commun.* **183** (2012) 2458–2469, [arXiv:1104.1573](#) [hep-ph].
- [41] S. Kraml, S. Kulkarni, U. Laa, A. Lessa, W. Magerl, D. Proschofsky-Spindler, and W. Waltenberger, “SModelS: a tool for interpreting simplified-model results from the LHC and its application to supersymmetry,” *Eur. Phys. J. C* **74** (2014) 2868, [arXiv:1312.4175](#) [hep-ph].
- [42] F. Ambrogio, S. Kraml, S. Kulkarni, U. Laa, A. Lessa, V. Magerl, J. Sonneveld, M. Traub, and W. Waltenberger, “SModelS v1.1 user manual: Improving simplified model constraints with efficiency maps,” *Comput. Phys. Commun.* **227** (2018) 72–98, [arXiv:1701.06586](#) [hep-ph].
- [43] F. Ambrogio *et al.*, “SModelS v1.2: long-lived particles, combination of signal regions, and other novelties,” *Comput. Phys. Commun.* **251** (2020) 106848, [arXiv:1811.10624](#) [hep-ph].
- [44] G. Alguero, J. Heisig, C. K. Khosa, S. Kraml, S. Kulkarni, A. Lessa, H. Reyes-González, W. Waltenberger, and A. Wongel, “Constraining new physics with SModelS version 2,” *JHEP* **08** (2022) 068, [arXiv:2112.00769](#) [hep-ph].
- [45] M. Mahdi Altakach, S. Kraml, A. Lessa, S. Narasimha, T. Pascal, and W. Waltenberger, “SModelS v2.3: Enabling global likelihood analyses,” *SciPost Phys.* **15** no. 5, (2023) 185, [arXiv:2306.17676](#) [hep-ph].

- [46] F. Mahmoudi, “SuperIso v2.3: A Program for calculating flavor physics observables in Supersymmetry,” *Comput. Phys. Commun.* **180** (2009) 1579–1613, [arXiv:0808.3144 \[hep-ph\]](#).
- [47] P. Bechtle, O. Brein, S. Heinemeyer, O. Stål, T. Stefaniak, *et al.*, “HiggsBounds – 4: Improved Tests of Extended Higgs Sectors against Exclusion Bounds from LEP, the Tevatron and the LHC,” *Eur. Phys. J.* **C74** (2014) 2693, [arXiv:1311.0055 \[hep-ph\]](#).
- [48] P. Bechtle, D. Dercks, S. Heinemeyer, T. Klingl, T. Stefaniak, G. Weiglein, and J. Wittbrodt, “HiggsBounds-5: Testing Higgs Sectors in the LHC 13 TeV Era,” [arXiv:2006.06007 \[hep-ph\]](#).
- [49] P. Bechtle, S. Heinemeyer, O. Stål, T. Stefaniak, and G. Weiglein, “*HiggsSignals*: Confronting arbitrary Higgs sectors with measurements at the Tevatron and the LHC,” *Eur. Phys. J.* **C74** no. 2, (2014) 2711, [arXiv:1305.1933 \[hep-ph\]](#).
- [50] P. Bechtle, S. Heinemeyer, T. Klingl, T. Stefaniak, G. Weiglein, and J. Wittbrodt, “HiggsSignals-2: Probing new physics with precision Higgs measurements in the LHC 13 TeV era,” *Eur. Phys. J. C* **81** no. 2, (2021) 145, [arXiv:2012.09197 \[hep-ph\]](#).
- [51] A. Belyaev, N. D. Christensen, and A. Pukhov, “CalcHEP 3.4 for collider physics within and beyond the Standard Model,” *Comput. Phys. Commun.* **184** (2013) 1729–1769, [arXiv:1207.6082 \[hep-ph\]](#).
- [52] M. Drees, F. Hajkarim, and E. R. Schmitz, “The Effects of QCD Equation of State on the Relic Density of WIMP Dark Matter,” *JCAP* **06** (2015) 025, [arXiv:1503.03513 \[hep-ph\]](#).
- [53] M. Laine and M. Meyer, “Standard Model thermodynamics across the electroweak crossover,” *JCAP* **07** (2015) 035, [arXiv:1503.04935 \[hep-ph\]](#).
- [54] M. Hindmarsh and O. Philipsen, “WIMP dark matter and the QCD equation of state,” *Phys. Rev.* **D71** (2005) 087302, [arXiv:hep-ph/0501232 \[hep-ph\]](#).
- [55] **Particle Data Group** Collaboration, K. Nakamura *et al.*, “Review of particle physics,” *J. Phys.* **G37** (2010) 075021.
- [56] P. Z. Skands, B. Allanach, H. Baer, C. Balazs, G. Belanger, *et al.*, “SUSY Les Houches accord: Interfacing SUSY spectrum calculators, decay packages, and event generators,” *JHEP* **0407** (2004) 036, [arXiv:hep-ph/0311123 \[hep-ph\]](#).
- [57] P. Agrawal, A. Parikh, and M. Reece, “Systematizing the Effective Theory of Self-Interacting Dark Matter,” *JHEP* **10** (2020) 191, [arXiv:2003.00021 \[hep-ph\]](#).
- [58] R. Iengo, “Sommerfeld enhancement: general results from field theory diagrams,” *Journal of High Energy Physics* **2009** no. 05, (May, 2009) 024–024. <https://doi.org/10.1088%2F1126-6708%2F2009%2F05%2F024>.

- [59] T. R. Slatyer, “The Sommerfeld enhancement for dark matter with an excited state,” *JCAP* **02** (2010) 028, [arXiv:0910.5713 \[hep-ph\]](#).
- [60] G. Bélanger, N. Bernal, and A. Pukhov, “Strongly interacting singlet scalar dark matter during reheating,” [arXiv:2603.05590 \[hep-ph\]](#).
- [61] G. B. Gelmini and P. Gondolo, “Neutralino with the right cold dark matter abundance in (almost) any supersymmetric model,” *Phys. Rev. D* **74** (2006) 023510, [arXiv:hep-ph/0602230](#).
- [62] G. Alguero, G. Belanger, F. Boudjema, S. Chakraborti, A. Goudelis, S. Kraml, A. Mjallal, and A. Pukhov, “micrOMEGAs 6.0: N-component dark matter,” *Comput. Phys. Commun.* **299** (2024) 109133, [arXiv:2312.14894 \[hep-ph\]](#).
- [63] **Particle Data Group** Collaboration, J. Beringer *et al.*, “Review of Particle Physics (RPP),” *Phys. Rev.* **D86** (2012) 010001.
- [64] M. Drees and M. Nojiri, “Neutralino - nucleon scattering revisited,” *Phys. Rev.* **D48** (1993) 3483–3501, [arXiv:hep-ph/9307208 \[hep-ph\]](#).
- [65] J. Hisano, R. Nagai, and N. Nagata, “Effective Theories for Dark Matter Nucleon Scattering,” *JHEP* **05** (2015) 037, [arXiv:1502.02244 \[hep-ph\]](#).
- [66] V. A. Bednyakov and F. Simkovic, “Nuclear spin structure in dark matter search: The Finit momentum transfer limit,” *Phys. Part. Nucl.* **37** (2006) S106–S128, [arXiv:hep-ph/0608097 \[hep-ph\]](#).
- [67] A. L. Fitzpatrick, W. Haxton, E. Katz, N. Lubbers, and Y. Xu, “The Effective Field Theory of Dark Matter Direct Detection,” *JCAP* **1302** (2013) 004, [arXiv:1203.3542 \[hep-ph\]](#).
- [68] P. Klos, J. Menéndez, D. Gazit, and A. Schwenk, “Large-scale nuclear structure calculations for spin-dependent WIMP scattering with chiral effective field theory currents,” *Phys. Rev.* **D88** no. 8, (2013) 083516, [arXiv:1304.7684 \[nucl-th\]](#). [Erratum: *Phys. Rev.* **D89**, no. 2, 029901 (2014)].
- [69] N. W. Evans, C. A. J. O’Hare, and C. McCabe, “SHM⁺⁺: A Refinement of the Standard Halo Model for Dark Matter Searches in Light of the Gaia Sausage,” [arXiv:1810.11468 \[astro-ph.GA\]](#).
- [70] V. Belokurov, D. Erkal, N. W. Evans, S. E. Koposov, and A. J. Deason, “Co-formation of the disc and the stellar halo,” *Monthly Notices of the Royal Astronomical Society* **478** no. 1, (Jun, 2018) 611. <http://dx.doi.org/10.1093/mnras/sty982>.
- [71] G. Myeong, N. Evans, V. Belokurov, J. Sanders, and S. Koposov, “The Sausage Globular Clusters,” *Astrophys. J. Lett.* **863** no. 2, (2018) L28, [arXiv:1805.00453 \[astro-ph.GA\]](#).
- [72] J. Aalbers *et al.*, “Dark matter search results from 4.2 tonne-years of exposure of the lux-zeplin (lz) experiment,” *Physical Review Letters* **135** no. 1, (July, 2025) , [arXiv:2410.17036 \[hep-ex\]](#). <http://dx.doi.org/10.1103/4dyc-z8zf>.

- [73] **XENON** Collaboration, E. Aprile *et al.*, “Dark Matter Search Results from a One Tonne $\times$ Year Exposure of XENON1T,” [arXiv:1805.12562 \[astro-ph.CO\]](#).
- [74] **DarkSide** Collaboration, P. Agnes *et al.*, “Low-Mass Dark Matter Search with the DarkSide-50 Experiment,” *Phys. Rev. Lett.* **121** no. 8, (2018) 081307, [arXiv:1802.06994 \[astro-ph.HE\]](#).
- [75] **PICO** Collaboration, C. Amole *et al.*, “Dark Matter Search Results from the Complete Exposure of the PICO-60 C₃F₈ Bubble Chamber,” *Phys. Rev.* **D100** no. 2, (2019) 022001, [arXiv:1902.04031 \[astro-ph.CO\]](#).
- [76] **CRESST** Collaboration, A. H. Abdelhameed *et al.*, “First results from the CRESST-III low-mass dark matter program,” [arXiv:1904.00498 \[astro-ph.CO\]](#).
- [77] **XENON** Collaboration, E. Aprile *et al.*, “First Search for Light Dark Matter in the Neutrino Fog with XENONnT,” *Phys. Rev. Lett.* **134** no. 11, (2025) 111802, [arXiv:2409.17868 \[hep-ex\]](#).
- [78] **PandaX** Collaboration, M. Zhang *et al.*, “Search for Light Dark Matter with 259 Days of Data in PandaX-4T,” *Phys. Rev. Lett.* **135** no. 21, (2025) 211001, [arXiv:2507.11930 \[hep-ex\]](#). [Erratum: *Phys.Rev.Lett.* 136, 069901 (2026)].
- [79] **PandaX-4T** Collaboration, Y. Meng *et al.*, “Dark Matter Search Results from the PandaX-4T Commissioning Run,” *Phys. Rev. Lett.* **127** no. 26, (2021) 261802, [arXiv:2107.13438 \[hep-ex\]](#).
- [80] **XENON** Collaboration, E. Aprile *et al.*, “Constraining the spin-dependent WIMP-nucleon cross sections with XENON1T,” *Phys. Rev. Lett.* **122** no. 14, (2019) 141301, [arXiv:1902.03234 \[astro-ph.CO\]](#).
- [81] L. Baudis, “DARWIN/XLZD: A future xenon observatory for dark matter and other rare interactions,” *Nucl. Phys. B* **1003** (2024) 116473, [arXiv:2404.19524 \[astro-ph.IM\]](#).
- [82] C. A. J. O’Hare, “New Definition of the Neutrino Floor for Direct Dark Matter Searches,” *Phys. Rev. Lett.* **127** no. 25, (2021) 251802, [arXiv:2109.03116 \[hep-ph\]](#).
- [83] G. Belanger, A. Mjallal, and A. Pukhov, “Recasting direct detection limits within micrOMEGAs and implication for non-standard Dark Matter scenarios,” [arXiv:2003.08621 \[hep-ph\]](#).
- [84] A. Coogan, L. Morrison, and S. Profumo, “Hazma: a python toolkit for studying indirect detection of sub-gev dark matter,” *Journal of Cosmology and Astroparticle Physics* **2020** no. 01, (Jan., 2020) 056–056, [arXiv:1907.11846](#). <http://dx.doi.org/10.1088/1475-7516/2020/01/056>.
- [85] S. Amoroso, S. Caron, A. Jueid, R. Ruiz de Austri, and P. Skands, “Estimating QCD uncertainties in Monte Carlo event generators for gamma-ray dark matter searches,” *JCAP* **05** (2019) 007, [arXiv:1812.07424 \[hep-ph\]](#).

- [86] A. Jueid, J. Kip, R. R. de Austri, and P. Skands, “Impact of QCD uncertainties on antiproton spectra from dark-matter annihilation,” *JCAP* **04** (2023) 068, [arXiv:2202.11546 \[hep-ph\]](#).
- [87] P. Ciafaloni, D. Comelli, A. Riotto, F. Sala, A. Strumia, and A. Urbano, “Weak corrections are relevant for dark matter indirect detection,” *Journal of Cosmology and Astroparticle Physics* **2011** no. 03, (Mar., 2011) 019–019, [arXiv:1009.0224](#). <http://dx.doi.org/10.1088/1475-7516/2011/03/019>.
- [88] M. Cirelli, G. Corcella, A. Hektor, G. Hütsi, M. Kadastik, P. Panci, M. Raidal, F. Sala, and A. Strumia, “Pppc 4 dm id: a poor particle physicist cookbook for dark matter indirect detection,” *Journal of Cosmology and Astroparticle Physics* **2011** no. 03, (Mar., 2011) 051–051, [arXiv:1012.4515](#). <http://dx.doi.org/10.1088/1475-7516/2011/03/051>.
- [89] C. Arina, M. Di Mauro, N. Fornengo, J. Heisig, A. Jueid, and R. R. de Austri, “CosmiXs: Cosmic messenger spectra for indirect dark matter searches,” [arXiv:2312.01153 \[astro-ph.HE\]](#).
- [90] H. Zhao, “Analytical models for galactic nuclei,” *Mon. Not. Roy. Astron. Soc.* **278** (1996) 488–496, [arXiv:astro-ph/9509122 \[astro-ph\]](#).
- [91] H.-N. Lin and X. Li, “The Dark Matter Profiles in the Milky Way,” *Mon. Not. Roy. Astron. Soc.* **487** no. 4, (2019) 5679–5684, [arXiv:1906.08419 \[astro-ph.GA\]](#).
- [92] Y. Jiao, F. Hammer, H. Wang, J. Wang, P. Amram, L. Chemin, and Y. Yang, “Detection of the Keplerian decline in the Milky Way rotation curve,” *Astron. Astrophys.* **678** (2023) A208, [arXiv:2309.00048 \[astro-ph.GA\]](#).
- [93] J. Lavalle, J. Pochon, P. Salati, and R. Taillet, “Clumpiness of dark matter and positron annihilation signal: computing the odds of the galactic lottery,” *Astron. Astrophys.* **462** (2007) 827–848, [arXiv:astro-ph/0603796 \[astro-ph\]](#).
- [94] F. Boudjema, A. Semenov, and D. Temes, “Self-annihilation of the neutralino dark matter into two photons or a Z and a photon in the MSSM,” *Phys. Rev.* **D72** (2005) 055024, [arXiv:hep-ph/0507127 \[hep-ph\]](#).
- [95] G. Chalons and A. Semenov, “Loop-induced photon spectral lines from neutralino annihilation in the NMSSM,” *JHEP* **12** (2011) 055, [arXiv:1110.2064 \[hep-ph\]](#).
- [96] J. Hisano, S. Matsumoto, M. M. Nojiri, and O. Saito, “Non-perturbative effect on dark matter annihilation and gamma ray signature from galactic center,” *Phys. Rev.* **D71** (2005) 063528, [arXiv:hep-ph/0412403 \[hep-ph\]](#).
- [97] A. Hryczuk and R. Iengo, “The one-loop and Sommerfeld electroweak corrections to the Wino dark matter annihilation,” *JHEP* **01** (2012) 163, [arXiv:1111.2916 \[hep-ph\]](#). [Erratum: JHEP06,137(2012)].

- [98] T. R. Slatyer, “Indirect dark matter signatures in the cosmic dark ages. i. generalizing the bound on s-wave dark matter annihilation from planck results,” *Physical Review D* **93** no. 2, (Jan., 2016) , [ArXiv:1506.03811](#).
<http://dx.doi.org/10.1103/PhysRevD.93.023527>.
- [99] M. Blennow, J. Edsjo, and T. Ohlsson, “Neutrinos from WIMP annihilations using a full three-flavor Monte Carlo,” *JCAP* **0801** (2008) 021, [arXiv:0709.3898 \[hep-ph\]](#).
- [100] P. Baratella, M. Cirelli, A. Hektor, J. Pata, M. Piibeleht, and A. Strumia, “PPPC 4 DM ν : a Poor Particle Physicist Cookbook for Neutrinos from Dark Matter annihilations in the Sun,” *JCAP* **1403** (2014) 053, [arXiv:1312.6408 \[hep-ph\]](#).
- [101] M. Cirelli, N. Fornengo, T. Montaruli, I. A. Sokalski, A. Strumia, and F. Vissani, “Spectra of neutrinos from dark matter annihilations,” *Nucl. Phys. B* **727** (2005) 99–138, [arXiv:hep-ph/0506298 \[hep-ph\]](#). [Erratum: Nucl. Phys.B790,338(2008)].
- [102] A. E. Erkoca, M. H. Reno, and I. Sarcevic, “Muon Fluxes From Dark Matter Annihilation,” *Phys. Rev. D* **80** (2009) 043514, [arXiv:0906.4364 \[hep-ph\]](#).
- [103] D. Chirkin and W. Rhode, “Muon Monte Carlo: A High-precision tool for muon propagation through matter,” [arXiv:hep-ph/0407075 \[hep-ph\]](#).
- [104] M. Honda, M. Sajjad Athar, T. Kajita, K. Kasahara, and S. Midorikawa, “Atmospheric neutrino flux calculation using the NRLMSISE-00 atmospheric model,” *Phys. Rev. D* **92** no. 2, (2015) 023004, [arXiv:1502.03916 \[astro-ph.HE\]](#).
- [105] G. Bélanger, J. Da Silva, T. Perrillat-Bottonet, and A. Pukhov, “Limits on dark matter proton scattering from neutrino telescopes using micrOMEGAs,” *JCAP* **1512** no. 12, (2015) 036, [arXiv:1507.07987 \[hep-ph\]](#).
- [106] T. Venumadhav, F.-Y. Cyr-Racine, K. N. Abazajian, and C. M. Hirata, “Sterile neutrino dark matter: Weak interactions in the strong coupling epoch,” *Physical Review D* **94** no. 4, (Aug., 2016) , [1507.06655](#).
<http://dx.doi.org/10.1103/PhysRevD.94.043515>.
- [107] P. Bechtle, O. Brein, S. Heinemeyer, G. Weiglein, and K. E. Williams, “HiggsBounds 2.0.0: Confronting Neutral and Charged Higgs Sector Predictions with Exclusion Bounds from LEP and the Tevatron,” *Comput. Phys. Commun.* **182** (2011) 2605–2631, [arXiv:1102.1898 \[hep-ph\]](#).
- [108] G. Belanger, F. Boudjema, and A. Pukhov, “[micrOMEGAs : a code for the calculation of Dark Matter properties in generic models of particle interaction](#),” in *The Dark Secrets of the Terascale*, pp. 739–790. 2013. [arXiv:1402.0787 \[hep-ph\]](#).
- [109] A. Djouadi, J. Kalinowski, and M. Spira, “HDECAY: A Program for Higgs boson decays in the standard model and its supersymmetric extension,” *Comput. Phys. Commun.* **108** (1998) 56–74, [arXiv:hep-ph/9704448 \[hep-ph\]](#).

- [110] M. M. Altakach, S. Kraml, A. Lessa, S. Narasimha, T. Pascal, T. Reymermier, and W. Waltenberger, “Global LHC constraints on electroweak-inos with SModelS v2.3” *SciPost Phys.* **16** no. 4, (2024) 101, [arXiv:2312.16635 \[hep-ph\]](#).
- [111] P. M. Nadolsky, H.-L. Lai, Q.-H. Cao, J. Huston, J. Pumplin, D. Stump, W.-K. Tung, and C. P. Yuan, “Implications of CTEQ global analysis for collider observables,” *Phys. Rev. D* **78** (2008) 013004, [arXiv:0802.0007 \[hep-ph\]](#).
- [112] G. Alguero, S. Kraml, and W. Waltenberger, “A SModelS interface for pyhf likelihoods,” *Comput. Phys. Commun.* **264** (2021) 107909, [arXiv:2009.01809 \[hep-ph\]](#).
- [113] A. Freitas, “Higher-order electroweak corrections to the partial widths and branching ratios of the Z boson,” *JHEP* **04** (2014) 070, [arXiv:1401.2447 \[hep-ph\]](#).
- [114] CMS Collaboration, A. M. Sirunyan *et al.*, “Search for resonant and nonresonant new phenomena in high-mass dilepton final states at $\sqrt{s} = 13$ TeV,” *JHEP* **07** (2021) 208, [arXiv:2103.02708 \[hep-ex\]](#).
- [115] OPAL Collaboration, G. Abbiendi *et al.*, “Search for chargino and neutralino production at $\sqrt{s} = 192 - 209$ GeV at LEP,” *Eur. Phys. J.* **C35** (2004) 1–20, [arXiv:hep-ex/0401026 \[hep-ex\]](#).
- [116] B. C. Allanach *et al.*, “SUSY Les Houches Accord 2,” *Comput. Phys. Commun.* **180** (2009) 8–25, [arXiv:0801.0045 \[hep-ph\]](#).
- [117] M. Carena, M. Quiros, and C. E. M. Wagner, “Effective potential methods and the Higgs mass spectrum in the MSSM,” *Nucl. Phys.* **B461** (1996) 407–436, [arXiv:hep-ph/9508343 \[hep-ph\]](#).
- [118] C. Bobeth, T. Ewerth, F. Kruger, and J. Urban, “Analysis of neutral Higgs boson contributions to the decays $\bar{B}(s) \rightarrow \ell^+ \ell^-$ and $\bar{B} \rightarrow K \ell^+ \ell^-$,” *Phys. Rev.* **D64** (2001) 074014, [arXiv:hep-ph/0104284 \[hep-ph\]](#).
- [119] F. Domingo and U. Ellwanger, “Updated Constraints from B Physics on the MSSM and the NMSSM,” *JHEP* **12** (2007) 090, [arXiv:0710.3714 \[hep-ph\]](#).
- [120] J. S. Lee, M. Carena, J. Ellis, A. Pilaftsis, and C. E. M. Wagner, “CPsuperH2.0: an Improved Computational Tool for Higgs Phenomenology in the MSSM with Explicit CP Violation,” *Comput. Phys. Commun.* **180** (2009) 312–331, [arXiv:0712.2360 \[hep-ph\]](#).
- [121] A. Buckley, J. Ferrando, S. Lloyd, K. Nordström, B. Page, M. Rüfenacht, M. Schönherr, and G. Watt, “LHAPDF6: parton density access in the LHC precision era,” *Eur. Phys. J. C* **75** (2015) 132, [arXiv:1412.7420 \[hep-ph\]](#).
- [122] A. Alloul, N. D. Christensen, C. Degrande, C. Duhr, and B. Fuks, “Feynrules 2.0 — a complete toolbox for tree-level phenomenology,” *Computer Physics Communications* **185** no. 8, (Aug., 2014) 2250–2300. <http://dx.doi.org/10.1016/j.cpc.2014.04.012>.

- [123] F. Staub, “Sarah 3.2: Dirac gauginos, ufo output, and more,” *Computer Physics Communications* **184** no. 7, (July, 2013) 1792–1809.
<http://dx.doi.org/10.1016/j.cpc.2013.02.019>.
- [124] A. Djouadi, “The Anatomy of electro-weak symmetry breaking. I: The Higgs boson in the standard model,” *Phys. Rept.* **457** (2008) 1–216, [arXiv:hep-ph/0503172](#).
- [125] **Particle Data Group** Collaboration, P. A. Zyla *et al.*, “Review of Particle Physics,” *PTEP* **2020** no. 8, (2020) 083C01.
- [126] P. Baikov, K. Chetyrkin, and J. Kühn, “Five-loop running of the qcd coupling constant,” *Physical Review Letters* **118** no. 8, (Feb., 2017) , [arXiv:1606.08659](#).
<http://dx.doi.org/10.1103/PhysRevLett.118.082002>.
- [127] K. Chetyrkin, J. Kühn, and C. Sturm, “Qcd decoupling at four loops,” *Nuclear Physics B* **744** no. 1–2, (June, 2006) 121–135, [arXiv:hep-ph/0512060](#).
<http://dx.doi.org/10.1016/j.nuclphysb.2006.03.020>.
- [128] F. Mahmoudi *et al.*, “Flavour Les Houches Accord: Interfacing Flavour related Codes,” *Comput. Phys. Commun.* **183** (2012) 285–298, [arXiv:1008.0762 \[hep-ph\]](#).
- [129] W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery, “Numerical Recipes: The Art of Scientific Computing.”
- [130] G. J. Feldman and R. D. Cousins, “A Unified approach to the classical statistical analysis of small signals,” *Phys. Rev.* **D57** (1998) 3873–3889,
[arXiv:physics/9711021 \[physics.data-an\]](#).
- [131] M. Misiak *et al.*, “Estimate of $\mathcal{B}(\bar{B} \rightarrow X_s \gamma)$ at $O(\alpha_s^2)$,” *Phys. Rev. Lett.* **98** (2007) 022002, [arXiv:hep-ph/0609232 \[hep-ph\]](#).
- [132] M. Misiak and M. Steinhauser, “NNLO QCD corrections to the anti-B $\rightarrow$ $X(s)$ gamma matrix elements using interpolation in $m(c)$,” *Nucl. Phys.* **B764** (2007) 62–82, [arXiv:hep-ph/0609241 \[hep-ph\]](#).
- [133] P. Gambino and P. Giordano, “Normalizing inclusive rare B decays,” *Phys. Lett.* **B669** (2008) 69–73, [arXiv:0805.0271 \[hep-ph\]](#).
- [134] **Particle Data Group** Collaboration, W. M. Yao *et al.*, “Review of Particle Physics,” *J. Phys.* **G33** (2006) 1–1232.